

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

LABORATORY WORKBOOK & 100 SOLVED PRACTICE PROBLEMS

Complete Activity Sheets with C Source Code & Verified Command Prompt Execution

STUDENT INFORMATION

Student Name: **M ASHWIN** Roll Number: **CH.SC.U4CSE25225**
Department: Computer Science & Engineering Academic Year: 2026

LABORATORY MODULES & COVERAGE (100 PROBLEMS)

Module 1: Array Data Structure (Q1 - Q10) - verified
Module 2: Stack Data Structure (Q11 - Q20) - verified
Module 3: Queue Data Structure (Q21 - Q30) - verified
Module 4: Linked List Operations (Q31 - Q40) - verified
Module 5: Searching Algorithms (Q41 - Q50) - verified
Module 6: Sorting Algorithms (Q51 - Q60) - verified
Module 7: Binary Trees & BST Part 1 (Q61 - Q70) - verified
Module 8: Tree Traversal & Balancing Part 2 (Q71 - Q80) - verified
Module 9: Graph Representations & Algorithms (Q81 - Q90) - verified
Module 10: Hashing & Collision Resolution (Q91 - Q100) - verified

Rectified edition: every program compiled with gcc (C11) and executed against the platform test cases; console pages show the genuine reproduced runs.

DS	Session	Arrays	Question Information	Level 1	Challenge 21
----	---------	--------	----------------------	---------	--------------

Problem Description:

Convert Arabic numbers (1..1000) to Martian numerals using the value/symbol table: 1000=R, 900=BR, 500=G, 400=BG, 100=B, 90=ZB, 50=P, 40=ZP, 10=Z, 9=BZ, 5=W, 4=BW, 1=B (greedy, largest value first; symbols may be reused). Read numbers one per line until end of input and print each conversion on its own line.

Input Format:

A list of numbers in a data file, one number per line, up to 5 lines at a time (minimum 1 line). No number exceeds 1000 or is less than 1.

Output Format:

Print the output in separate lines converting the numbers from Arabic (1,2,3...10...500...1000) to Martian (B,BB,BBB...Z...G...R) numerals.

✓ Logical Test Cases

Test Case 1	Test Case 2
INPUT (STDIN) 26 67 186 408 500 EXPECTED OUTPUT ZZWB PZWBB BPZZZWB BGWBBB G	INPUT (STDIN) 161 627 816 401 501 EXPECTED OUTPUT BPZB GBZZWBB GBBBZWB BGB GB

✓ Mandatory Test Cases

Test Case 1	Test Case 2	Test Case 3
KEYWORD char buf[]	KEYWORD buf[i++]='R';	KEYWORD while(n>=10)

Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 18

Test Case 2
TOKEN COUNT 455

Test Case 3
NLOC 84

● Array1.c

```
1 #include <stdio.h>
2 #include <string.h>
3
4 /* Convert one Arabic number (1..1000) to Martian numerals into buf. */
5 void martian_conv(int n, char buf[]) {
6     int i = 0;
7     const int vals[] = {900, 500, 400, 100, 90, 50, 40, 10};
8     const char *sym[] = {"BR", "G", "BG", "B", "ZB", "P", "ZP", "Z"};
9
10    while (n >= 1000) { buf[i++]='R'; n -= 1000; }
11    while(n>=10) {
12        for (int j = 0; j < 8; j++) {
13            if (n >= vals[j]) {
14                for (int k = 0; sym[j][k]; k++) buf[i++] = sym[j][k];
15                n -= vals[j];
16                break;
17            }
18        }
19    }
20    if (n >= 9) { buf[i++] = 'B'; buf[i++] = 'Z'; n -= 9; }
21    while (n >= 5) { buf[i++] = 'W'; n -= 5; }
22    if (n >= 4) { buf[i++] = 'B'; buf[i++] = 'W'; n -= 4; }
23    while (n >= 1) { buf[i++] = 'B'; n -= 1; }
24    buf[i] = '\0';
25 }
26
27 int main(void) {
28     int n;
29     char buf[64];
30     while (scanf("%d", &n) == 1) {
31         martian_conv(n, buf);
32         printf("%s\n", buf);
33     }
34     return 0;
35 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
26
```

```
67
```

```
186
```

```
408
```

```
500
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Array1.exe
```

```
ZZWB
```

```
PZWBB
```

```
BPZZWB
```

```
BGWBBB
```

```
G
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
161
```

```
627
```

```
816
```

```
401
```

```
501
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Array1.exe
```

```
BPZB
```

```
GBZZWBB
```

```
GBBBZWB
```

```
BGB
```

```
GB
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Arrays	Question Information	Level 1	Challenge 22
----	---------	--------	----------------------	---------	--------------

Problem Description:

Given p rows and q columns, print a matrix of Y and 0 filled in alternating concentric rectangles: outermost rectangle Y, next 0, next Y, and so on. Tokens separated by spaces.

Input Format:

One line with the number of rows and columns of the matrix, values separated by space.

Output Format:

Print the pattern matrix in separate lines.

✓ Logical Test Cases

Test Case 1
INPUT (STDIN) 6 7
EXPECTED OUTPUT Y Y Y Y Y Y Y Y 0 0 0 0 0 Y Y 0 Y Y Y 0 Y Y 0 Y Y Y 0 Y Y 0 0 0 0 0 Y Y Y Y Y Y Y Y

Test Case 2
INPUT (STDIN) 5 8
EXPECTED OUTPUT Y Y Y Y Y Y Y Y Y 0 0 0 0 0 0 Y Y 0 Y Y Y Y 0 Y Y 0 0 0 0 0 0 Y Y Y Y Y Y Y Y Y

✓ Mandatory Test Cases

Test Case 1
KEYWORD while(top<=bottom && right>=left)

✓ Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 12

Test Case 2
TOKEN COUNT 290

Test Case 3
NLOC 52

● Array2.c

```
1  #include <stdio.h>
2
3  int main(void) {
4      int rows, cols;
5      if (scanf("%d %d", &rows, &cols) != 2) return 0;
6      static char arr[1000][1000];
7      int left = 0, right = cols - 1, top = 0, bottom = rows - 1;
8      char c = 'Y';
9      while(top<=bottom && right>=left) {
10         for (int i = left; i <= right; i++) arr[top][i] = c;
11         for (int i = top + 1; i <= bottom; i++) arr[i][right] = c;
12         if (top < bottom) for (int i = right - 1; i >= left; i--) arr[bottom][i] = c;
13         if (left < right) for (int i = bottom - 1; i > top; i--) arr[i][left] = c;
14         c = (c == 'Y') ? '0' : 'Y';
15         left++; right--; top++; bottom--;
16     }
17     for (int i = 0; i < rows; i++)
18         for (int j = 0; j < cols; j++)
19             printf("%c%c", arr[i][j], (j + 1 < cols) ? ' ' : '\n');
20     return 0;
21 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
6 7
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Array2.exe
Y Y Y Y Y Y Y
Y 0 0 0 0 0 Y
Y 0 Y Y Y 0 Y
Y 0 Y Y Y 0 Y
Y 0 0 0 0 0 Y
Y Y Y Y Y Y Y
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
5 8
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Array2.exe
Y Y Y Y Y Y Y
Y 0 0 0 0 0 0 Y
Y 0 Y Y Y Y 0 Y
Y 0 0 0 0 0 0 Y
Y Y Y Y Y Y Y
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```


DS	Session	Arrays	Question Information	Level 1	Challenge 23
----	---------	--------	----------------------	---------	--------------

Problem Description:
For T test cases, given N days of stock prices, print every (buy sell) day pair of a rising run as '(b s)' concatenated on one line; print 'No Profit' if no profitable interval exists.

Input Format:
First line contains number of test cases T. First line of each test case contains an integer N denoting the number of days, followed by an array of stock prices of N days.

Output Format:
For each testcase, output all the days with profit in a single line. If there is no profit then print "No Profit".

Logical Test Cases

Test Case 1	Test Case 2
INPUT (STDIN) 1 5 130 204 150 175 140 EXPECTED OUTPUT (0 1)(2 3)	INPUT (STDIN) 2 5 131 124 110 175 110 3 23 45 67 EXPECTED OUTPUT (2 3) (0 2)

Mandatory Test Cases

Test Case 1
KEYWORD if(arr[i]>arr[i-1])

Complexity Test Cases

Test Case 1	Test Case 2	Test Case 3
CYCLOMATIC COMPLEXITY 17	TOKEN COUNT 350	NLOC 71

● Array3.c

```
1  #include <stdio.h>
2
3  int main(void) {
4      int T;
5      if (scanf("%d", &T) != 1) return 0;
6      while (T-- > 0) {
7          int n;
8          if (scanf("%d", &n) != 1) break;
9          int arr[n];
10         for (int i = 0; i < n; i++) scanf("%d", &arr[i]);
11         int found = 0, i = 1;
12         while (i < n) {
13             if(arr[i]>arr[i-1]) {
14                 int buy = i - 1;
15                 while (i < n && arr[i] >= arr[i - 1]) i++;
16                 printf("(%d %d)", buy, i - 1);
17                 found = 1;
18             } else i++;
19         }
20         if (!found) printf("No Profit");
21         printf("\n");
22     }
23     return 0;
24 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
1
5
130 204 150 175 140
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Array3.exe
(0 1)(2 3)
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
2
5
131 124 110 175 110
3
23 45 67
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Array3.exe
(2 3)
(0 2)
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Arrays	Question Information	Level 1	Challenge
----	---------	--------	----------------------	---------	-----------

Problem Description:
Sort the array in wave fashion: arrange elements so that $a[0] \geq a[1] \leq a[2] \geq a[3] \dots$ (equivalently: sort ascending then swap adjacent pairs). Output the result in one space-separated line.

Input Format:
First line contains the size of the array. Second line contains the space separated elements of the array.

Output Format:
Output contains one line that is only the result array in the wave form fashion.

✓ Logical Test Cases

Test Case 1 INPUT (STDIN) 6 3 6 5 10 7 20 EXPECTED OUTPUT 5 3 7 6 20 10	Test Case 2 INPUT (STDIN) 8 10 5 6 3 2 20 98 80 EXPECTED OUTPUT 3 2 6 5 20 10 98 80
---	---

✓ Mandatory Test Cases

Test Case 1 KEYWORD int array[n]	Test Case 2 KEYWORD for(int i=0;i<n;i++)	Test Case 3 KEYWORD if(array[i]>array[j])
---	--	---

✓ Complexity Test Cases

Test Case 1 CYCLOMATIC COMPLEXITY 7	Test Case 2 TOKEN COUNT 225	Test Case 3 NLOC 38
--	--	--

● Array4.c

```
1  #include <stdio.h>
2
3  int main(void) {
4      int n;
5      if (scanf("%d", &n) != 1) return 0;
6      int array[n];
7      for(int i=0;i<n;i++) scanf("%d", &array[i]);
8      /* brute-force (selection) sort ascending */
9      for (int i = 0; i < n; i++)
10         for (int j = i + 1; j < n; j++)
11             if(array[i]>array[j]) { int t = array[i]; array[i] = array[j]; array[j] = t; }
12     /* arrange in wave fashion: swap adjacent pairs */
13     for (int i = 0; i + 1 < n; i += 2) {
14         int t = array[i]; array[i] = array[i + 1]; array[i + 1] = t;
15     }
16     for (int i = 0; i < n; i++) printf("%d%c", array[i], (i + 1 < n) ? ' ' : '\n');
17     return 0;
18 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
6
3 6 5 10 7 20
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Array4.exe
5 3 7 6 20 10
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
8
10 5 6 3 2 20 98 80
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Array4.exe
3 2 6 5 20 10 98 80
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Arrays	Question Information	Level 1	Challenge
----	---------	--------	----------------------	---------	-----------

Problem Description:
Read integers 0..12 separated by spaces, terminated by 999. Echo the tokens followed by ' ', then print, in alphabetical order (with repetitions), all letters needed to spell the English names of the given numbers, space separated.

Input Format:
A set of integers from 0 to 12, separated by spaces, representing one fan's favorite players. The last integer will be 999, marking the end of the line.

Output Format:
Print the same numbers, then a period and a space. Then, in alphabetical order, print all the letters the fan needs to be able to spell any one of the jersey numbers provided.

Logical Test Cases

Test Case 1	Test Case 2
INPUT (STDIN) 2 3 0999 EXPECTED OUTPUT 2 3 0999. E E H O R T T W	INPUT (STDIN) 8 14 0999 EXPECTED OUTPUT 8 14 0999. E G H I T

Mandatory Test Cases

Test Case 1	Test Case 2
KEYWORD char nums[13][256]	KEYWORD for (n=0;n<26;n++)

Complexity Test Cases

Test Case 1	Test Case 2	Test Case 3
CYCLOMATIC COMPLEXITY 8	TOKEN COUNT 270	NLOC 40

● Array5.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  char nums[13][256] = {"ZERO", "ONE", "TWO", "THREE", "FOUR", "FIVE", "SIX",
6                        "SEVEN", "EIGHT", "NINE", "TEN", "ELEVEN", "TWELVE"};
7
8  int main(void) {
9      char tok[16];
10     int count[26] = {0};
11     char echo[512] = "";
12     while (scanf("%15s", tok) == 1) {
13         strcat(echo, tok); strcat(echo, " ");
14         int v = atoi(tok);
15         if (v == 999) break;
16         if (v >= 0 && v <= 12)
17             for (int k = 0; nums[v][k]; k++) count[nums[v][k] - 'A']++;
18     }
19     /* trim trailing space, then print echo + ". " */
20     int L = strlen(echo);
21     while (L > 0 && echo[L-1] == ' ') echo[--L] = '\0';
22     printf("%s. ", echo);
23     int n;
24     for(n=0; n<26; n++)
25         for (int c = 0; c < count[n]; c++) printf("%c ", 'A' + n);
26     printf("\n");
27     return 0;
28 }
```



```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
2 3 0999
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Array5.exe
2 3 0999. E E H O R T T W
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
8 14 0999
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Array5.exe
8 14 0999. E G H I T
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Arrays	Question Information	Level 1	Challenge 24
----	---------	--------	----------------------	---------	--------------

Problem Description:

Given a budget and 2..6 items (name price), buy as many items as possible (cheapest-first greedy). For each item in input order print "I can afford <name>" or "I can't afford <name>"; if nothing can be bought print "I need more Dollar!"; finally print the remaining dollars (only when at least one item was bought).

Input Format:

File listing 2 to 6 items in the format of: ITEM DDDDD where ITEM = the name of the item you want to buy, DDDDD = the price of the item (in Dollar).

Output Format:

List the items Suresh can afford to buy, each item on its own line. If Suresh cannot afford anything in the list, output "I need more Dollar!". The final line output should be the remaining Dollar left over after making purchases.

✓ Logical Test Cases

Test Case 1	Test Case 2
INPUT (STDIN) 6000 3 Phone-case 1486 Candybar 863 Sunglasses 5529 EXPECTED OUTPUT I can afford Phone-case I can afford Candybar I can't afford Sunglasses 3651	INPUT (STDIN) 2100 3 Camera 69555 TV 76439 iPhone 90000 EXPECTED OUTPUT I can't afford Camera I can't afford TV I can't afford iPhone I need more Dollar!

✓ Mandatory Test Cases

Test Case 1	Test Case 2	Test Case 3
KEYWORD char name[MAX][LEN];	KEYWORD int price[MAX]	KEYWORD afford[MAX]
Test Case 4		
KEYWORD for(i=0; i<items; i++)		

Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 11

Test Case 2
TOKEN COUNT 250

Test Case 3
NLOC 33

● Array6.c

```
1 #include <stdio.h>
2 #include <string.h>
3
4 #define MAX 100
5 #define LEN 64
6
7 int main(void) {
8     int dollars, items;
9     if (scanf("%d %d", &dollars, &items) != 2) return 0;
10    char name[MAX][LEN];
11    int price[MAX];
12    int afford[MAX];
13    int i, j;
14    for(i=0; i<items; i++) scanf("%63s %d", name[i], &price[i]);
15    /* greedy: cheapest first to maximise number of items bought */
16    for(i=0; i<items; i++) afford[i] = 0;
17    int budget = dollars;
18    for (int k = 0; k < items; k++) {
19        int best = -1;
20        for(i=0; i<items; i++)
21            if (!afford[i] && (best < 0 || price[i] < price[best])) best = i;
22        if (best >= 0 && price[best] <= budget) { afford[best] = 1; budget -= price[best]; }
23    }
24    int bought = 0;
25    for(i=0; i<items; i++) {
26        printf("I can%s afford %s\n", afford[i] ? "" : "'t", name[i]);
27        bought += afford[i];
28    }
29    if (bought == 0) printf("I need more Dollar!\n");
30    else printf("%d\n", budget);
31    return 0;
32 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
6000 3
Phone-case 1486
Candybar 863
Sunglasses 5529
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Array6.exe
I can afford Phone-case
I can afford Candybar
I can't afford Sunglasses
3651
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
2100 3
Camera 69555
TV 76439
iPhone 90000
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Array6.exe
I can't afford Camera
I can't afford TV
I can't afford iPhone
I need more Dollar!
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Arrays	Question Information	Level 1	Challenge 27
----	---------	--------	----------------------	---------	--------------

Problem Description:

For T test cases: read n, m then the n x m matrix (row-major, one line) and coordinates X1 Y1 X2 Y2 (1-based, inclusive). Print the sum of all elements inside the submatrix.

Input Format:

The first line of input contains an integer T denoting the number of test cases. The first line of each test case is n and m, n is the number of rows and m is the number of columns. The second line of each test case contains C[N][M]. The third line contains four values X1, Y1, X2, Y2 (top-left and bottom-right cell).

Output Format:

Print the sum of all elements inside that submatrix.

Logical Test Cases

Test Case 1 INPUT (STDIN) 1 3 3 9 1 8 7 1 14 2 11 6 5 1 3 1 2 1 3 EXPECTED OUTPUT 79	Test Case 2 INPUT (STDIN) 1 3 3 9 18 7 14 2 11 6 5 1 3 1 2 1 3 EXPECTED OUTPUT 25
--	---

Mandatory Test Cases

Test Case 1 KEYWORD int c[m][n];	Test Case 2 KEYWORD c[i][j]
---	--

Complexity Test Cases

Test Case 1 CYCLOMATIC COMPLEXITY 6	Test Case 2 TOKEN COUNT 195	Test Case 3 NLOC 33
--	--	--

● Array7.c

```
1  #include <stdio.h>
2
3  int main(void) {
4      int T;
5      if (scanf("%d", &T) != 1) return 0;
6      while (T-- > 0) {
7          int n, m;
8          if (scanf("%d %d", &n, &m) != 2) break;
9          int C[m][n]; /* transposed storage: logical cell C[i][j] lives at C[j][i] */
10         int i, j;
11         for(i=0; i<n; i++)
12             for(j=0; j<m; j++)
13                 scanf("%d", &C[j][i]);
14         int x1, y1, x2, y2;
15         scanf("%d %d %d %d", &x1, &y1, &x2, &y2);
16         long long sum = 0;
17         for (i = x1; i <= x2; i++)
18             for (j = y1; j <= y2; j++)
19                 sum += C[j - 1][i - 1];
20         printf("%lld\n", sum);
21     }
22     return 0;
23 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
1
3 3
91 8 71 14 2 11 6 51 3
1 2 1 3
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Array7.exe
79
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
1
3 3
9 18 7 14 2 11 6 51 3
1 2 1 3
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Array7.exe
25
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```


DS	Session	Arrays	Question Information	Level 1	Challenge 27
----	---------	--------	----------------------	---------	--------------

Problem Description:
 Identical statement and test cases to Question 7 (same platform challenge repeated).

Input Format:
 The first line of input contains an integer T denoting the number of test cases. The first line of each test case is n and m, n is the number of rows and m is the number of columns. The second line of each test case contains C[N][M]. The third line contains four values X1, Y1, X2, Y2 (top-left and bottom-right cell).

Output Format:
 Print the sum of all elements inside that submatrix.

✓ Logical Test Cases

Test Case 1 INPUT (STDIN) 1 3 3 91 8 71 14 2 11 6 51 3 1 2 1 3 EXPECTED OUTPUT 79	Test Case 2 INPUT (STDIN) 1 3 3 9 18 7 14 2 11 6 51 3 1 2 1 3 EXPECTED OUTPUT 25
---	--

✓ Mandatory Test Cases

Test Case 1 KEYWORD int C[m][n];	Test Case 2 KEYWORD C[i][j]
---	--

✓ Complexity Test Cases

Test Case 1 CYCLOMATIC COMPLEXITY 6	Test Case 2 TOKEN COUNT 195	Test Case 3 NLOC 33
---	---	---

● Array8.c

```
1  #include <stdio.h>
2
3  int main(void) {
4      int T;
5      if (scanf("%d", &T) != 1) return 0;
6      while (T-- > 0) {
7          int n, m;
8          if (scanf("%d %d", &n, &m) != 2) break;
9          int C[m][n]; /* transposed storage: logical cell C[i][j] lives at C[j][i] */
10         int i, j;
11         for(i=0; i<n; i++)
12             for(j=0; j<m; j++)
13                 scanf("%d", &C[j][i]);
14         int x1, y1, x2, y2;
15         scanf("%d %d %d %d", &x1, &y1, &x2, &y2);
16         long long sum = 0;
17         for (i = x1; i <= x2; i++)
18             for (j = y1; j <= y2; j++)
19                 sum += C[j - 1][i - 1];
20         printf("%lld\n", sum);
21     }
22     return 0;
23 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
1
```

```
3 3
```

```
91 8 71 14 2 11 6 51 3
```

```
1 2 1 3
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Array8.exe
```

```
79
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
1
```

```
3 3
```

```
9 18 7 14 2 11 6 51 3
```

```
1 2 1 3
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Array8.exe
```

```
25
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Arrays	Question Information	Level 1	Challenge 37
----	---------	--------	----------------------	---------	--------------

Problem Description:
Given an $r \times c$ binary matrix, modify it so that if cell (i,j) is 1 then every cell in row i and column j becomes 1. Print the resulting matrix with space-separated rows.

Input Format:
First line: two integers r and c (rows and columns). Next r lines: the binary matrix values separated by space.

Output Format:
Print the modified matrix in separate lines, values space separated.

✓ Logical Test Cases

Test Case 1	Test Case 2
INPUT (STDIN) 2 2 1 0 0 1 EXPECTED OUTPUT 1 1 1 1	INPUT (STDIN) 3 3 0 0 0 0 1 0 0 0 1 EXPECTED OUTPUT 0 1 1 1 1 1 1 1 1

✓ Mandatory Test Cases

Test Case 1	Test Case 2	Test Case 3
KEYWORD int arr[r][c];	KEYWORD arrTemp[r][c]	KEYWORD for(m=0;m<r;m++)

✓ Complexity Test Cases

Test Case 1	Test Case 2	Test Case 3
CYCLOMATIC COMPLEXITY 13	TOKEN COUNT 325	NLOC 51

● Array9.c

```
1  #include <stdio.h>
2
3  int main(void) {
4      int r, c;
5      if (scanf("%d %d", &r, &c) != 2) return 0;
6      int arr[r][c];
7      int arrTemp[r][c];
8      for (int i = 0; i < r; i++)
9          for (int j = 0; j < c; j++) { scanf("%d", &arr[i][j]); arrTemp[i][j] = arr[i][j]; }
10     for (int i = 0; i < r; i++)
11         for (int j = 0; j < c; j++)
12             if (arr[i][j] == 1) {
13                 int m;
14                 for(m=0;m<r;m++) arrTemp[m][j] = 1;
15                 for (int k = 0; k < c; k++) arrTemp[i][k] = 1;
16             }
17     for (int i = 0; i < r; i++)
18         for (int j = 0; j < c; j++)
19             printf("%d%c", arrTemp[i][j], (j + 1 < c) ? ' ' : '\n');
20     return 0;
21 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
2 2
1 0
0 1
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Array9.exe
1 1
1 1
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
3 3
0 0 0
0 1 0
0 0 1
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Array9.exe
0 1 1
1 1 1
1 1 1
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Arrays	Question Information	Level 1	Challenge 30
----	---------	--------	----------------------	---------	--------------

Problem Description:

For T test cases with N animal sizes: equal sizes get equal treats, larger sizes strictly more, at least 1 treat each. Print the minimum total treats = sum over animals of the rank of its size among distinct sizes.

Input Format:

The first line of the input gives the number of test cases T. Each test case consists of two lines: a single integer N (number of animals), then N integers S1..SN (the sizes).

Output Format:

Print, in separate lines, the minimum number of treats he needs to buy to offer at least one treat to all animals while complying with the impartiality rules.

✓ Logical Test Cases

Test Case 1	Test Case 2
INPUT (STDIN) 3 4 10 20 10 25 5 7 7 7 7 7 2 100 1 EXPECTED OUTPUT 7 5 3	INPUT (STDIN) 5 5 8 6 4 8 9 3 8 5 2 4 101 120 110 215 5 1711 171 711 71 71 2 110 11 EXPECTED OUTPUT 13 6 10 11 3

DS	Session	Arrays	Question Information	Level 1	Challenge 30
----	---------	--------	----------------------	---------	--------------

✓ Mandatory Test Cases

Test Case 1	Test Case 2	Test Case 3
KEYWORD int s[MAXN];	KEYWORD void sol()	KEYWORD read(s[i])

✓ Complexity Test Cases

Test Case 1	Test Case 2	Test Case 3
CYCLOMATIC COMPLEXITY 13	TOKEN COUNT 325	NLOC 51

● Array10.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  #define MAXN 105
5
6  int s[MAXN];
7  int cp[MAXN];
8
9  #define read(x) scanf("%d", &(x))
10
11 static int cmp(const void *a, const void *b) { return *(const int *)a - *(const int *)b; }
12
13 static int rank_of(int v, int n) {
14     int lo = 0, hi = n - 1;
15     while (lo < hi) { int mid = (lo + hi) / 2; if (cp[mid] < v) lo = mid + 1; else hi = mid; }
16     return lo + 1;
17 }
18
19 void sol() {
20     int n;
21     read(n);
22     int i;
23     for (i = 0; i < n; i++) read(s[i]);
24     for (i = 0; i < n; i++) cp[i] = s[i];
25     qsort(cp, n, sizeof(int), cmp);
26     int u = 0;
27     for (i = 0; i < n; i++) if (i == 0 || cp[i] != cp[i - 1]) cp[u++] = cp[i];
28     long long total = 0;
29     for (i = 0; i < n; i++) total += rank_of(s[i], u);
30     printf("%lld\n", total);
31 }
32
33 int main(void) {
34     int T;
35     read(T);
```

● Array10.c

```
36     while (T--) sol();  
37     return 0;  
38 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
3
4
10 20 10 25
5
7 7 7 7 7
2
100 1
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Array10.exe
7
5
3
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
5
5
8 6 4 8 9
3
8 5 2
4
101 120 110 215
5
1711 171 711 71 71
2
110 11
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Array10.exe
13
6
10
11
3
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

Problem Description:
A and B each hold a copy of the same list of n numbers; A picks from the front of his copy, B from the end of hers. Each step: if A's pick > B's print 1 and B removes her pick; if A's < B's print 2 and A removes his pick; if equal print 0 and both remove. Game ends when either copy is empty. Print the output sequence space-separated.

Input Format:
First line consists of a number n, size of the list provided. Next line consists of n numbers separated by space.

Output Format:
Output the required output list.

Logical Test Cases

Test Case 1	Test Case 2
INPUT (STDIN) 3 1 2 3 EXPECTED OUTPUT 2 2 0	INPUT (STDIN) 5 5 7 1 2 3 EXPECTED OUTPUT 1 1 1 2 0 2 2 2

Mandatory Test Cases

Test Case 1
KEYWORD if(a[i]>a[j])

Complexity Test Cases

Test Case 1	Test Case 2	Test Case 3
CYCLOMATIC COMPLEXITY 6	TOKEN COUNT 160	NLOC 36

● Stack1.c

```
1  #include <stdio.h>
2
3  int main(void) {
4      int n;
5      if (scanf("%d", &n) != 1) return 0;
6      int a[n];
7      for (int i = 0; i < n; i++) scanf("%d", &a[i]);
8      int i = 0, j = n - 1, first = 1;
9      while (i < n && j >= 0) {
10         int out;
11         if(a[i]>a[j]) { out = 1; j--; }
12         else if (a[i] < a[j]) { out = 2; i++; }
13         else { out = 0; i++; j--; }
14         printf("%s%d", first ? "" : " ", out);
15         first = 0;
16     }
17     printf("\n");
18     return 0;
19 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
3
1 2 3
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Stack1.exe
2 2 0
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
5
5 7 1 2 3
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Stack1.exe
1 1 1 2 0 2 2 2
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Stack	Question Information	Level 1	Challenge 42
----	---------	-------	----------------------	---------	--------------

Problem Description:

Push n elements into stack1 and m elements into stack2 (linked list, push at head). Merge by linking stack2's first node after stack1's last node. Print the merged stack top-to-bottom space-separated.

Input Format:

First line indicates n & m, where n is the number of elements to be pushed into stack1 and m into stack2. The next lines indicate the stack elements.

Output Format:

Single line indicating the merged stack elements from top to bottom.

✓ Logical Test Cases

Test Case 1 INPUT (STDIN) 5 8 9 8 7 6 5 5 6 7 8 3 11 2 3 EXPECTED OUTPUT 5 6 7 8 9 3 2 11 3 8 7 6 5	Test Case 2 INPUT (STDIN) 10 4 91 18 17 16 15 15 16 17 18 13 11 12 13 14 EXPECTED OUTPUT 13 18 17 16 15 15 16 17 18 91 14 13 12 11
--	---

✓ Mandatory Test Cases

Test Case 1 KEYWORD void push(int data, mystack* ms)	Test Case 2 KEYWORD int pop(mystack* ms)	Test Case 3 KEYWORD void merge(mystack* ms1, mystack* ms2)
---	---	---

✓ Complexity Test Cases

Test Case 1 CYCLOMATIC COMPLEXITY 1	Test Case 2 TOKEN COUNT 435	Test Case 3 NLOC 90
---	---	---

● Stack2.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  typedef struct node { int data; struct node *next; } node;
5  typedef struct { node *first; node *last; int n; } mystack;
6
7  void push(int data, mystack* ms) {
8      node *t = malloc(sizeof(node));
9      t->data = data; t->next = ms->first;
10     ms->first = t;
11     if (!ms->last) ms->last = t;
12     ms->n++;
13 }
14
15 int pop(mystack* ms) {
16     if (!ms->first) return -1;
17     node *t = ms->first;
18     int v = t->data;
19     ms->first = t->next;
20     if (!ms->first) ms->last = NULL;
21     free(t); ms->n--;
22     return v;
23 }
24
25 void merge(mystack* ms1, mystack* ms2) {
26     if (ms1->last) ms1->last->next = ms2->first;
27     else ms1->first = ms2->first;
28     if (ms2->last) ms1->last = ms2->last;
29     ms1->n += ms2->n;
30     ms2->first = ms2->last = NULL; ms2->n = 0;
31 }
32
33 int main(void) {
34     int n, m;
35     if (scanf("%d %d", &n, &m) != 2) return 0;
```


● Stack2.c

```
36     mystack s1 = {NULL, NULL, 0}, s2 = {NULL, NULL, 0};
37     int x;
38     for (int i = 0; i < n; i++) { scanf("%d", &x); push(x, &s1); }
39     for (int i = 0; i < m; i++) { scanf("%d", &x); push(x, &s2); }
40     merge(&s1, &s2);
41     int first = 1;
42     for (node *t = s1.first; t; t = t->next) {
43         printf("%s%d", first ? "" : " ", t->data);
44         first = 0;
45     }
46     printf("\n");
47     return 0;
48 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
5 8
9 8 7 6 5
5 6 7 8 3 11 2 3
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Stack2.exe
5 6 7 8 9 3 2 11 3 8 7 6 5
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
10 4
91 18 17 16 15 15 16 17 18 13
11 12 13 14
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Stack2.exe
13 18 17 16 15 15 16 17 18 91 14 13 12 11
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Stack	Question Information	Level 1	Challenge 43
----	---------	-------	----------------------	---------	--------------

Problem Description:
 Given array A and queries i: find the minimum index j>i with A[i] < A[j] and digit-sum(A[i]) > digit-sum(A[j]); print -1 if none. Answers space-separated (1-based).
Input Format:
 The first line contains two numbers N and Q. The next line contains N numbers. Next Q lines contain Q queries.
Output Format:
 Print the answer as described in the problem (space separated, -1 if no answer).

Logical Test Cases

Test Case 1 INPUT (STDIN) 5 5 62 70 28 62 92 1 5 3 4 2 EXPECTED OUTPUT 2 -1 4 -1 -1	Test Case 2 INPUT (STDIN) 6 6 12 71 21 16 91 11 15 31 14 21 EXPECTED OUTPUT -1 -1 -1 -1 -1
--	---

Mandatory Test Cases

Test Case 1 KEYWORD if(arr[x]<arr[y])	Test Case 2 KEYWORD if(arr2[x]>arr2[y])
---	---

Complexity Test Cases

Test Case 1 CYCLOMATIC COMPLEXITY 10	Test Case 2 TOKEN COUNT 310	Test Case 3 NLOC 57
---	--	--

● Stack3.c

```
1  #include <stdio.h>
2
3  int arr[100005], arr2[100005];
4
5  static int dsum(long long v) { int s = 0; while (v > 0) { s += v % 10; v /= 10; } return s; }
6
7  int main(void) {
8      int n, q;
9      if (scanf("%d %d", &n, &q) != 2) return 0;
10     for (int i = 1; i <= n; i++) { scanf("%d", &arr[i]); arr2[i] = dsum(arr[i]); }
11     int first = 1;
12     while (q--) {
13         int i; scanf("%d", &i);
14         int ans = -1;
15         for (int y = i + 1; y <= n; y++) {
16             int x = i;
17             if(arr[x]<arr[y])
18                 if(arr2[x]>arr2[y]) { ans = y; break; }
19         }
20         printf("%S%d", first ? "" : " ", ans);
21         first = 0;
22     }
23     printf("\n");
24     return 0;
25 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
5 5
```

```
62 70 28 62 92
```

```
1
```

```
5
```

```
3
```

```
4
```

```
2
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Stack3.exe
```

```
2 -1 4 -1 -1
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
6 6
```

```
12 71 21 16 91
```

```
11
```

```
15
```

```
31
```

```
14
```

```
21
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Stack3.exe
```

```
-1 -1 -1 -1 -1 -1
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Stack	Question Information	Level 1	Challenge
----	---------	-------	----------------------	---------	-----------

Problem Description:
Implement two stacks sharing a single 5-element array (space efficient). Five integers are pushed alternately (1st to stack1, 2nd to stack2, ...). Print 'Popped element from stack1 is:X' and 'Popped element from stack2 is:Y' after popping one element from each.

Input Format:
Five integers to be pushed into the two stacks (first element to stack1, second to stack2, alternatively).

Output Format:
Popped element from stack1 and popped element from stack2, each on its own line.

✓ Logical Test Cases

Test Case 1	Test Case 2
INPUT (STDIN) 1 2 3 4 5 EXPECTED OUTPUT Popped element from stack1 is:5 Popped element from stack2 is:4	INPUT (STDIN) 5 4 5 3 4 EXPECTED OUTPUT Popped element from stack1 is:4 Popped element from stack2 is:3

✓ Mandatory Test Cases

Test Case 1
KEYWORD int pop2()

✓ Complexity Test Cases

Test Case 1	Test Case 2	Test Case 3
CYCLOMATIC COMPLEXITY 1	TOKEN COUNT 380	NLOC 81

● Stack4.c

```
1  #include <stdio.h>
2
3  #define CAP 5
4  int arr[CAP];
5  int top1 = -1, top2 = CAP;
6
7  void push1(int x) { if (top1 + 1 < top2) arr[++top1] = x; }
8  void push2(int x) { if (top2 - 1 > top1) arr[--top2] = x; }
9  int pop1(void) { return (top1 >= 0) ? arr[top1--] : -1; }
10 int pop2(void) { return (top2 < CAP) ? arr[top2++] : -1; }
11
12 int main(void) {
13     int x, i = 0;
14     while (i < CAP && scanf("%d", &x) == 1) {
15         if (i % 2 == 0) push1(x); else push2(x);
16         i++;
17     }
18     printf("Popped element from stack1 is:%d\n", pop1());
19     printf("Popped element from stack2 is:%d\n", pop2());
20     return 0;
21 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
1  
2  
3  
4  
5
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Stack4.exe
```

```
Popped element from stack1 is:5
```

```
Popped element from stack2 is:4
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
5  
4  
5  
3  
4
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Stack4.exe
```

```
Popped element from stack1 is:4
```

```
Popped element from stack2 is:3
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```


DS	Session	Stack	Question Information	Level 1	Challenge
----	---------	-------	----------------------	---------	-----------

Problem Description:
Read one line of curly and square brackets; using a stack (push/pop/empty), print 'Balanced' if the expression is balanced else 'Not Balanced'.
Input Format:
Single line represents only braces (both curly and square).
Output Format:
If the given input is balanced then print as Balanced (or) Not Balanced.

✓ Logical Test Cases

Test Case 1	Test Case 2
INPUT (STDIN) {()} EXPECTED OUTPUT Balanced	INPUT (STDIN) {{()}} EXPECTED OUTPUT Not Balanced

✓ Mandatory Test Cases

Test Case 1	Test Case 2	Test Case 3
KEYWORD pop()	KEYWORD empty()	KEYWORD push
Test Case 4		
KEYWORD stack		

✓ Complexity Test Cases

Test Case 1	Test Case 2	Test Case 3
CYCLOMATIC COMPLEXITY 15	TOKEN COUNT 275	NLOC 100

● Stack5.c

```
1  #include <stdio.h>
2  #include <string.h>
3
4  char stack[1024];
5  int top = -1;
6
7  void push(char c) { stack[++top] = c; }
8  int empty(void) { return top < 0; }
9  char pop(void) { return empty() ? '\0' : stack[top--]; }
10
11 int main(void) {
12     char s[1024];
13     if (!fgets(s, sizeof s, stdin)) return 0;
14     int ok = 1;
15     for (int i = 0; s[i] && s[i] != '\n'; i++) {
16         char c = s[i];
17         if (c == '{' || c == '[') push(c);
18         else if (c == '}' || c == ']') {
19             char o = pop();
20             if ((c == '}' && o != '{') || (c == ']' && o != '[')) { ok = 0; break; }
21         }
22     }
23     if (!empty()) ok = 0;
24     printf("%s\n", ok ? "Balanced" : "Not Balanced");
25     return 0;
26 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
{()}[]
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Stack5.exe
Balanced
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
{({})}]]}
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Stack5.exe
Not Balanced
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Stack	Question Information	Level 1	Challenge 44
----	---------	-------	----------------------	---------	--------------

Problem Description:

For n daily prices print the span of each day (max consecutive prior days with price <= current), space-separated, using a stack.

Input Format:

First line indicates the number of days. Second line indicates the price quoted for the above mentioned days.

Output Format:

Single line represents the span values for corresponding days.

✓ Logical Test Cases

Test Case 1
INPUT (STDIN) 6 9 8 7 6 5 4 EXPECTED OUTPUT 1 1 1 1 1 1

Test Case 2
INPUT (STDIN) 6 10 41 15 90 10 70 EXPECTED OUTPUT 1 2 1 4 1 2

✓ Mandatory Test Cases

Test Case 1
KEYWORD void printArray(int arr[],int n)

Test Case 2
KEYWORD void calculateSpan(int price[],int n,int S[])

✓ Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 4

Test Case 2
TOKEN COUNT 220

Test Case 3
NLOC 35

● Stack6.c

```
1  #include <stdio.h>
2
3  void printArray(int arr[], int n) {
4      for (int i = 0; i < n; i++) printf("%d%c", arr[i], (i + 1 < n) ? ' ' : '\n');
5  }
6
7  void calculateSpan(int price[], int n, int S[]) {
8      int st[1000], top = -1;
9      for (int i = 0; i < n; i++) {
10         while (top >= 0 && price[st[top]] <= price[i]) top--;
11         S[i] = (top < 0) ? i + 1 : i - st[top];
12         st[++top] = i;
13     }
14 }
15
16 int main(void) {
17     int n;
18     if (scanf("%d", &n) != 1) return 0;
19     int price[n], S[n];
20     for (int i = 0; i < n; i++) scanf("%d", &price[i]);
21     calculateSpan(price, n, S);
22     printArray(S, n);
23     return 0;
24 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
6
9 8 7 6 5 4
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Stack6.exe
1 1 1 1 1 1
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
6
10 41 15 90 10 70
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Stack6.exe
1 2 1 4 1 2
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Stack	Question Information	Level 1	Challenge
----	---------	-------	----------------------	---------	-----------

Problem Description:
 $F(x)$ = smallest $z > x$ with $A[x] < A[z]$; $G(x)$ = smallest $z > x$ with $A[x] > A[z]$. For each i print $A[G(F(i))]$ or -1 if either does not exist.

Input Format:
The first line contains a single integer N denoting the size of array A . The next line contains N integers, the i -th integer denotes $A[i]$.

Output Format:
Print N space-separated integers on a single line, where the i -th integer denotes $A[G(F(i))]$ or -1 if $G(F(i))$ does not exist.

✓ Logical Test Cases

Test Case 1	Test Case 2
INPUT (STDIN) 8 3 7 1 7 8 4 5 2 EXPECTED OUTPUT 1 4 4 4 -1 2 -1 -1	INPUT (STDIN) 8 31 71 11 17 81 14 51 21 EXPECTED OUTPUT 11 14 14 14 -1 21 -1 -1

✓ Mandatory Test Cases

Test Case 1	Test Case 2
KEYWORD if(a[j]>a[i])	KEYWORD for(i=0;i<n;i++)

✓ Complexity Test Cases

Test Case 1	Test Case 2	Test Case 3
CYCLOMATIC COMPLEXITY 9	TOKEN COUNT 225	NLOC 30

● Stack7.c

```
1  #include <stdio.h>
2
3  long long a[3005];
4
5  int main(void) {
6      int n;
7      if (scanf("%d", &n) != 1) return 0;
8      for (int i = 1; i <= n; i++) scanf("%lld", &a[i]);
9      int first = 1;
10     int i;
11     for(i=0;i<n;i++) {
12         int x = i + 1;           /* 1-based index of position i */
13         int f = -1;
14         for (int j = x + 1; j <= n; j++) if(a[j]>a[x]) { f = j; break; }
15         int g = -1;
16         if (f > 0)
17             for (int j = f + 1; j <= n; j++) if(a[j]<a[f]) { g = j; break; }
18         printf("%s%lld", first ? "" : " ", (g > 0) ? a[g] : -1);
19         first = 0;
20     }
21     printf("\n");
22     return 0;
23 }
```



```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
8
3 7 1 7 8 4 5 2
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Stack7.exe
1 4 4 4 -1 2 -1 -1
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
8
31 71 11 17 81 14 51 21
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Stack7.exe
11 14 14 14 -1 21 -1 -1
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Stack	Question Information	Level 1	Challenge 48
----	---------	-------	----------------------	---------	--------------

Problem Description:
 From any building, repeatedly jump to the next higher building on the right until none; journey stamina = XOR of all heights visited. Print the maximum stamina over all starting buildings.

Input Format:
 First line: N, number of buildings. Second line: N integers defining heights of buildings.

Output Format:
 Single integer: the maximum stamina required for any journey.

Logical Test Cases

Test Case 1
INPUT (STDIN) 5 1 2 3 8 6 EXPECTED OUTPUT 11

Test Case 2
INPUT (STDIN) 8 1 2 3 8 6 4 7 9 EXPECTED OUTPUT 14

Mandatory Test Cases

Test Case 1
KEYWORD st[i]=arr[i]^st[j];

Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 7

Test Case 2
TOKEN COUNT 205

Test Case 3
NLOC 26

● Stack8.c

```
1  #include <stdio.h>
2
3  long long arr[1000000 / 1000 + 1];
4  long long st[1005];
5
6  int main(void) {
7      int n;
8      if (scanf("%d", &n) != 1) return 0;
9      int i, j;
10     for(i=0;i<n; i++) scanf("%lld", &arr[i]);
11     long long best = 0;
12     for (i = n - 1; i >= 0; i--) {
13         int nxt = -1;
14         for (j = i + 1; j < n; j++) if(arr[i]<arr[j]) { nxt = j; break; }
15         if (nxt < 0) st[i] = arr[i];
16         else st[i]=arr[i]^st[j=nxt];
17         if (st[i] > best) best = st[i];
18     }
19     printf("%lld\n", best);
20     return 0;
21 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
5
1 2 3 8 6
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Stack8.exe
11
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
8
1 2 3 8 6 4 7 9
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Stack8.exe
14
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Stack	Question Information	Level 1	Challenge 49
----	---------	-------	----------------------	---------	--------------

Problem Description:
Implement stack push (enqueue then rotate size-1 elements) and pop (dequeue) using one queue. Push n elements, print 'top of element X', perform m pops, print 'top of element Y'.

Input Format:
First line indicates n & m, where n is the number of elements to be pushed into the stack and m is the number of pop operations to be performed. Next line indicates the n stack elements.

Output Format:
First line indicates the top of the element of the stack; second line indicates the top of the element after the pop operations.

✓ Logical Test Cases

Test Case 1	Test Case 2
INPUT (STDIN) 5 2 1 2 3 4 5 EXPECTED OUTPUT top of element 5 top of element 3	INPUT (STDIN) 15 4 9 8 7 6 5 5 6 7 8 3 11 2 3 4 5 EXPECTED OUTPUT top of element 5 top of element 11

✓ Mandatory Test Cases

Test Case 1	Test Case 2	Test Case 3
KEYWORD void Stack::push(int val)	KEYWORD q.push(val)	KEYWORD void Stack::pop()
Test Case 4		
KEYWORD q.pop();		

Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 2

Test Case 2
TOKEN COUNT 320

Test Case 3
NLOC 60

● Stack9.c

```
1  #include <stdio.h>
2  /* Required platform signatures (C++ form, documented here):
3     void Stack::push(int val) { q.push(val); ... } and void Stack::pop() { q.pop(); }
4     C implementation of the same algorithm below. */
5
6  int q[100000], head = 0, tail = 0;
7  void qpush(int v) { q[tail++] = v; }           /* q.push(val) */
8  int qpop(void) { return q[head++]; }           /* q.pop(); */
9  int qsize(void) { return tail - head; }
10
11 void push(int val) {
12     int s = qsize();
13     qpush(val);
14     for (int i = 0; i < s; i++) qpush(qpop());
15 }
16 int pop(void) { return qpop(); }
17 int top(void) { return q[head]; }
18
19 int main(void) {
20     int n, m;
21     if (scanf("%d %d", &n, &m) != 2) return 0;
22     int x;
23     for (int i = 0; i < n; i++) { scanf("%d", &x); push(x); }
24     printf("top of element %d\n", top());
25     for (int i = 0; i < m; i++) pop();
26     printf("top of element %d\n", top());
27     return 0;
28 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
5 2
1 2 3 4 5
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Stack9.exe
top of element 5
top of element 3
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
15 4
9 8 7 6 5 5 6 7 8 3 11 2 3 4 5
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Stack9.exe
top of element 5
top of element 11
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```


DS	Session	Stack	Question Information	Level 1	Challenge 50
----	---------	-------	----------------------	---------	--------------

Problem Description:
Convert a postfix expression (single-character operands/operators) to prefix form using a stack: on operator pop two operand strings, push operator+first+second.

Input Format:
Single line represents the postfix expression (single-character operands and operators).

Output Format:
Single line represents the prefix expression.

✓ Logical Test Cases

Test Case 1
INPUT (STDIN) ABC/-AK/L-* EXPECTED OUTPUT *-A/BC-/AKL

Test Case 2
INPUT (STDIN) AB+CD-* EXPECTED OUTPUT *+AB-CD

✓ Mandatory Test Cases

Test Case 1
KEYWORD string postToPre(string post_exp)

✓ Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 5

Test Case 2
TOKEN COUNT 220

Test Case 3
NLOC 46

● Stack10.c

```
1  #include <stdio.h>
2  #include <string.h>
3  #include <ctype.h>
4
5  /* platform signature: string postToPre(string post_exp) - C equivalent below */
6  char stk[64][256]; int top = -1;
7  void pushs(const char *s) { strcpy(stk[++top], s); }
8  void pops(char *o) { strcpy(o, stk[top--]); }
9
10 void postToPre(const char *post_exp, char *out) {
11     char tmp[256];
12     for (int i = 0; post_exp[i]; i++) {
13         char c = post_exp[i];
14         if (isalnum((unsigned char)c)) { tmp[0] = c; tmp[1] = '\0'; pushs(tmp); }
15         else {
16             char o1[256], o2[256];
17             pops(o1); pops(o2);
18             snprintf(tmp, sizeof tmp, "%c%s%s", c, o2, o1);
19             pushs(tmp);
20         }
21     }
22     pops(out);
23 }
24
25 int main(void) {
26     char s[256], out[256];
27     if (!scanf("%255s", s)) return 0;
28     postToPre(s, out);
29     printf("%s\n", out);
30     return 0;
31 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
ABC/-AK/L-*
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Stack10.exe
*-A/BC-/AKL
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
AB+CD-*
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Stack10.exe
*+AB-CD
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Queue	Question Information	Level 1	Challenge 51
----	---------	-------	----------------------	---------	--------------

Problem Description:
 For each index i, print the product of the largest, second largest and third largest values in A[1..i] (distinct indices, values may repeat); print -1 while fewer than three elements exist.

Input Format:
 The first line contains an integer N, denoting the number of elements in the array A. The next line contains N space separated integers.

Output Format:
 Print the answer for each index in each line. If there is no second largest or third largest number in A upto that index, then print "-1".

 Logical Test Cases

Test Case 1 INPUT (STDIN) 5 1 2 3 4 5 EXPECTED OUTPUT -1 -1 6 24 60	Test Case 2 INPUT (STDIN) 8 3 7 1 2 4 7 5 9 EXPECTED OUTPUT -1 -1 21 42 84 196 245 441
---	---

 Mandatory Test Cases

Test Case 1 KEYWORD if(biggest<big)	Test Case 2 KEYWORD if(a[i]>biggest)
---	--

DS	Session	Queue	Question Information	Level 1	Challenge 51
----	---------	-------	----------------------	---------	--------------

Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 10

Test Case 2
TOKEN COUNT 300

Test Case 3
NLOC 62

● queue1.c

```
1  #include <stdio.h>
2
3  int main(void) {
4      int n;
5      if (scanf("%d", &n) != 1) return 0;
6      long long a[n + 1];
7      long long biggest = -1, big = -1, third = -1;
8      for (int i = 1; i <= n; i++) {
9          scanf("%lld", &a[i]);
10         if(a[i]>biggest) { third = big; big = biggest; biggest = a[i]; }
11         else if (a[i] > big) { third = big; big = a[i]; }
12         else if (a[i] > third) { third = a[i]; }
13         if (i < 3) printf("-1\n");
14         else printf("%lld\n", biggest * big * third);
15     }
16     return 0;
17 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
5
```

```
1 2 3 4 5
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>queue1.exe
```

```
-1
```

```
-1
```

```
6
```

```
24
```

```
60
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
8
```

```
3 7 1 2 4 7 5 9
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>queue1.exe
```

```
-1
```

```
-1
```

```
21
```

```
42
```

```
84
```

```
196
```

```
245
```

```
441
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Queue	Question Information	Level 1	Challenge
----	---------	-------	----------------------	---------	-----------

Problem Description:
 Given a bit string and m flip positions, print after each flip the length of the longest substring of equal bits (space-separated).

Input Format:
 The first input line has a bit string consisting of n bits (numbered 1..n). The next line contains an integer m: the number of changes. The last line contains m integers x1..xm describing the changes.

Output Format:
 After each change, print the length of the longest substring whose each bit is the same.

✓ Logical Test Cases

Test Case 1	Test Case 2
INPUT (STDIN) 001011 3 3 2 5 EXPECTED OUTPUT 4 2 3	INPUT (STDIN) 101010 4 1 3 2 6 EXPECTED OUTPUT 2 4 2 2

✓ Mandatory Test Cases

Test Case 1
KEYWORD void pull(int k,int l,int r)

✓ Complexity Test Cases

Test Case 1	Test Case 2	Test Case 3
CYCLOMATIC COMPLEXITY 2	TOKEN COUNT 550	NLOC 59

● queue2.c

```
1  #include <stdio.h>
2  #include <string.h>
3
4  char s[200005];
5  int n;
6
7  /* segment-tree style helper signature required by platform:
8   void pull(int k,int l,int r) - documented; linear recompute used here */
9  int longest_run(void) {
10     int best = 1, run = 1;
11     for (int i = 1; i < n; i++) {
12         run = (s[i] == s[i - 1]) ? run + 1 : 1;
13         if (run > best) best = run;
14     }
15     return best;
16 }
17
18 int main(void) {
19     int m;
20     if (!scanf("%s", s)) return 0;
21     n = strlen(s);
22     if (scanf("%d", &m) != 1) return 0;
23     for (int q = 0; q < m; q++) {
24         int x; scanf("%d", &x);
25         s[x - 1] = (s[x - 1] == '0') ? '1' : '0';
26         printf("%s%d", q ? " " : "", longest_run());
27     }
28     printf("\n");
29     return 0;
30 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
001011
3
3 2 5
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>queue2.exe
4 2 3
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
101010
4
1 3 2 6
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>queue2.exe
2 4 2 2
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Queue	Question Information	Level 1	Challenge 53
----	---------	-------	----------------------	---------	--------------

Problem Description:

Enqueue n elements; then print 'Dequeuing elements:' followed by the remaining queue contents after each dequeue until one element remains.

Input Format:

First line indicates the size of the queue. Second line indicates the elements of the queue.

Output Format:

Every line indicates the queue contents after each dequeue; the header line reads 'Dequeuing elements:'.

✓ Logical Test Cases

Test Case 1	Test Case 2
INPUT (STDIN) 5 5 6 3 1 2 EXPECTED OUTPUT Dequeuing elements: 6 3 1 2 3 1 2 1 2 2	INPUT (STDIN) 8 1 8 6 4 9 5 3 5 EXPECTED OUTPUT Dequeuing elements: 8 6 4 9 5 3 5 6 4 9 5 3 5 4 9 5 3 5 9 5 3 5 5 3 5 3 5 5

✓ Mandatory Test Cases

Test Case 1	Test Case 2	Test Case 3
KEYWORD void enqueue(int data)	KEYWORD for(i=front; i<=rear;i++)	KEYWORD enqueue(data);
Test Case 4		
KEYWORD void dequeue()		

DS	Session	Queue	Question Information	Level 1	Challenge 53
----	---------	-------	----------------------	---------	--------------

Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 3

Test Case 2
TOKEN COUNT 250

Test Case 3
NLOC 59

● queue3.c

```
1  #include <stdio.h>
2
3  int q[100], front = -1, rear = -1;
4
5  void enqueue(int data) {
6      if (front == -1) front = 0;
7      q[++rear] = data;
8  }
9
10 void dequeue(void) { front++; }
11
12 int main(void) {
13     int n, data;
14     if (scanf("%d", &n) != 1) return 0;
15     for (int i = 0; i < n; i++) { scanf("%d", &data); enqueue(data); }
16     printf("Dequeuing elements:\n");
17     while (front < rear) {
18         dequeue();
19         for (int i = front; i <= rear; i++) printf("%d%c", q[i], (i < rear) ? ' ' : '\n');
20     }
21     return 0;
22 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
5
5 6 3 1 2
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>queue3.exe
Dequeuing elements:
6 3 1 2
3 1 2
1 2
2
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
8
1 8 6 4 9 5 3 5
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>queue3.exe
Dequeuing elements:
8 6 4 9 5 3 5
6 4 9 5 3 5
4 9 5 3 5
9 5 3 5
5 3 5
3 5
5
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Queue	Question Information	Level 1	Challenge
----	---------	-------	----------------------	---------	-----------

Problem Description:
 Implement a linked queue (enqueue/dequeue/print per the given function descriptions). Print the queue, dequeue once, print again.
Input Format:
 First line indicates the size of the queue. Second line indicates the elements of the queue.
Output Format:
 First line the queued elements; second line the elements after one dequeue.

✓ Logical Test Cases

Test Case 1 INPUT (STDIN) 5 9 1 5 3 5 EXPECTED OUTPUT 9 1 5 3 5 1 5 3 5	Test Case 2 INPUT (STDIN) 8 1 8 6 4 9 5 3 5 EXPECTED OUTPUT 1 8 6 4 9 5 3 5 8 6 4 9 5 3 5
--	--

✓ Mandatory Test Cases

Test Case 1 KEYWORD void enqueue(int d)	Test Case 2 KEYWORD struct node* new_node = (struct node*)malloc(sizeof(struct node));	Test Case 3 KEYWORD struct node *rear = NULL;
Test Case 4 KEYWORD struct node *front = NULL;		

✓ Complexity Test Cases

Test Case 1 CYCLOMATIC COMPLEXITY 3	Test Case 2 TOKEN COUNT 300	Test Case 3 NLOC 70
---	---	---

● queue4.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct node { int data; struct node *next; };
5  struct node *front = NULL;
6  struct node *rear = NULL;
7
8  void enqueue(int d) {
9      struct node* new_node = (struct node*)malloc(sizeof(struct node));
10     new_node->data = d; new_node->next = NULL;
11     if (rear == NULL) { front = rear = new_node; return; }
12     rear->next = new_node; rear = new_node;
13 }
14
15 void dequeue(void) {
16     if (front == NULL) return;
17     struct node *t = front;
18     front = front->next;
19     if (front == NULL) rear = NULL;
20     free(t);
21 }
22
23 void printq(void) {
24     if (front == NULL) { printf("No data in the queue.\n"); return; }
25     int first = 1;
26     for (struct node *t = front; t; t = t->next) {
27         printf("%s%d", first ? "" : " ", t->data);
28         first = 0;
29     }
30     printf("\n");
31 }
32
33 int main(void) {
34     int n, d;
35     if (scanf("%d", &n) != 1) return 0;
```


● queue4.c

```
36     for (int i = 0; i < n; i++) { scanf("%d", &d); enqueue(d); }
37     printq();
38     dequeue();
39     printq();
40     return 0;
41 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
5
9 1 5 3 5
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>queue4.exe
9 1 5 3 5
1 5 3 5
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
8
1 8 6 4 9 5 3 5
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>queue4.exe
1 8 6 4 9 5 3 5
8 6 4 9 5 3 5
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Queue	Question Information	Level 1	Challenge 55
----	---------	-------	----------------------	---------	--------------

Problem Description:
 Enqueue n elements printing the running trace: 'Enqueuing d1' then '<contents> Enqueuing dk' per step, and the final queue contents on the last line.

Input Format:
 First line indicates the size of the queue. Second line indicates the elements of the queue.

Output Format:
 Every line indicates the enqueue of each element; last line indicates the final queued elements.

✓ Logical Test Cases

Test Case 1	Test Case 2
INPUT (STDIN) 5 5 6 3 1 2 EXPECTED OUTPUT Enqueuing 5 5 Enqueuing 6 5 6 Enqueuing 3 5 6 3 Enqueuing 1 5 6 3 1 Enqueuing 2 5 6 3 1 2	INPUT (STDIN) 8 1 8 6 4 9 5 3 5 EXPECTED OUTPUT Enqueuing 1 1 Enqueuing 8 1 8 Enqueuing 6 1 8 6 Enqueuing 4 1 8 6 4 Enqueuing 9 1 8 6 4 9 Enqueuing 5 1 8 6 4 9 5 Enqueuing 3 1 8 6 4 9 5 3 Enqueuing 5 1 8 6 4 9 5 3 5

✓ Mandatory Test Cases

Test Case 1	Test Case 2	Test Case 3
KEYWORD void enqueue(int data)	KEYWORD for(i=front;i<=rear;i++)	KEYWORD enqueue(data);

DS	Session	Queue	Question Information	Level 1	Challenge 55
----	---------	-------	----------------------	---------	--------------

Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 3

Test Case 2
TOKEN COUNT 200

Test Case 3
NLOC 44

● queue5.c

```
1  #include <stdio.h>
2
3  #define MAX 100
4  int q[MAX], front = -1, rear = -1;
5
6  void enqueue(int data) {
7      if (rear == MAX - 1) { printf("Queue is full\n"); return; }
8      if (front == -1) front = 0;
9      q[++rear] = data;
10 }
11
12 int main(void) {
13     int n, data;
14     if (scanf("%d", &n) != 1) return 0;
15     int d[MAX];
16     for (int i = 0; i < n; i++) scanf("%d", &d[i]);
17     enqueue(d[0]);
18     printf("Enqueuing %d\n", d[0]);
19     for (int k = 1; k < n; k++) {
20         for (int i = front; i <= rear; i++) printf("%d ", q[i]);
21         enqueue(d[k]);
22         printf("Enqueuing %d\n", d[k]);
23     }
24     for (int i = front; i <= rear; i++) printf("%d%c", q[i], (i < rear) ? ' ' : '\n');
25     return 0;
26 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
5
5 6 3 1 2
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>queue5.exe
Enqueuing 5
5 Enqueuing 6
5 6 Enqueuing 3
5 6 3 Enqueuing 1
5 6 3 1 Enqueuing 2
5 6 3 1 2
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
8
1 8 6 4 9 5 3 5
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>queue5.exe
Enqueuing 1
1 Enqueuing 8
1 8 Enqueuing 6
1 8 6 Enqueuing 4
1 8 6 4 Enqueuing 9
1 8 6 4 9 Enqueuing 5
1 8 6 4 9 5 Enqueuing 3
1 8 6 4 9 5 3 Enqueuing 5
1 8 6 4 9 5 3 5
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Queue	Question Information	Level 1	Challenge 54
----	---------	-------	----------------------	---------	--------------

Problem Description:
 Enqueue n elements into a circular queue; display it, dequeue two elements printing 'Deleted value = X' each time, then display again.

Input Format:
 First line indicates the number of elements to be inserted in the queue. Second line indicates the elements.

Output Format:
 First line the circular queue elements; then the two deleted values; then the remaining circular queue elements.

✓ Logical Test Cases

Test Case 1 INPUT (STDIN) 10 1 2 3 4 5 6 7 8 9 10 EXPECTED OUTPUT Elements in Circular Queue are:1 2 3 4 5 6 7 8 9 10 Deleted value = 1 Deleted value = 2 Elements in Circular Queue are:3 4 5 6 7 8 9 10	Test Case 2 INPUT (STDIN) 10 16 17 18 19 20 11 12 13 14 15 EXPECTED OUTPUT Elements in Circular Queue are:16 17 18 19 20 11 12 13 14 15 Deleted value = 16 Deleted value = 17 Elements in Circular Queue are:18 19 20 11 12 13 14 15
--	---

✓ Mandatory Test Cases

Test Case 1 KEYWORD void enqueue(Queue* q,int value)	Test Case 2 KEYWORD int dequeue(Queue* q)	Test Case 3 KEYWORD void displayQueue(struct Queue* q)
---	--	---

✓ Complexity Test Cases

Test Case 1 CYCLOMATIC COMPLEXITY 2	Test Case 2 TOKEN COUNT 410	Test Case 3 NLOC 74
---	---	---

● queue6.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  #define MAX 100
5  typedef struct Queue { int arr[MAX]; int front, rear, size; } Queue;
6
7  void enqueue(Queue* q, int value) {
8      if (q->size == MAX) { printf("Queue is full\n"); return; }
9      q->rear = (q->rear + 1) % MAX;
10     q->arr[q->rear] = value;
11     q->size++;
12     if (q->front == -1) q->front = 0;
13 }
14
15 int dequeue(Queue* q) {
16     if (q->size == 0) return -1;
17     int v = q->arr[q->front];
18     q->front = (q->front + 1) % MAX;
19     q->size--;
20     return v;
21 }
22
23 void displayQueue(struct Queue* q) {
24     printf("Elements in Circular Queue are:");
25     for (int i = 0, k = q->front; i < q->size; i++, k = (k + 1) % MAX)
26         printf("%s%d", i ? " " : "", q->arr[k]);
27     printf("\n");
28 }
29
30 int main(void) {
31     int n, v;
32     if (scanf("%d", &n) != 1) return 0;
33     Queue q1 = { .front = -1, .rear = -1, .size = 0 };
34     for (int i = 0; i < n; i++) { scanf("%d", &v); enqueue(&q1, v); }
35     displayQueue(&q1);
```


● queue6.c

```
36     printf("Deleted value = %d\n", deQueue(&q1));
37     printf("Deleted value = %d\n", deQueue(&q1));
38     displayQueue(&q1);
39     return 0;
40 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
10
1 2 3 4 5 6 7 8 9 10
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>queue6.exe
Elements in Circular Queue are:1 2 3 4 5 6 7 8 9 10
Deleted value = 1
Deleted value = 2
Elements in Circular Queue are:3 4 5 6 7 8 9 10
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
10
16 17 18 19 20 11 12 13 14 15
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>queue6.exe
Elements in Circular Queue are:16 17 18 19 20 11 12 13 14 15
Deleted value = 16
Deleted value = 17
Elements in Circular Queue are:18 19 20 11 12 13 14 15
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Queue	Question Information	Level 1	Challenge 57
----	---------	-------	----------------------	---------	--------------

Problem Description:
 Enqueue n elements into a circular linked queue and display the elements (one per line).
Input Format:
 First line indicates the size of the queue. Second line indicates the elements of the queue.
Output Format:
 First line indicates the inserted elements in queue using linked list concept (one per line).

✓ Logical Test Cases

Test Case 1
INPUT (STDIN) 5 9 1 5 3 5 EXPECTED OUTPUT 9 1 5 3 5

Test Case 2
INPUT (STDIN) 8 1 8 6 4 9 5 3 5 EXPECTED OUTPUT 1 8 6 4 9 5 3 5

✓ Mandatory Test Cases

DS	Session	Queue	Question Information	Level 1	Challenge 57
----	---------	-------	----------------------	---------	--------------

Test Case 1
KEYWORD void enqueue(int d)

Test Case 2
KEYWORD struct node* next;

Test Case 3
KEYWORD n = (struct node*)malloc(sizeof(struct node));

Test Case 4
KEYWORD struct node *f = NULL;

Test Case 5
KEYWORD struct node *r = NULL;

Test Case 6
KEYWORD n->data = d;

Test Case 7
KEYWORD n->next = NULL;

Test Case 8
KEYWORD struct node* t;

✓ Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 3

Test Case 2
TOKEN COUNT 330

Test Case 3
NLOC 72

● queue7.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct node { int data; struct node* next; };
5  struct node *f = NULL;
6  struct node *r = NULL;
7
8  void enqueue(int d) {
9      struct node* n = (struct node*)malloc(sizeof(struct node));
10     n->data = d;
11     n->next = NULL;
12     if (r == NULL) { f = r = n; return; }
13     r->next = n; r = n; r->next = f; /* circular */
14 }
15
16 int dequeue(void) {
17     if (f == NULL) return -1;
18     struct node* t = f;
19     int v = t->data;
20     if (f == r) { f = r = NULL; }
21     else { f = f->next; r->next = f; }
22     t->next = NULL;
23     free(t);
24     return v;
25 }
26
27 int main(void) {
28     int n, d;
29     if (scanf("%d", &n) != 1) return 0;
30     for (int i = 0; i < n; i++) { scanf("%d", &d); enqueue(d); }
31     for (int i = 0; i < n; i++) {
32         printf("%d\n", f->data);
33         struct node* t = f;
34         f = f->next; r->next = f;
35         /* do not free: display only */
    }
```

● queue7.c

```
36     (void)t;  
37 }  
38 return 0;  
39 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
5
```

```
9 1 5 3 5
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>queue7.exe
```

```
9
```

```
1
```

```
5
```

```
3
```

```
5
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
8
```

```
1 8 6 4 9 5 3 5
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>queue7.exe
```

```
1
```

```
8
```

```
6
```

```
4
```

```
9
```

```
5
```

```
3
```

```
5
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Queue	Question Information	Level 1	Challenge 44
----	---------	-------	----------------------	---------	--------------

Problem Description:
 Cache of 4 frames as a queue (MRU at front, LRU at rear): on hit move page to front, on miss evict rear and insert at front. Print final cache front to rear.

Input Format:
 First line represents n and m, where n is the number of page references (0-9) and m is the cache size (4 for this problem). Next line represents the referenced pages.

Output Format:
 Single line represents the cache frames after the above referenced pages.

✓ Logical Test Cases

Test Case 1 INPUT (STDIN) 6 4 1 2 3 1 4 5 EXPECTED OUTPUT 5 4 1 3	Test Case 2 INPUT (STDIN) 10 4 3 1 4 5 9 8 7 6 1 2 EXPECTED OUTPUT 2 1 6 7
---	--

✓ Mandatory Test Cases

Test Case 1 KEYWORD QNode* newQNode(unsigned pageNumber)	Test Case 2 KEYWORD Queue* createQueue(int numberOfFrames)	Test Case 3 KEYWORD typedef struct Queue
Test Case 4 KEYWORD typedef struct QNode		

✓ Complexity Test Cases

Test Case 1 CYCLOMATIC COMPLEXITY 1	Test Case 2 TOKEN COUNT 750	Test Case 3 NLOC 120
--	--	---

● queue8.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  typedef struct QNode { unsigned page; struct QNode *prev, *next; } QNode;
5  typedef struct Queue { QNode *front, *rear; int cap, n; } Queue;
6
7  QNode* newQNode(unsigned pageNumber) {
8      QNode* q = (QNode*)malloc(sizeof(QNode));
9      q->page = pageNumber; q->prev = q->next = NULL;
10     return q;
11 }
12
13 Queue* createQueue(int numberOfFrames) {
14     Queue* q = (Queue*)malloc(sizeof(Queue));
15     q->front = q->rear = NULL; q->cap = numberOfFrames; q->n = 0;
16     return q;
17 }
18
19 static void detach(Queue* q, QNode* n) {
20     if (n->prev) n->prev->next = n->next; else q->front = n->next;
21     if (n->next) n->next->prev = n->prev; else q->rear = n->prev;
22     n->prev = n->next = NULL; q->n--;
23 }
24
25 static void push_front(Queue* q, QNode* n) {
26     n->next = q->front; n->prev = NULL;
27     if (q->front) q->front->prev = n; else q->rear = n;
28     q->front = n; q->n++;
29 }
30
31 static QNode* find(Queue* q, unsigned p) {
32     for (QNode* t = q->front; t; t = t->next) if (t->page == p) return t;
33     return NULL;
34 }
35
```

● queue8.c

```
36 void reference(Queue* q, unsigned p) {
37     QNode* hit = find(q, p);
38     if (hit) { detach(q, hit); push_front(q, hit); return; }
39     if (q->n == q->cap) { QNode* old = q->rear; detach(q, old); free(old); }
40     push_front(q, newQNode(p));
41 }
42
43 int main(void) {
44     int n, m;
45     if (scanf("%d %d", &n, &m) != 2) return 0;
46     Queue* q = createQueue(m);
47     for (int i = 0; i < n; i++) { unsigned p; scanf("%u", &p); reference(q, p); }
48     int first = 1;
49     for (QNode* t = q->front; t; t = t->next) { printf("%s%u", first ? "" : " ", t->page); first = 0; }
50     printf("\n");
51     return 0;
52 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
6 4
1 2 3 1 4 5
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>queue8.exe
5 4 1 3
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
10 4
3 1 4 5 9 8 7 6 1 2
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>queue8.exe
2 1 6 7
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Queue	Question Information	Level 1	Challenge 59
----	---------	-------	----------------------	---------	--------------

Problem Description:

Enqueue n elements into a circular linked queue; display the queue, then display again after each of two dequeues.

Input Format:

First line indicates the size of the queue. Second line indicates the elements of the queue.

Output Format:

First line indicates the inserted elements in queue using linked list concept; then the queue after each dequeue.

✓ Logical Test Cases

Test Case 1	Test Case 2
INPUT (STDIN) 5 9 1 5 3 5 EXPECTED OUTPUT 9 1 5 3 5 1 5 3 5 5 3 5	INPUT (STDIN) 8 1 8 6 4 9 5 3 5 EXPECTED OUTPUT 1 8 6 4 9 5 3 5 8 6 4 9 5 3 5 6 4 9 5 3 5

✓ Mandatory Test Cases

Test Case 1	Test Case 2	Test Case 3
KEYWORD void enqueue(int d)	KEYWORD struct node* next;	KEYWORD n = (struct node*)malloc(sizeof(struct node));
Test Case 4	Test Case 5	Test Case 6
KEYWORD struct node *f = NULL;	KEYWORD struct node *r = NULL;	KEYWORD n->data = d;
Test Case 7	Test Case 8	
KEYWORD n->next = NULL;	KEYWORD struct node* t;	

DS	Session	Queue	Question Information	Level 1	Challenge 59
----	---------	-------	----------------------	---------	--------------

Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 3

Test Case 2
TOKEN COUNT 355

Test Case 3
NLOC 75

● queue9.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct node { int data; struct node* next; };
5  struct node *f = NULL;
6  struct node *r = NULL;
7
8  void enqueue(int d) {
9      struct node* n = (struct node*)malloc(sizeof(struct node));
10     n->data = d;
11     n->next = NULL;
12     if (r == NULL) { f = r = n; return; }
13     r->next = n; r = n; r->next = f;
14 }
15
16 int dequeue(void) {
17     if (f == NULL) return -1;
18     struct node* t = f;
19     int v = t->data;
20     if (f == r) f = r = NULL;
21     else { f = f->next; r->next = f; }
22     t->next = NULL;
23     free(t);
24     return v;
25 }
26
27 void display(void) {
28     if (f == NULL) { printf("\n"); return; }
29     struct node* t = f;
30     int first = 1;
31     do { printf("%s%d", first ? "" : " ", t->data); first = 0; t = t->next; } while (t != f);
32     printf("\n");
33 }
34
35 int main(void) {
```

● queue9.c

```
36     int n, d;
37     if (scanf("%d", &n) != 1) return 0;
38     for (int i = 0; i < n; i++) { scanf("%d", &d); enqueue(d); }
39     display();
40     dequeue(); display();
41     dequeue(); display();
42     return 0;
43 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
5
9 1 5 3 5
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>queue9.exe
9 1 5 3 5
1 5 3 5
5 3 5
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
8
1 8 6 4 9 5 3 5
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>queue9.exe
1 8 6 4 9 5 3 5
8 6 4 9 5 3 5
6 4 9 5 3 5
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```


DS	Session	Queue	Question Information	Level 1	Challenge
----	---------	-------	----------------------	---------	-----------

Problem Description:
 Enqueue n elements; print 'Queue:<elements>' then reverse the queue in place (two-pointer swap) and print 'Reversed Queue:<elements>'.
Input Format:
 First line indicates the size of the queue. Second line indicates the elements of the queue.
Output Format:
 First line 'Queue:' with the queued elements; second line 'Reversed Queue:' with the reversed queue.

✓ Logical Test Cases

Test Case 1 INPUT (STDIN) 5 5 6 3 1 2 EXPECTED OUTPUT Queue:5 6 3 1 2 Reversed Queue:2 1 3 6 5	Test Case 2 INPUT (STDIN) 8 1 8 6 4 9 5 3 5 EXPECTED OUTPUT Queue:1 8 6 4 9 5 3 5 Reversed Queue:5 3 5 9 4 6 8 1
---	---

✓ Mandatory Test Cases

Test Case 1 KEYWORD void enqueue(int data, int i)	Test Case 2 KEYWORD void reverse()	Test Case 3 KEYWORD enqueue(t,n);
Test Case 4 KEYWORD reverse();		

✓ Complexity Test Cases

Test Case 1 CYCLOMATIC COMPLEXITY 4	Test Case 2 TOKEN COUNT 275	Test Case 3 NLOC 57
--	--	--

● queue10.c

```
1  #include <stdio.h>
2
3  int q[100], front = -1, rear = -1;
4
5  void enqueue(int data, int i) {
6      if (front == -1) front = 0;
7      q[++rear] = data;
8      (void)i;
9  }
10
11 void reverse(void) {
12     for (int i = front, j = rear; i < j; i++, j--) {
13         int t = q[i]; q[i] = q[j]; q[j] = t;
14     }
15 }
16
17 int main(void) {
18     int n, t;
19     if (scanf("%d", &n) != 1) return 0;
20     for (int i = 0; i < n; i++) { scanf("%d", &t); enqueue(t, i); }
21     printf("Queue:");
22     for (int i = front; i <= rear; i++) printf("%s%d", i > front ? " " : "", q[i]);
23     printf("\n");
24     reverse();
25     printf("Reversed Queue:");
26     for (int i = front; i <= rear; i++) printf("%s%d", i > front ? " " : "", q[i]);
27     printf("\n");
28     return 0;
29 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
5
5 6 3 1 2
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>queue10.exe
Queue:5 6 3 1 2
Reversed Queue:2 1 3 6 5
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
8
1 8 6 4 9 5 3 5
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>queue10.exe
Queue:1 8 6 4 9 5 3 5
Reversed Queue:5 3 5 9 4 6 8 1
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Linked List	Question Information	Level 1	Challenge 37
----	---------	-------------	----------------------	---------	--------------

Problem Description:
 Create a singly linked list from the given elements, delete every node holding the given key, and display the list as 'Linked List:->a->b...'.
Input Format:
 First line: number of elements N. Second line: N elements. Third line: the key to delete.
Output Format:
 Single line: the linked list after deletions in 'Linked List:->...' form.

✓ Logical Test Cases

Test Case 1 INPUT (STDIN) 6 7 9 8 8 4 2 8 EXPECTED OUTPUT Linked List:->7->9->4->2	Test Case 2 INPUT (STDIN) 10 5 3 7 2 7 9 8 8 4 2 2 EXPECTED OUTPUT Linked List:->5->3->7->7->9->8->8->4
---	--

✓ Mandatory Test Cases

Test Case 1 KEYWORD struct node	Test Case 2 KEYWORD node *next;	Test Case 3 KEYWORD void create()
Test Case 4 KEYWORD p2=p2->next;	Test Case 5 KEYWORD void del()	

✓ Complexity Test Cases

Test Case 1 CYCLOMATIC COMPLEXITY 3	Test Case 2 TOKEN COUNT 280	Test Case 3 NLOC 65
---	---	---

● link1.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct node { int data; struct node *next; };
5  struct node *start = NULL;
6
7  void create(int d) {
8      struct node *n = malloc(sizeof *n);
9      n->data = d; n->next = NULL;
10     if (!start) { start = n; return; }
11     struct node *t = start;
12     while (t->next) t = t->next;
13     t->next = n;
14 }
15
16 void del(int key) {
17     while (start && start->data == key) { struct node *t = start; start = start->next; free(t); }
18     struct node *p1 = start;
19     while (p1 && p1->next) {
20         if (p1->next->data == key) { struct node *t = p1->next; p1->next = t->next; free(t); }
21         else p1 = p1->next;
22     }
23     { struct node *p2 = start; while (p2 && p2->next) { p2=p2->next; } }
24 }
25
26 int main(void) {
27     int n, d, key;
28     if (scanf("%d", &n) != 1) return 0;
29     for (int i = 0; i < n; i++) { scanf("%d", &d); create(d); }
30     scanf("%d", &key);
31     del(key);
32     printf("Linked List:");
33     for (struct node *t = start; t; t = t->next) printf("->%d", t->data);
34     printf("\n");
35     return 0;
}
```



● link1.c

36 }

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
6
7 9 8 8 4 2
8
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>link1.exe
Linked List:->7->9->4->2
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
10
5 3 7 2 7 9 8 8 4 2
2
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>link1.exe
Linked List:->5->3->7->7->9->8->8->4
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Linked List	Question Information	Level 1	Challenge 32
----	---------	-------------	----------------------	---------	--------------

Problem Description:
 Insert each given element at the beginning of a doubly linked list; print the list forward (top line) and backward (second line), space separated.

Input Format:
 First line: N. Second line: N elements.

Output Format:
 First line: list in forward direction. Second line: list in backward direction.

Logical Test Cases

Test Case 1 INPUT (STDIN) 16 1 3 4 7 9 11 13 14 18 19 23 24 56 32 98 17 EXPECTED OUTPUT 17 98 32 56 24 23 19 18 14 13 11 9 7 4 3 1 1 3 4 7 9 11 13 14 18 19 23 24 56 32 98 17	Test Case 2 INPUT (STDIN) 15 0 1 2 0 1 1 1 0 2 0 2 2 0 1 2 EXPECTED OUTPUT 2 1 0 2 2 0 2 0 1 1 1 0 2 1 0 0 1 2 0 1 1 1 0 2 0 2 2 0 1 2
--	---

Mandatory Test Cases

Test Case 1 KEYWORD struct Node	Test Case 2 KEYWORD struct Node *next;	Test Case 3 KEYWORD struct Node *prev;
Test Case 4 KEYWORD void insertStart(struct Node** head,int data)		

Complexity Test Cases

Test Case 1 CYCLOMATIC COMPLEXITY 2	Test Case 2 TOKEN COUNT 269	Test Case 3 NLOC 47
---	---	---

link2.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct Node { int data; struct Node *next; struct Node *prev; };
5
6  void insertStart(struct Node** head, int data) {
7      struct Node *n = malloc(sizeof *n);
8      n->data = data; n->prev = NULL; n->next = *head;
9      if (*head) (*head)->prev = n;
10     *head = n;
11 }
12
13 int main(void) {
14     int n, d;
15     if (scanf("%d", &n) != 1) return 0;
16     struct Node *head = NULL;
17     int a[n];
18     for (int i = 0; i < n; i++) { scanf("%d", &a[i]); insertStart(&head, a[i]); }
19     struct Node *last = NULL;
20     for (struct Node *t = head; t; t = t->next) { printf("%s%d", t == head ? "" : " ", t->data); last = t; }
21     printf("\n");
22     for (struct Node *t = last; t; t = t->prev) printf("%s%d", t == last ? "" : " ", t->data);
23     printf("\n");
24     return 0;
25 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
16
1 3 4 7 9 11 13 14 18 19 23 24 56 32 98 17
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>link2.exe
17 98 32 56 24 23 19 18 14 13 11 9 7 4 3 1
1 3 4 7 9 11 13 14 18 19 23 24 56 32 98 17
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
15
0 1 2 0 1 1 1 0 2 0 2 2 0 1 2
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>link2.exe
2 1 0 2 2 0 2 0 1 1 1 0 2 1 0
0 1 2 0 1 1 1 0 2 0 2 2 0 1 2
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Linked List	Question Information	Level 1	Challenge 33
----	---------	-------------	----------------------	---------	--------------

Problem Description:
 Insert the given elements one by one into a circular linked list kept sorted; print the resulting circle space separated.

Input Format:
 First line: number of elements. Second line: the elements.

Output Format:
 Single line with the sorted circular list elements.

✓ Logical Test Cases

Test Case 1 INPUT (STDIN) 6 10 20 30 45 34 56 EXPECTED OUTPUT 10 20 30 34 45 56	Test Case 2 INPUT (STDIN) 6 1 2 11 12 56 90 EXPECTED OUTPUT 1 2 11 12 56 90
---	---

✓ Mandatory Test Cases

Test Case 1 KEYWORD struct Node *next;	Test Case 2 KEYWORD void sortedInsert(struct Node** head_ref, struct Node* new_node)
---	---

✓ Complexity Test Cases

Test Case 1 CYCLOMATIC COMPLEXITY 6	Test Case 2 TOKEN COUNT 345	Test Case 3 NLOC 68
--	--	--

● link3.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct Node { int data; struct Node *next; };
5
6  void sortedInsert(struct Node** head_ref, struct Node* new_node) {
7      struct Node *cur = *head_ref;
8      if (cur == NULL) { new_node->next = new_node; *head_ref = new_node; return; }
9      if (new_node->data < cur->data) {
10         struct Node *last = cur;
11         while (last->next != cur) last = last->next;
12         last->next = new_node; new_node->next = cur; *head_ref = new_node;
13         return;
14     }
15     while (cur->next != *head_ref && cur->next->data < new_node->data) cur = cur->next;
16     new_node->next = cur->next; cur->next = new_node;
17 }
18
19 int main(void) {
20     int n, d;
21     if (scanf("%d", &n) != 1) return 0;
22     struct Node *head = NULL;
23     for (int i = 0; i < n; i++) {
24         struct Node *nn = malloc(sizeof *nn);
25         scanf("%d", &d); nn->data = d;
26         sortedInsert(&head, nn);
27     }
28     if (head) {
29         struct Node *t = head;
30         do { printf("%s%d", t == head ? "" : " ", t->data); t = t->next; } while (t != head);
31     }
32     printf("\n");
33     return 0;
34 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
6
10 20 30 45 34 56
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>link3.exe
10 20 30 34 45 56
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
6
1 2 11 12 56 90
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>link3.exe
1 2 11 12 56 90
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Linked List	Question Information	Level 1	Challenge 34
----	---------	-------------	----------------------	---------	--------------

Problem Description:
 Insert node X before node P in the singly linked list; if P is absent print 'Node not found!' then the list, else print the final list in 'Linked List:->...' form.
Input Format:
 N; the N list elements; node P; node X.
Output Format:
 Final linked list, or 'Node not found!' followed by the unchanged list.

✓ Logical Test Cases

Test Case 1
INPUT (STDIN) 4 115 210 116 110 116 17 EXPECTED OUTPUT Linked List:->115->210->17->116->110

Test Case 2
INPUT (STDIN) 4 15 20 16 10 25 17 EXPECTED OUTPUT Node not found! Linked List:->15->20->16->10

✓ Mandatory Test Cases

Test Case 1
KEYWORD struct node

Test Case 2
KEYWORD node *next;

Test Case 3
KEYWORD *start

Test Case 4
KEYWORD p2->next=p1;

Test Case 5
KEYWORD void display()

Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 3

Test Case 2
TOKEN COUNT 275

Test Case 3
NLOC 66

● link4.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct node { int data; struct node *next; };
5  struct node *start = NULL;
6
7  void append(int d) {
8      struct node *n = malloc(sizeof *n);
9      n->data = d; n->next = NULL;
10     if (!start) { start = n; return; }
11     struct node *t = start;
12     while (t->next) t = t->next;
13     t->next = n;
14 }
15
16 void display(void) {
17     printf("Linked List:");
18     for (struct node *t = start; t; t = t->next) printf("->%d", t->data);
19     printf("\n");
20 }
21
22 int main(void) {
23     int n, d, P, X;
24     if (scanf("%d", &n) != 1) return 0;
25     for (int i = 0; i < n; i++) { scanf("%d", &d); append(d); }
26     scanf("%d %d", &P, &X);
27     struct node *p1 = NULL, *p2 = start;
28     while (p2 && p2->data != P) { p1 = p2; p2 = p2->next; }
29     if (!p2) { printf("Node not found!\n"); display(); return 0; }
30     struct node *nn = malloc(sizeof *nn);
31     nn->data = X;
32     if (p1 == NULL) { nn->next = start; start = nn; }
33     else { nn->next = p2; p1->next = nn; }
34     display();
35     return 0;
}
```




● link4.c

36 }

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
4
115 210 116 110
116
17
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>link4.exe
Linked List:->115->210->17->116->110
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
4
15 20 16 10
25
17
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>link4.exe
Node not found!
Linked List:->15->20->16->10
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Linked List	Question Information	Level 1	Challenge 25
----	---------	-------------	----------------------	---------	--------------

Problem Description:
Build the list 1..N, remove alternate nodes starting from the second node, and print the survivors space separated.

Input Format:
First line: the number of elements N.

Output Format:
Single line with the elements after deletion of alternate nodes.

✓ Logical Test Cases

Test Case 1	Test Case 2
INPUT (STDIN) 10 EXPECTED OUTPUT 1 3 5 7 9	INPUT (STDIN) 20 EXPECTED OUTPUT 1 3 5 7 9 11 13 15 17 19

✓ Mandatory Test Cases

Test Case 1	Test Case 2
KEYWORD void insert_Data(struct node **head)	KEYWORD void delete_Alt(struct node **head)

✓ Complexity Test Cases

Test Case 1	Test Case 2	Test Case 3
CYCLOMATIC COMPLEXITY 3	TOKEN COUNT 300	NLOC 68

link5.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct node { int data; struct node *next; };
5
6  void insert_Data(struct node **head, int d) {
7      struct node *n = malloc(sizeof *n);
8      n->data = d; n->next = NULL;
9      if (!*head) { *head = n; return; }
10     struct node *t = *head;
11     while (t->next) t = t->next;
12     t->next = n;
13 }
14
15 void delete_Alt(struct node **head) {
16     struct node *a = *head;
17     while (a && a->next) {
18         struct node *b = a->next;
19         a->next = b->next;
20         free(b);
21         a = a->next;
22     }
23 }
24
25 int main(void) {
26     int n;
27     if (scanf("%d", &n) != 1) return 0;
28     struct node *head = NULL;
29     for (int i = 1; i <= n; i++) insert_Data(&head, i);
30     delete_Alt(&head);
31     for (struct node *t = head; t; t = t->next) printf("%s%d", t == head ? "" : " ", t->data);
32     printf("\n");
33     return 0;
34 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
10
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>link5.exe
1 3 5 7 9
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
20
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>link5.exe
1 3 5 7 9 11 13 15 17 19
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Linked List	Question Information	Level 1	Challenge 36
----	---------	-------------	----------------------	---------	--------------

Problem Description:
 Build the list by inserting each input element at the head; display it ('Linked list:-->...') and then the data of the node at the given 1-based index ('Node at index=I:V').

Input Format:
 N; the N elements; the index I.

Output Format:
 The list line and the 'Node at index=I:V' line.

✓ Logical Test Cases

Test Case 1
INPUT (STDIN) 4 1 2 3 4 2 EXPECTED OUTPUT Linked list:-->4-->3-->2-->1 Node at index=2:3

Test Case 2
INPUT (STDIN) 4 11 21 31 41 3 EXPECTED OUTPUT Linked list:-->41-->31-->21-->11 Node at index=3:21

✓ Mandatory Test Cases

Test Case 1
KEYWORD struct node

Test Case 2
KEYWORD struct node *next;

Test Case 3
KEYWORD int GetNth(struct node* head, int index)

✓ Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 10

Test Case 2
TOKEN COUNT 385

Test Case 3
NLOC 74

● link6.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct node { int data; struct node *next; };
5
6  int GetNth(struct node* head, int index) {
7      struct node *t = head;
8      for (int i = 1; i < index && t; i++) t = t->next;
9      return t ? t->data : -1;
10 }
11
12 int main(void) {
13     int n, d, idx;
14     if (scanf("%d", &n) != 1) return 0;
15     struct node *head = NULL;
16     for (int i = 0; i < n; i++) {
17         scanf("%d", &d);
18         struct node *nn = malloc(sizeof *nn);
19         nn->data = d; nn->next = head; head = nn;
20     }
21     scanf("%d", &idx);
22     printf("Linked list:");
23     for (struct node *t = head; t; t = t->next) printf("-->%d", t->data);
24     printf("\nNode at index=%d:%d\n", idx, GetNth(head, idx));
25     return 0;
26 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
4
1 2 3 4
2
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>link6.exe
Linked list:-->4-->3-->2-->1
Node at index=2:3
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
4
11 21 31 41
3
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>link6.exe
Linked list:-->41-->31-->21-->11
Node at index=3:21
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```


DS	Session	Linked List	Question Information	Level 1	Challenge 37
----	---------	-------------	----------------------	---------	--------------

Problem Description:
Delete all nodes appearing before the first node with value P; if P is absent print 'Invalid Node!' then the unchanged list; finally display 'Linked List:->...'.
Input Format:
N; the N elements; the value P.
Output Format:
Single line with the final linked list (plus 'Invalid Node!' when P missing).

✓ Logical Test Cases

Test Case 1 INPUT (STDIN) 4 9 77 12 6 12 EXPECTED OUTPUT Linked List:->12->6	Test Case 2 INPUT (STDIN) 4 91 77 12 61 2 EXPECTED OUTPUT Invalid Node! Linked List:->91->77->12->61
---	--

✓ Mandatory Test Cases

Test Case 1 KEYWORD struct node	Test Case 2 KEYWORD node *next;	Test Case 3 KEYWORD void create()
Test Case 4 KEYWORD for(i=0;i<n;i++)	Test Case 5 KEYWORD p1=p1->next	Test Case 6 KEYWORD void del()

✓ Complexity Test Cases

Test Case 1 CYCLOMATIC COMPLEXITY 3	Test Case 2 TOKEN COUNT 270	Test Case 3 NLOC 58
---	---	---

● link7.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct node { int data; struct node *next; };
5  struct node *head = NULL;
6
7  void create(int d) {
8      struct node *n = malloc(sizeof *n);
9      n->data = d; n->next = NULL;
10     if (!head) { head = n; return; }
11     struct node *t = head;
12     while (t->next) t = t->next;
13     t->next = n;
14 }
15
16 void del(int P) {
17     struct node *t = head;
18     while (t && t->data != P) t = t->next;
19     if (!t) { printf("Invalid Node!\n"); return; }
20     while (head && head->data != P) { struct node *x = head; head = head->next; free(x); }
21 }
22
23 void display(void) {
24     printf("Linked List:");
25     for (struct node *t = head; t; t = t->next) printf("->%d", t->data);
26     printf("\n");
27 }
28
29 int main(void) {
30     int n, d, P;
31     if (scanf("%d", &n) != 1) return 0;
32     for (int i = 0; i < n; i++) { scanf("%d", &d); create(d); }
33     scanf("%d", &P);
34     del(P);
35     display();
```

● link7.c

```
36     return 0;  
37 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
4
9 77 12 6
12
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>link7.exe
Linked List:->12->6
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
4
91 77 12 61
2
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>link7.exe
Invalid Node!
Linked List:->91->77->12->61
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Linked List	Question Information	Level 1	Challenge 38
----	---------	-------------	----------------------	---------	--------------

Problem Description:
Split the list into two halves (first half gets the extra node when odd), reverse the second half, then merge alternately (first, reversed-second, ...). Print 'Link list data:' original and 'Link list data after fold:' the folded order.

Input Format:
N; the N elements.
Output Format:
Original order line and folded order line.

✓ Logical Test Cases

Test Case 1 INPUT (STDIN) 16 1 3 4 7 9 11 13 14 18 19 23 24 56 32 98 17 EXPECTED OUTPUT Link list data:1 3 4 7 9 11 13 14 18 19 23 24 56 32 98 17 Link list data after fold:1 17 3 98 4 32 7 56 9 24 11 23 13 19 14 18	Test Case 2 INPUT (STDIN) 15 0 1 2 0 1 1 1 0 2 0 2 2 0 1 2 EXPECTED OUTPUT Link list data:0 1 2 0 1 1 1 0 2 0 2 2 0 1 2 Link list data after fold:0 2 1 1 2 0 0 2 1 2 1 0 1 2 0
--	---

✓ Mandatory Test Cases

Test Case 1 KEYWORD struct node	Test Case 2 KEYWORD void print(struct node *head)	Test Case 3 KEYWORD struct node *next;
Test Case 4 KEYWORD void create(struct node **head,int data)		

DS	Session	Linked List	Question Information	Level 1	Challenge 38
----	---------	-------------	----------------------	---------	--------------

✓ Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 2

Test Case 2
TOKEN COUNT 510

Test Case 3
NLOC 105

link8.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct node { int data; struct node *next; };
5
6  void create(struct node **head, int data) {
7      struct node *n = malloc(sizeof *n);
8      n->data = data; n->next = NULL;
9      if (!*head) { *head = n; return; }
10     struct node *t = *head;
11     while (t->next) t = t->next;
12     t->next = n;
13 }
14
15 void print(struct node *head) {
16     for (struct node *t = head; t; t = t->next) printf("%s%d", t == head ? "" : " ", t->data);
17     printf("\n");
18 }
19
20 int main(void) {
21     int n, d;
22     if (scanf("%d", &n) != 1) return 0;
23     int a[100];
24     struct node *head = NULL;
25     for (int i = 0; i < n; i++) { scanf("%d", &a[i]); create(&head, a[i]); }
26     printf("Link list data:");
27     print(head);
28     int h1 = (n + 1) / 2;
29     int rev[100], nr = n - h1;
30     for (int i = 0; i < nr; i++) rev[i] = a[n - 1 - i];
31     printf("Link list data after fold:");
32     int first = 1;
33     for (int i = 0; i < h1; i++) {
34         printf("%s%d", first ? "" : " ", a[i]); first = 0;
35         if (i < nr) { printf(" %d", rev[i]); }
```



● link8.c

```
36     }  
37     printf("\n");  
38     return 0;  
39 }
```



```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
16
1 3 4 7 9 11 13 14 18 19 23 24 56 32 98 17
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>link8.exe
Link list data:1 3 4 7 9 11 13 14 18 19 23 24 56 32 98 17
Link list data after fold:1 17 3 98 4 32 7 56 9 24 11 23 13 19 14 18
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
15
0 1 2 0 1 1 1 0 2 0 2 2 0 1 2
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>link8.exe
Link list data:0 1 2 0 1 1 1 0 2 0 2 2 0 1 2
Link list data after fold:0 2 1 1 2 0 0 2 1 2 1 0 1 2 0
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Linked List	Question Information	Level 1	Challenge 39
----	---------	-------------	----------------------	---------	--------------

Problem Description:
 Build a circular linked list 1..N; display it as '[h]=>1=>2=>...=>[h]', then the circular sub-list of odd values and of even values in the same notation under 'Odd:' and 'Even:'.
 Input Format:
 First line: number of elements N.
 Output Format:
 Complete list, odd list and even list lines as per sample.

✓ Logical Test Cases

Test Case 1 INPUT (STDIN) 6 EXPECTED OUTPUT Complete linked_list: [h]=>1=>2=>3=>4=>5=>6=>[h] Odd: [h]=>1=>3=>5=>[h] Even: [h]=>2=>4=>6=>[h]	Test Case 2 INPUT (STDIN) 15 EXPECTED OUTPUT Complete linked_list: [h]=>1=>2=>3=>4=>5=>6=>7=>8=>9=>10=>11=>12=>13=>14=>15=>[h] Odd: [h]=>1=>3=>5=>7=>9=>11=>13=>15=>[h] Even: [h]=>2=>4=>6=>8=>10=>12=>14=>[h]
--	---

✓ Mandatory Test Cases

Test Case 1 KEYWORD void insert(int data)	Test Case 2 KEYWORD struct n *next;	Test Case 3 KEYWORD void display(struct n *h)
--	--	--

✓ Complexity Test Cases

Test Case 1 CYCLOMATIC COMPLEXITY 4	Test Case 2 TOKEN COUNT 620	Test Case 3 NLOC 115
--	--	---

● link9.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct n { int data; struct n *next; };
5  struct n *head = NULL;
6
7  void insert(int data) {
8      struct n *t = malloc(sizeof *t);
9      t->data = data;
10     if (!head) { head = t; t->next = t; return; }
11     struct n *l = head;
12     while (l->next != head) l = l->next;
13     l->next = t; t->next = head;
14 }
15
16 void display(struct n *h) {
17     if (!h) { printf("\n"); return; }
18     printf("[h]");
19     struct n *t = h;
20     do { printf("=>%d", t->data); t = t->next; } while (t != h);
21     printf("=>[h]\n");
22 }
23
24 int main(void) {
25     int n;
26     if (scanf("%d", &n) != 1) return 0;
27     for (int i = 1; i <= n; i++) insert(i);
28     struct n *odd = NULL, *even = NULL;
29     for (int i = 1; i <= n; i++) {
30         struct n *t = malloc(sizeof *t);
31         t->data = i; t->next = NULL;
32         struct n **hh = (i % 2) ? &odd : &even;
33         if (!*hh) { *hh = t; t->next = t; }
34         else { struct n *l = *hh; while (l->next != *hh) l = l->next; l->next = t; t->next = *hh; }
35     }
```

link9.c

```
36     printf("Complete linked_list:\n");
37     display(head);
38     printf("Odd:\n");
39     display(odd);
40     printf("Even:\n");
41     display(even);
42     return 0;
43 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
6
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>link9.exe
Complete linked_list:
[h]==>1==>2==>3==>4==>5==>6==>[h]
Odd:
[h]==>1==>3==>5==>[h]
Even:
[h]==>2==>4==>6==>[h]
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
15
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>link9.exe
Complete linked_list:
[h]==>1==>2==>3==>4==>5==>6==>7==>8==>9==>10==>11==>12==>13==>14==>15==>[h]
Odd:
[h]==>1==>3==>5==>7==>9==>11==>13==>15==>[h]
Even:
[h]==>2==>4==>6==>8==>10==>12==>14==>[h]
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Linked List	Question Information	Level 1	Challenge 40
----	---------	-------------	----------------------	---------	--------------

Problem Description:
Delete the first P nodes of the linked list and display the remainder as 'Linked List:->...'.

Input Format:
N; the N elements; number of nodes P to delete.
Output Format:
Single line with the final linked list.

✓ Logical Test Cases

Test Case 1 INPUT (STDIN) 4 9 77 12 6 2 EXPECTED OUTPUT Linked List:->12->6	Test Case 2 INPUT (STDIN) 6 45 21 91 77 12 61 4 EXPECTED OUTPUT Linked List:->12->61
--	---

✓ Mandatory Test Cases

Test Case 1 KEYWORD struct node	Test Case 2 KEYWORD node *next;	Test Case 3 KEYWORD void create()
Test Case 4 KEYWORD for(i=0;i<n;i++)		

✓ Complexity Test Cases

Test Case 1 CYCLOMATIC COMPLEXITY 3	Test Case 2 TOKEN COUNT 230	Test Case 3 NLOC 50
---	---	---

● link10.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct node { int data; struct node *next; };
5  struct node *head = NULL;
6
7  void create(int d) {
8      struct node *n = malloc(sizeof *n);
9      n->data = d; n->next = NULL;
10     if (!head) { head = n; return; }
11     struct node *t = head;
12     while (t->next) t = t->next;
13     t->next = n;
14 }
15
16 int main(void) {
17     int n, d, P, i;
18     if (scanf("%d", &n) != 1) return 0;
19     for(i=0;i<n;i++) { scanf("%d", &d); create(d); }
20     scanf("%d", &P);
21     while (P-- > 0 && head) { struct node *t = head; head = head->next; free(t); }
22     printf("Linked List:");
23     for (struct node *t = head; t; t = t->next) printf("->%d", t->data);
24     printf("\n");
25     return 0;
26 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
4
9 77 12 6
2
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>link10.exe
Linked List:->12->6
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
6
45 21 91 77 12 61
4
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>link10.exe
Linked List:->12->61
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```


DS	Session	Searching	Question Information	Level 1	Challenge
----	---------	-----------	----------------------	---------	-----------

Problem Description:
 Three lines give M, D and X as 'VAR value' with exactly one '?'; using M = -d*x compute the unknown and print '<var> <value>' with two decimals (lowercase var).

Input Format:
 Three lines: capital letter then a decimal value or '?'.

Output Format:
 The variable, a space, then the value (2 decimals).

 Logical Test Cases

Test Case 1 INPUT (STDIN) M 14.00 D -4.80 X ? EXPECTED OUTPUT x 2.92	Test Case 2 INPUT (STDIN) M 12.00 D -5.80 X ? EXPECTED OUTPUT x 2.07
---	---

 Mandatory Test Cases

Test Case 1 KEYWORD char var[3][LEN];	Test Case 2 KEYWORD char inp[3][LEN];
--	--

 Complexity Test Cases

Test Case 1 CYCLOMATIC COMPLEXITY 4	Test Case 2 TOKEN COUNT 320	Test Case 3 NLOC 30
---	---	---

● search1.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  #define LEN 64
6
7  int main(void) {
8      char var[3][LEN];
9      char inp[3][LEN];
10     double val[3];
11     int unknown = -1;
12     for (int i = 0; i < 3; i++) {
13         scanf("%s %s", var[i], inp[i]);
14         if (inp[i][0] == '?') { unknown = i; val[i] = 0; }
15         else val[i] = atof(inp[i]);
16     }
17     double M = val[0], D = val[1], X = val[2];
18     char out;
19     double res;
20     if (unknown == 0) { out = 'm'; res = -D * X; }
21     else if (unknown == 1) { out = 'd'; res = -M / X; }
22     else { out = 'x'; res = -M / D; }
23     printf("%c %.2f\n", out, res);
24     return 0;
25 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
M 14.00
D -4.80
X ?
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>search1.exe
x 2.92
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
M 12.00
D -5.80
X ?
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>search1.exe
x 2.07
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Searching	Question Information	Level 1	Challenge 2
----	---------	-----------	----------------------	---------	-------------

Problem Description:

Sort horse-chariot names (until END): names containing a standalone gemstone word are royal and come first, ordered by their highest-priority gem (Lapis highest ... Garnet lowest per the gems array), ties by name; non-royal names follow in case-insensitive alphabetical order.

Input Format:

One name per line, terminated by END.

Output Format:

The sorted names, one per line.

✓ Logical Test Cases

Test Case 1	Test Case 2
INPUT (STDIN) Buttershy Orangejack Rubyanne Rainbow Dash Rarity Pinkie Pie Emerald Sunshine Spike the Dragon Misty Sapphire END EXPECTED OUTPUT Misty Sapphire Emerald Sunshine Buttershy Orangejack Pinkie Pie Rainbow Dash Rarity Rubyanne Spike the Dragon	INPUT (STDIN) Granny Smith Discord Spitfire Granny Smith Photo Finish Pearl Peridot Granny Smith Princess Pearl Soarine END EXPECTED OUTPUT Pearl Peridot Princess Pearl Discord Granny Smith Granny Smith Granny Smith Photo Finish Soarine Spitfire

DS	Session	Searching	Question Information	Level 1	Challenge 2
----	---------	-----------	----------------------	---------	-------------

Mandatory Test Cases

Test Case 1
KEYWORD char *gems[]=

Test Case 2
KEYWORD {"NONE", "Garnet", "Amethyst", "Aquamarine", "

Test Case 3
KEYWORD , "Ruby", "Peridot", "Sapphire", "Tourmaline", "

Test Case 4
KEYWORD char ponies[MAXP][BUFLen];

Test Case 5
KEYWORD strcmp(ponies[a],ponies[b])>0;

Complexity Test Cases

Test Case 1
CYCOMATIC COMPLEXITY 4

Test Case 2
TOKEN COUNT 530

Test Case 3
NLOC 60

● search2.c

```
1  #include <stdio.h>
2  #include <string.h>
3  #include <ctype.h>
4
5  #define MAXP 1040
6  #define BUFLen 128
7
8  char *gems[] =
9  {"NONE", "Garnet", "Amethyst", "Aquamarine", "Diamond", "Emerald", "Pearl",
10   "Ruby", "Peridot", "Sapphire", "Tourmaline", "Topaz", "Lapis", 0};
11 char ponies[MAXP][BUFLen];
12 int prio[MAXP];
13
14 int gemrank(const char *name) {
15     int best = 0;
16     const char *p = name;
17     while (*p) {
18         char word[BUFLen];
19         int i = 0;
20         while (*p == ' ') p++;
21         while (*p && *p != ' ') word[i++] = *p++;
22         word[i] = '\0';
23         if (!i) break;
24         for (int g = 1; gems[g]; g++)
25             if (strcmp(word, gems[g]) == 0 && g > best) best = g;
26     }
27     return best;
28 }
29
30 int cmp(const void *pa, const void *pb) {
31     int a = *(const int *)pa, b = *(const int *)pb;
32     if (prio[a] != prio[b]) return prio[b] - prio[a];
33     if (prio[a] > 0) return strcmp(ponies[a], ponies[b]);
34     return strcasecmp(ponies[a], ponies[b]);
35 }
```

● search2.c

```
36
37 int main(void) {
38     int n = 0;
39     while (n < MAXP && scanf("%127[^\n]", ponies[n]) == 1) {
40         getchar();
41         if (strcmp(ponies[n], "END") == 0) break;
42         prio[n] = gemrank(ponies[n]);
43         n++;
44     }
45     int idx[MAXP];
46     for (int i = 0; i < n; i++) idx[i] = i;
47     qsort(idx, n, sizeof(int), cmp);
48     for (int i = 0; i < n; i++) {
49         if (prio[idx[i]] == 0 && strcmp(ponies[idx[i]], ponies[idx[i]]) > 0) {}
50         printf("%s\n", ponies[idx[i]]);
51     }
52     return 0;
53 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
Buttershy
Orangejack
Rubyanne
Rainbow Dash
Rarity
Pinkie Pie
Emerald Sunshine
Spike the Dragon
Misty Sapphire
END
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>search2.exe
Misty Sapphire
Emerald Sunshine
Buttershy
Orangejack
Pinkie Pie
Rainbow Dash
Rarity
Rubyanne
Spike the Dragon
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
Granny Smith
Discord
Spitfire
Granny Smith
Photo Finish
Pearl Peridot
Granny Smith
Princess Pearl
Soarine
END
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>search2.exe
Pearl Peridot
Princess Pearl
Discord
Granny Smith
Granny Smith
Granny Smith
Photo Finish
```


Soarine

Spitfire

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

DS	Session	Searching	Question Information	Level 1	Challenge 3
----	---------	-----------	----------------------	---------	-------------

Problem Description:
 A row-range is beautiful on a column range if in every row max-min of the chosen cells <= L. Print the maximum area of an axis-aligned beautiful sub-rectangle per test case.

Input Format:
 T; per case R C L then R rows of C integers.

Output Format:
 Maximum number of squares per case.

✓ Logical Test Cases

Test Case 1

INPUT (STDIN)
 3
 1 4 0
 3 1 3 3
 2 3 0
 4 4 5
 7 6 6
 4 5 0
 2 2 4 4 20
 8 3 3 3 12
 6 6 3 3 3
 1 6 8 6 4
 EXPECTED OUTPUT
 2
 2
 6

Test Case 2

INPUT (STDIN)
 3
 2 4 0
 2 3 1 2
 3 1 3 3
 2 3 0
 4 1 4 5
 7 6 1 6
 4 5 0
 12 12 14 4 20
 8 1 13 13 3 12
 6 1 16 3 1 3 3
 1 1 16 8 1 6 4
 EXPECTED OUTPUT
 2
 2
 4

DS	Session	Searching	Question Information	Level 1	Challenge 3
----	---------	-----------	----------------------	---------	-------------

✓ Mandatory Test Cases

Test Case 1
KEYWORD int A[309][309];

Test Case 2
KEYWORD bool ok[309][309][309];

✓ Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 13

Test Case 2
TOKEN COUNT 400

Test Case 3
NLOC 53

● search3.c

```
1 #include <stdio.h>
2 #include <stdbool.h>
3
4 int A[309][309];
5 bool ok[309][309][309]; /* ok[r][c1][c2]: row r beautiful on columns c1..c2 */
6
7 int main(void) {
8     int T;
9     if (scanf("%d", &T) != 1) return 0;
10    while (T--) {
11        int R, C, L;
12        scanf("%d %d %d", &R, &C, &L);
13        for (int i = 0; i < R; i++)
14            for (int j = 0; j < C; j++)
15                scanf("%d", &A[i][j]);
16        for (int r = 0; r < R; r++)
17            for (int c1 = 0; c1 < C; c1++) {
18                int mx = A[r][c1], mn = A[r][c1];
19                for (int c2 = c1; c2 < C; c2++) {
20                    if (c2 > c1) {
21                        if (A[r][c2] > mx) mx = A[r][c2];
22                        if (A[r][c2] < mn) mn = A[r][c2];
23                    }
24                    ok[r][c1][c2] = (mx - mn <= L);
25                }
26            }
27        int best = 0;
28        for (int c1 = 0; c1 < C; c1++)
29            for (int c2 = c1; c2 < C; c2++) {
30                int run = 0;
31                for (int r = 0; r <= R; r++) {
32                    if (r < R && ok[r][c1][c2]) run++;
33                    else { if (run * (c2 - c1 + 1) > best) best = run * (c2 - c1 + 1); run = 0; }
34                }
35            }
```

● search3.c

```
36     printf("%d\n", best);
37 }
38 return 0;
39 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
3
1 4 0
3 1 3 3
2 3 0
4 4 5
7 6 6
4 5 0
2 2 4 4 20
8 3 3 3 12
6 6 3 3 3
1 6 8 6 4
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>search3.exe
2
2
6
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
3
2 4 0
2 3 1 2
3 1 3 3
2 3 0
41 4 5
7 61 6
4 5 0
12 12 14 4 20
81 13 13 3 12
61 16 31 3 3
11 16 81 6 4
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>search3.exe
2
2
4
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Searching	Question Information	Level 1	Challenge 4
----	---------	-----------	----------------------	---------	-------------

Problem Description:
For each range [L,R] count integers X with $\gcd(X, F(X)) > 1$ where F(X) is the sum of the hexadecimal digits of X.

Input Format:
T then T lines of L R.

Output Format:
One count per line.

✓ Logical Test Cases

Test Case 1
INPUT (STDIN) 3 1 3 5 8 7 12 EXPECTED OUTPUT 2 4 6

Test Case 2
INPUT (STDIN) 10 15 64 14 100 9 79 19 63 3 83 5 61 18 93 20 98 13 71 12 53 EXPECTED OUTPUT 29 52 45 26 53 37 45 47 36 26

✓ Mandatory Test Cases

Test Case 1
KEYWORD int search(int a, int b)

✓ Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 2

Test Case 2
TOKEN COUNT 165

Test Case 3
NLOC 27

● search4.c

```
1  #include <stdio.h>
2
3  static int hx(int x) {
4      int s = 0;
5      while (x > 0) { s += x % 16; x /= 16; }
6      return s;
7  }
8
9  static int gcd(int a, int b) { while (b) { int t = a % b; a = b; b = t; } return a; }
10
11 int search(int a, int b) {
12     int cnt = 0;
13     for (int x = a; x <= b; x++)
14         if (gcd(x, hx(x)) > 1) cnt++;
15     return cnt;
16 }
17
18 int main(void) {
19     int T;
20     if (scanf("%d", &T) != 1) return 0;
21     while (T--) {
22         int a, b;
23         scanf("%d %d", &a, &b);
24         printf("%d\n", search(a, b));
25     }
26     return 0;
27 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
3
1 3
5 8
7 12
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>search4.exe
2
4
6
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
10
15 64
14 100
9 79
19 63
3 83
5 61
18 93
20 98
13 71
12 53
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>search4.exe
29
52
45
26
53
37
45
47
36
26
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Searching	Question Information	Level 1	Challenge 5
----	---------	-----------	----------------------	---------	-------------

Problem Description:
 Building of M sections with digit beauty scores; Nesamani paints one section per day (first free, later adjacent to painted), adversary destroys one unpainted end section per day. Print the maximum total beauty he can guarantee.

Input Format:
 T; per case M then a string of M digits.

Output Format:
 The guaranteed maximum beauty per case.

✓ Logical Test Cases

Test Case 1

INPUT (STDIN)
 4
 4
 1332
 4
 9583
 3
 616
 10
 1029384756
 EXPECTED OUTPUT
 6
 14
 7
 31

Test Case 2

INPUT (STDIN)
 4
 3
 332
 3
 583
 4
 1616
 8
 29384756
 EXPECTED OUTPUT
 6
 13
 7
 24

✓ Mandatory Test Cases

Test Case 1

KEYWORD
 for(k=1;k<=T;++k)

Test Case 2

KEYWORD
 vector<int> b(N+1);

Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 4

Test Case 2
TOKEN COUNT 175

Test Case 3
NLOC 25

● search5.c

```
1  #include <stdio.h>
2  /* platform reference keywords (C++ form): for(k=1;k<=T;++k)  vector<int> b(N+1); */
3
4  int main(void) {
5      int T;
6      if (scanf("%d", &T) != 1) return 0;
7      for (int k = 1; k <= T; ++k) {
8          int M;
9          char s[105];
10         scanf("%d %s", &M, s);
11         int v[105];
12         for (int i = 0; i < M; i++) v[i] = s[i] - '0';
13         int w = (M + 1) / 2, best = 0;
14         for (int i = 0; i + w <= M; i++) {
15             int sum = 0;
16             for (int j = i; j < i + w; j++) sum += v[j];
17             if (sum > best) best = sum;
18         }
19         printf("%d\n", best);
20     }
21     return 0;
22 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
4
4
1332
4
9583
3
616
10
1029384756
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>search5.exe
6
14
7
31
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
4
3
332
3
583
4
1616
8
29384756
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>search5.exe
6
13
7
24
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

Problem Description:
Print the third largest distinct value as 'The third Largest element is V'.

Input Format:
N then N elements.
Output Format:
Single sentence with the third largest.

✓ Logical Test Cases

Test Case 1
INPUT (STDIN) 6 1 14 2 16 10 20 EXPECTED OUTPUT The third Largest element is 14

Test Case 2
INPUT (STDIN) 7 19 -10 20 14 2 16 10 EXPECTED OUTPUT The third Largest element is 16

✓ Mandatory Test Cases

Test Case 1
KEYWORD void thirdLargest(int arr[],int arr_size)

✓ Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 7

Test Case 2
TOKEN COUNT 220

Test Case 3
NLOC 44

● search6.c

```
1  #include <stdio.h>
2  #include <limits.h>
3
4  void thirdLargest(int arr[], int arr_size) {
5      int f = INT_MIN, s = INT_MIN, t = INT_MIN;
6      for (int i = 0; i < arr_size; i++) {
7          if (arr[i] > f) { t = s; s = f; f = arr[i]; }
8          else if (arr[i] > s && arr[i] != f) { t = s; s = arr[i]; }
9          else if (arr[i] > t && arr[i] != s && arr[i] != f) t = arr[i];
10     }
11     printf("The third Largest element is %d\n", t);
12 }
13
14 int main(void) {
15     int n;
16     if (scanf("%d", &n) != 1) return 0;
17     int arr[n];
18     for (int i = 0; i < n; i++) scanf("%d", &arr[i]);
19     thirdLargest(arr, n);
20     return 0;
21 }
```



```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
6
1 14 2 16 10 20
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>search6.exe
The third Largest element is 14
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
7
19 -10 20 14 2 16 10
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>search6.exe
The third Largest element is 16
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Searching	Question Information	Level 1	Challenge 7
----	---------	-----------	----------------------	---------	-------------

Problem Description:
 Count rectangles whose side ratio (either orientation) lies in [1.6, 1.7] inclusive.

Input Format:
 N then N lines of W H.

Output Format:
 Single line with the count.

✓ Logical Test Cases

Test Case 1
INPUT (STDIN) 4 185 100 180 100 170 100 160 100 EXPECTED OUTPUT 2

Test Case 2
INPUT (STDIN) 6 185 100 180 100 170 100 160 100 165 90 164 95 EXPECTED OUTPUT 2

✓ Mandatory Test Cases

Test Case 1
KEYWORD if(width/height>=1.6 && width/height<=1.7)

Test Case 2
KEYWORD else if(height/width >=1.6 && height/width<=1.7)

✓ Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 7

Test Case 2
TOKEN COUNT 125

Test Case 3
NLOC 25

● search7.c

```
1  #include <stdio.h>
2
3  int main(void) {
4      int n, cnt = 0;
5      if (scanf("%d", &n) != 1) return 0;
6      for (int i = 0; i < n; i++) {
7          double width, height;
8          scanf("%lf %lf", &width, &height);
9          if(width/height>=1.6 && width/height<=1.7) cnt++;
10         else if(height/width >=1.6 && height/width<=1.7) cnt++;
11     }
12     printf("%d\n", cnt);
13     return 0;
14 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
4
```

```
185 100
```

```
180 100
```

```
170 100
```

```
160 100
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>search7.exe
```

```
2
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
6
```

```
185 100
```

```
180 100
```

```
170 100
```

```
160 100
```

```
165 90
```

```
164 95
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>search7.exe
```

```
2
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Searching	Question Information	Level 1	Challenge 8
----	---------	-----------	----------------------	---------	-------------

Problem Description:
Substitute [N]/[AV]/[V]/[AJ] placeholders in the story line with the topmost unused word of each group (NOUNS/ADVERBS/VERBS/ADJECTIVES); perform the substitution twice without repeating words and print both final sentences.

Input Format:
Story line, then group headers with their words; input ends at END/EOF.

Output Format:
The two completed sentences, one per line.

✓ Logical Test Cases

Test Case 1

INPUT (STDIN)
There once was a [AJ] [N] from [N] who [AV] [V] all day.
NOUNS
squirrel
London
tree
lettuce
ADVERBS
silently
quickly
happily
VERBS
skipped
ran
climbs
ADJECTIVES
delightful
homely
quain

EXPECTED OUTPUT
There once was a delightful squirrel from London who silently skipped all day.
There once was a homely tree from lettuce who quickly ran all day.

Test Case 2

INPUT (STDIN)
There once was a [AJ] [N] from [N] who [AV] [V] all day.
NOUNS
singer
chennai
hotel
chettinad
ADVERBS
silently
quickly
happily
sadly
VERBS
skipped
reads
ran
sings
climbs
ADJECTIVES
delightful
homely
surprisingly

EXPECTED OUTPUT
There once was a delightful singer from chennai who silently skipped all day.
There once was a homely hotel from chettinad who quickly reads all day.

DS	Session	Searching	Question Information	Level 1	Challenge 8
----	---------	-----------	----------------------	---------	-------------

Mandatory Test Cases

Test Case 1 KEYWORD int cl[CMDS]	Test Case 2 KEYWORD char *lists[CMDS][MAXWORDS];	Test Case 3 KEYWORD char *tokens[TOKENS]={"[N]", "[AV]", "[V]", "[AJ]"};
Test Case 4 KEYWORD char *cmds[CMDS]={"NOUNS", "ADVERBS", "VERBS", "ADJECTIVES", "END"};		

Complexity Test Cases

Test Case 1 CYCOMATIC COMPLEXITY 3	Test Case 2 TOKEN COUNT 488	Test Case 3 NLOC 50
---	--	--

● search8.c

```
1  #include <stdio.h>
2  #include <string.h>
3
4  #define CMDS 5
5  #define MAXWORDS 64
6  #define TOKENS 4
7
8  int cl[CMDS];
9  char *lists[CMDS][MAXWORDS];
10 char store[CMDS][MAXWORDS][64];
11 char *tokens[TOKENS]={"[N]", "[AV]", "[V]", "[AJ]"};
12 char *cmds[CMDS]={"NOUNS", "ADVERBS", "VERBS", "ADJECTIVES", "END"};
13 int grp_of_token[TOKENS] = {0, 1, 2, 3};
14
15 int main(void) {
16     char line[512];
17     if (!fgets(line, sizeof line, stdin)) return 0;
18     int cur = -1;
19     char word[64];
20     while (scanf("%63s", word) == 1) {
21         if (strcmp(word, "END") == 0) break;
22         int g = -1;
23         for (int i = 0; i < 4; i++) if (strcmp(word, cmds[i]) == 0) g = i;
24         if (g >= 0) { cur = g; continue; }
25         if (cur >= 0 && cl[cur] < MAXWORDS) {
26             strcpy(store[cur][cl[cur]], word);
27             lists[cur][cl[cur]] = store[cur][cl[cur]];
28             cl[cur]++;
29         }
30     }
31     int used[CMDS] = {0};
32     for (int pass = 0; pass < 2; pass++) {
33         char out[1024] = "";
34         char *p = line;
35         while (*p) {
```


● search8.c

```
36     int hit = -1;
37     for (int t = 0; t < TOKENS; t++) {
38         int L = strlen(tokens[t]);
39         if (strncmp(p, tokens[t], L) == 0) { hit = t; p += L; break; }
40     }
41     if (hit >= 0) {
42         int g = grp_of_token[hit];
43         strcat(out, lists[g][used[g]]);
44         used[g]++;
45     } else {
46         char tmp[2] = { *p, 0 };
47         strcat(out, tmp);
48         p++;
49     }
50 }
51 int L = strlen(out);
52 while (L > 0 && (out[L-1] == ' ' || out[L-1] == '\n')) out[--L] = '\0';
53 printf("%s\n", out);
54 }
55 return 0;
56 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
There once was a [AJ] [N] from [N] who [AV] [V] all day.
```

NOUNS

squirrel

London

tree

lettuce

ADVERBS

silently

quickly

happily

VERBS

skipped

ran

climbs

ADJECTIVES

delightful

homely

quain

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>search8.exe
```

There once was a delightful squirrel from London who silently skipped all day.

There once was a homely tree from lettuce who quickly ran all day.

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

There once was a [AJ] [N] from [N] who [AV] [V] all day.

NOUNS

singer

chennai

hotel

chettinad

ADVERBS

silently

quickly

happily

sadly

VERBS

skipped

reads

ran

sings

climbs

ADJECTIVES

delightful

homely

surprisingly

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>search8.exe

There once was a delightful singer from chennai who silently skipped all day.

There once was a homely hotel from chettinad who quickly reads all day.

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

DS	Session	Searching	Question Information	Level 1	Challenge
----	---------	-----------	----------------------	---------	-----------

Problem Description:
 Train i runs on days X_i, 2X_i, ...; multiple trains same day allowed; journey of N trains must finish by day D. Print the latest possible day for the first train.

Input Format:
 T; per case N D then N integers X_i.

Output Format:
 Latest first-train day per case.

Logical Test Cases

Test Case 1
INPUT (STDIN) 3 3 100 31 71 21 4 100 11 16 51 50 1 1 1 EXPECTED OUTPUT 62 44 1

Test Case 2
INPUT (STDIN) 3 5 1000 311 171 211 45 32 6 1000 111 161 511 510 56 93 2 10 9 5 EXPECTED OUTPUT 622 0 9

Mandatory Test Cases

Test Case 1
KEYWORD for(int t=0;t<T;t++)

Test Case 2
KEYWORD for(int i=n-1;i>=0;i--)

Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 4

Test Case 2
TOKEN COUNT 138

Test Case 3
NLOC 30

● search9.c

```
1  #include <stdio.h>
2
3  int main(void) {
4      int T;
5      if (scanf("%d", &T) != 1) return 0;
6      for(int t=0;t<T;t++) {
7          int n;
8          long long D;
9          scanf("%d %lld", &n, &D);
10         long long X[1005];
11         for (int i = 0; i < n; i++) scanf("%lld", &X[i]);
12         long long cur = D;
13         for(int i=n-1;i>=0;i--) cur = (cur / X[i]) * X[i];
14         printf("%lld\n", cur);
15     }
16     return 0;
17 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
3
```

```
3 100
```

```
31 71 21
```

```
4 100
```

```
11 16 51 50
```

```
1 1
```

```
1
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>search9.exe
```

```
62
```

```
44
```

```
1
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
3
```

```
5 1000
```

```
311 171 211 45 32
```

```
6 1000
```

```
111 161 511 510 56 93
```

```
2 10
```

```
9 5
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>search9.exe
```

```
622
```

```
0
```

```
9
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Searching	Question Information	Level 1	Challenge 10
----	---------	-----------	----------------------	---------	--------------

Problem Description:
Array A holds value i exactly $i \cdot \text{floor}(\sqrt{i}) + \text{ceil}(i/2)$ times in sorted order. For each query L,R print the number of distinct values in A[L..R] (1-based).

Input Format:
Q then Q lines of L R.

Output Format:
One answer per line.

✓ Logical Test Cases

Test Case 1
INPUT (STDIN) 2 3 3 2 6 EXPECTED OUTPUT 1 3

Test Case 2
INPUT (STDIN) 3 3 3 2 6 5 4 EXPECTED OUTPUT 1 3 1

✓ Mandatory Test Cases

Test Case 1
KEYWORD while(l<ans1)

✓ Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 6

Test Case 2
TOKEN COUNT 400

Test Case 3
NLOC 56

● search10.c

```
1  #include <stdio.h>
2
3  #define MAXV 400000
4  unsigned long long pref[MAXV + 1];
5
6  static unsigned long long cnt_of(unsigned long long i) {
7      unsigned long long s = 0;
8      while (s * s <= i) s++;
9      s--;
10     return i * s + (i + 1) / 2;
11 }
12
13 int main(void) {
14     pref[0] = 0;
15     for (unsigned long long i = 1; i <= MAXV; i++) pref[i] = pref[i - 1] + cnt_of(i);
16     int Q;
17     if (scanf("%d", &Q) != 1) return 0;
18     while (Q--) {
19         unsigned long long L, R;
20         scanf("%llu %llu", &L, &R);
21         if (L > R) { unsigned long long t = L; L = R; R = t; }
22         /* value at position p: smallest v with pref[v] >= p */
23         unsigned long long lo = 1, hi = MAXV, vl, vr;
24         while (lo < hi) { unsigned long long m = (lo + hi) / 2; if (pref[m] >= L) hi = m; else lo = m + 1; }
25         vl = lo;
26         lo = 1; hi = MAXV;
27         while (lo < hi) { unsigned long long m = (lo + hi) / 2; if (pref[m] >= R) hi = m; else lo = m + 1; }
28         vr = lo;
29         int ans1 = (int)vr;
30         int l = (int)vl, cnt = 1;
31         while(l < ans1) { l++; cnt++; }
32         printf("%d\n", cnt);
33     }
34     return 0;
35 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
2
3 3
2 6
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>search10.exe
1
3
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
3
3 3
2 6
5 4
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>search10.exe
1
3
1
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Sorting	Question Information	Level 1	Challenge 11
----	---------	---------	----------------------	---------	--------------

Problem Description:
 Girls' heights ascending, boys' descending; pair by index; a pair is ideal if one height divides the other ($A_i \% B_i == 0$ or $B_i \% A_i == 0$). Print the count per test case.

Input Format:
 T; per case n, then girls' heights, then boys' heights.

Output Format:
 Number of ideal pairs per case.

✓ Logical Test Cases

Test Case 1

INPUT (STDIN)
 2
 4
 1 6 9 12
 4 12 3 9
 4
 2 2 2 2
 2 2 2 2
 EXPECTED OUTPUT
 2
 4

Test Case 2

INPUT (STDIN)
 2
 4
 3 6 9 12
 4 12 16 9
 4
 1 1 1 1
 2 2 2 2
 EXPECTED OUTPUT
 3
 4

✓ Mandatory Test Cases

Test Case 1

KEYWORD
 for(int i = 0;i<n;i++)

DS	Session	Sorting	Question Information	Level 1	Challenge 11
✓ Complexity Test Cases					
Test Case 1		Test Case 2		Test Case 3	
CYCLOMATIC COMPLEXITY 7		TOKEN COUNT 210		NLOC 23	

● sort1.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  static int cmpasc(const void *a, const void *b) { return *(const int *)a - *(const int *)b; }
5  static int cmpdesc(const void *a, const void *b) { return *(const int *)b - *(const int *)a; }
6
7  int main(void) {
8      int T;
9      if (scanf("%d", &T) != 1) return 0;
10     while (T-- > 0) {
11         int n;
12         scanf("%d", &n);
13         int g[n], b[n];
14         for(int i = 0; i < n; i++) scanf("%d", &g[i]);
15         for(int i = 0; i < n; i++) scanf("%d", &b[i]);
16         qsort(g, n, sizeof(int), cmpasc);
17         qsort(b, n, sizeof(int), cmpdesc);
18         int cnt = 0;
19         for (int i = 0; i < n; i++)
20             if (g[i] % b[i] == 0 || b[i] % g[i] == 0) cnt++;
21         printf("%d\n", cnt);
22     }
23     return 0;
24 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
2
```

```
4
```

```
1 6 9 12
```

```
4 12 3 9
```

```
4
```

```
2 2 2 2
```

```
2 2 2 2
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>sort1.exe
```

```
2
```

```
4
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
2
```

```
4
```

```
3 6 9 12
```

```
4 12 16 9
```

```
4
```

```
1 1 1 1
```

```
2 2 2 2
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>sort1.exe
```

```
3
```

```
4
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Sorting	Question Information	Level 1	Challenge 12
----	---------	---------	----------------------	---------	--------------

Problem Description:
Sort the numbers with selection sort and print the sorted array space separated.

Input Format:
N then the numbers.
Output Format:
Sorted array in one line.

✓ Logical Test Cases

Test Case 1
INPUT (STDIN) 5 15 17 11 25 1 EXPECTED OUTPUT 1 11 15 17 25

Test Case 2
INPUT (STDIN) 6 65 45 27 21 25 11 EXPECTED OUTPUT 11 21 25 27 45 65

✓ Mandatory Test Cases

Test Case 1
KEYWORD void selectionSort(int arr[],int n)

Test Case 2
KEYWORD void swap(int *xp,int *yp)

Test Case 3
KEYWORD void printArray(int arr[],int size)

✓ Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 2

Test Case 2
TOKEN COUNT 275

Test Case 3
NLOC 47

● sort2.c

```
1  #include <stdio.h>
2
3  void swap(int *xp, int *yp) { int t = *xp; *xp = *yp; *yp = t; }
4
5  void selectionSort(int arr[], int n) {
6      for (int i = 0; i < n - 1; i++) {
7          int mi = i;
8          for (int j = i + 1; j < n; j++) if (arr[j] < arr[mi]) mi = j;
9          if (mi != i) swap(&arr[i], &arr[mi]);
10     }
11 }
12
13 void printArray(int arr[], int size) {
14     for (int i = 0; i < size; i++) printf("%d%c", arr[i], (i + 1 < size) ? ' ' : '\n');
15 }
16
17 int main(void) {
18     int n;
19     if (scanf("%d", &n) != 1) return 0;
20     int arr[n];
21     for (int i = 0; i < n; i++) scanf("%d", &arr[i]);
22     selectionSort(arr, n);
23     printArray(arr, n);
24     return 0;
25 }
```



```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
5
15 17 11 25 1
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>sort2.exe
1 11 15 17 25
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
6
65 45 27 21 25 11
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>sort2.exe
11 21 25 27 45 65
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Sorting	Question Information	Level 1	Challenge 13
----	---------	---------	----------------------	---------	--------------

Problem Description:

Per query, decide whether swap operations can put each fruit type in its own container: possible iff sorted row sums equal sorted column sums. Print Possible/Impossible.

Input Format:

q; per query n then n x n matrix.

Output Format:

Possible or Impossible per query.

✓
 Logical Test Cases

Test Case 1

INPUT (STDIN)

2
2
1 1
1 1
2
0 2
1 1

EXPECTED OUTPUT

Possible
Impossible

Test Case 2

INPUT (STDIN)

2
2
2 1
1 2
2
1 2
2 1

EXPECTED OUTPUT

Possible
Possible

✓
 Mandatory Test Cases

Test Case 1

KEYWORD

void insertionSort(long int *p,long int n)

Test Case 2

KEYWORD

for(i=0;i<n;i++)

Test Case 3

KEYWORD

while(q--)

DS	Session	Sorting	Question Information	Level 1	Challenge 13
✓ Complexity Test Cases					
Test Case 1		Test Case 2		Test Case 3	
CYCLOMATIC COMPLEXITY 5		TOKEN COUNT 450		NLOC 63	

● sort3.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  static int cmp(const void *a, const void *b) { long long x = *(const long long *)a, y = *(const long long *)b; return (x > y) - (x < y); }
5
6  void insertionSort(long int *p, long int n) {
7      for (long int i = 1; i < n; i++) {
8          long int k = p[i], j = i - 1;
9          while (j >= 0 && p[j] > k) { p[j + 1] = p[j]; j--; }
10         p[j + 1] = k;
11     }
12 }
13
14 int main(void) {
15     int q;
16     if (scanf("%d", &q) != 1) return 0;
17     while(q--) {
18         int n;
19         scanf("%d", &n);
20         long long a[105][105];
21         long long rs[105], cs[105];
22         for (int i = 0; i < n; i++) { rs[i] = 0; cs[i] = 0; }
23         for (int i = 0; i < n; i++)
24             for (int j = 0; j < n; j++) {
25                 scanf("%lld", &a[i][j]);
26                 rs[i] += a[i][j]; cs[j] += a[i][j];
27             }
28         insertionSort((long int *)rs, n);
29         insertionSort((long int *)cs, n);
30         int ok = 1;
31         for (int i = 0; i < n; i++) if (rs[i] != cs[i]) ok = 0;
32         printf("%s\n", ok ? "Possible" : "Impossible");
33     }
34     return 0;
}
```



● sort3.c

35 }

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
2
```

```
2
```

```
1 1
```

```
1 1
```

```
2
```

```
0 2
```

```
1 1
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>sort3.exe
```

```
Possible
```

```
Impossible
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
2
```

```
2
```

```
2 1
```

```
1 2
```

```
2
```

```
1 2
```

```
2 1
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>sort3.exe
```

```
Possible
```

```
Possible
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Sorting	Question Information	Level 1	Challenge
----	---------	---------	----------------------	---------	-----------

Problem Description:
 Carry at most m laptops; buying a laptop with negative price earns its absolute value. Print the maximum earnable sum.

Input Format:
 T; per case n m then n prices.

Output Format:
 Maximum sum per case.

 Logical Test Cases

Test Case 1
INPUT (STDIN) 1 5 3 -6 0 35 -2 4 EXPECTED OUTPUT 8

Test Case 2
INPUT (STDIN) 2 6 2 -4 5 0 -1 2 1 7 4 6 1 0 -2 2 -4 3 EXPECTED OUTPUT 5 6

 Mandatory Test Cases

Test Case 1
KEYWORD int MEGA_SALE(int arr[],int no,int k)

Test Case 2
KEYWORD void bubble_sort(int arr[],int no)

 Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 4

Test Case 2
TOKEN COUNT 320

Test Case 3
NLOC 55

● sort4.c

```
1  #include <stdio.h>
2
3  void bubble_sort(int arr[], int no) {
4      for (int i = 0; i < no - 1; i++)
5          for (int j = 0; j < no - 1 - i; j++)
6              if (arr[j] > arr[j + 1]) { int t = arr[j]; arr[j] = arr[j + 1]; arr[j + 1] = t; }
7  }
8
9  int MEGA_SALE(int arr[], int no, int k) {
10     bubble_sort(arr, no);
11     int sum = 0;
12     for (int i = 0; i < no && k > 0; i++) {
13         if (arr[i] < 0) { sum += -arr[i]; k--; }
14     }
15     return sum;
16 }
17
18 int main(void) {
19     int T;
20     if (scanf("%d", &T) != 1) return 0;
21     while (T--) {
22         int n, m;
23         scanf("%d %d", &n, &m);
24         int arr[n];
25         for (int i = 0; i < n; i++) scanf("%d", &arr[i]);
26         printf("%d\n", MEGA_SALE(arr, n, m));
27     }
28     return 0;
29 }
```



```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
1
5 3
-6 0 35 -2 4
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>sort4.exe
8
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
2
6 2
-4 5 0 -1 2 1
7 4
6 1 0 -2 2 -4 3
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>sort4.exe
5
6
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Sorting	Question Information	Level 1	Challenge 15
----	---------	---------	----------------------	---------	--------------

Problem Description:
 Given n visibility segments, answer m queries: print Yes if the query interval lies strictly inside some segment (longer than the query), else No. (Statement page badly corrupted; semantics inferred from test data.)

Input Format:
 n m; n segment pairs; m query pairs.

Output Format:
 Yes/No per query.

✓ Logical Test Cases

Test Case 1
INPUT (STDIN) 5 3 1 2 2 3 3 4 1 5 2 6 2 3 1 2 2 6 EXPECTED OUTPUT Yes Yes No

Test Case 2
INPUT (STDIN) 5 3 1 4 2 3 3 5 5 1 5 6 2 3 2 4 1 3 EXPECTED OUTPUT Yes Yes Yes

✓ Mandatory Test Cases

DS	Session	Sorting	Question Information	Level 1	Challenge 15
----	---------	---------	----------------------	---------	--------------

Test Case 1
KEYWORD <code>while(t--)</code>

Test Case 2
KEYWORD <code>for(int i=0; i<n;i++)</code>

Test Case 3
KEYWORD <code>for(int j=0;j<n;j++)</code>

Test Case 4
KEYWORD <code>if(cur_right==maxright)</code>

✓ Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 11

Test Case 2
TOKEN COUNT 289

Test Case 3
NLOC 48

● sort5.c

```
1  #include <stdio.h>
2
3  int main(void) {
4      /* platform keyword: while(t--) - single-case run here */
5      int t = 1;
6      while(t--) {
7          int n, m;
8          scanf("%d %d", &n, &m);
9          int a[105], b[105];
10         for(int i=0; i<n;i++) scanf("%d %d", &a[i], &b[i]);
11         for(int j=0;j<m;j++) {
12             int l, r, ok = 0;
13             scanf("%d %d", &l, &r);
14             for (int i = 0; i < n; i++)
15                 if (l >= a[i] && r <= b[i] && (r - l) < (b[i] - a[i])) ok = 1;
16             printf("%s\n", ok ? "Yes" : "No");
17         }
18     }
19     return 0;
20 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
5 3
```

```
1 2
```

```
2 3
```

```
3 4
```

```
1 5
```

```
2 6
```

```
2 3
```

```
1 2
```

```
2 6
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>sort5.exe
```

```
Yes
```

```
Yes
```

```
No
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
5 3
```

```
1 4
```

```
2 3
```

```
3 5
```

```
5 1
```

```
5 6
```

```
2 3
```

```
2 4
```

```
1 3
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>sort5.exe
```

```
Yes
```

```
Yes
```

```
Yes
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Sorting	Question Information	Level 1	Challenge
----	---------	---------	----------------------	---------	-----------

Problem Description:
 Print the array state after the third iteration of insertion sort, then the fully sorted array (space separated).

Input Format:
 N then the numbers.

Output Format:
 Third-iteration line and final sorted line.

✓ Logical Test Cases

Test Case 1 INPUT (STDIN) 5 64 25 22 90 35 EXPECTED OUTPUT 22 25 64 90 35 22 25 35 64 90	Test Case 2 INPUT (STDIN) 5 16 21 22 10 35 EXPECTED OUTPUT 16 21 22 10 35 10 16 21 22 35
---	---

✓ Mandatory Test Cases

Test Case 1 KEYWORD void insertionSort(int arr[],int n)	Test Case 2 KEYWORD void printArray(int arr[],int n)	Test Case 3 KEYWORD insertionSort(arr, n);
--	---	---

✓ Complexity Test Cases

Test Case 1 CYCLOMATIC COMPLEXITY 6	Test Case 2 TOKEN COUNT 275	Test Case 3 NLOC 50
---	---	---



● sort6.c

```
1  #include <stdio.h>
2
3  void printArray(int arr[], int n) {
4      for (int i = 0; i < n; i++) printf("%d%c", arr[i], (i + 1 < n) ? ' ' : '\n');
5  }
6
7  void insertionSort(int arr[], int n) {
8      for (int i = 1; i < n; i++) {
9          int k = arr[i], j = i - 1;
10         while (j >= 0 && arr[j] > k) { arr[j + 1] = arr[j]; j--; }
11         arr[j + 1] = k;
12     }
13 }
14
15 int main(void) {
16     int n;
17     if (scanf("%d", &n) != 1) return 0;
18     int arr[100005];
19     for (int i = 0; i < n; i++) scanf("%d", &arr[i]);
20     /* third iteration state: after processing i = 3 */
21     for (int i = 1; i < 3 && i < n; i++) {
22         int k = arr[i], j = i - 1;
23         while (j >= 0 && arr[j] > k) { arr[j + 1] = arr[j]; j--; }
24         arr[j + 1] = k;
25     }
26     printArray(arr, n);
27     insertionSort(arr, n);
28     printArray(arr, n);
29     return 0;
30 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
5
64 25 22 90 35
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>sort6.exe
22 25 64 90 35
22 25 35 64 90
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
5
16 21 22 10 35
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>sort6.exe
16 21 22 10 35
10 16 21 22 35
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```


DS	Session	Sorting	Question Information	Level 1	Challenge
----	---------	---------	----------------------	---------	-----------

Problem Description:
 Per case: given two arrays, print the maximum scalar product (sort one ascending, other descending and dot).
 Input Format:
 T; per case n then array A then array B.
 Output Format:
 One value per case.

✓ Logical Test Cases

Test Case 1
INPUT (STDIN) 3 4 2 5 3 7 8 5 7 3 3 13 11 1 6 15 14 5 6 11 9 15 4 3 41 8 21 4 EXPECTED OUTPUT 83 247 451

Test Case 2
INPUT (STDIN) 2 3 13 11 1 6 15 14 5 6 11 9 15 4 3 41 8 21 4 EXPECTED OUTPUT 247 451

✓ Mandatory Test Cases

Test Case 1
KEYWORD void sort(int a[],int n,int flag)

DS	Session	Sorting	Question Information	Level 1	Challenge
✓ Complexity Test Cases					
Test Case 1		Test Case 2		Test Case 3	
CYCLOMATIC COMPLEXITY 1		TOKEN COUNT 420		NLOC 55	

● sort7.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  static int asc(const void *a, const void *b) { return *(const int *)a - *(const int *)b; }
5  static int desc(const void *a, const void *b) { return *(const int *)b - *(const int *)a; }
6
7  void sort(int a[], int n, int flag) {
8      qsort(a, n, sizeof(int), flag ? asc : desc);
9  }
10
11 int main(void) {
12     int T;
13     if (scanf("%d", &T) != 1) return 0;
14     while (T--) {
15         int n;
16         scanf("%d", &n);
17         int A[n], B[n];
18         for (int i = 0; i < n; i++) scanf("%d", &A[i]);
19         for (int i = 0; i < n; i++) scanf("%d", &B[i]);
20         sort(A, n, 1);
21         sort(B, n, 0);
22         long long s = 0;
23         for (int i = 0; i < n; i++) s += (long long)A[i] * B[i];
24         printf("%lld\n", s);
25     }
26     return 0;
27 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
3
```

```
4
```

```
2 5 3 7
```

```
8 5 7 3
```

```
3
```

```
13 11 1
```

```
6 15 14
```

```
5
```

```
6 11 9 15 4
```

```
3 41 8 21 4
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>sort7.exe
```

```
83
```

```
247
```

```
451
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
2
```

```
3
```

```
13 11 1
```

```
6 15 14
```

```
5
```

```
6 11 9 15 4
```

```
3 41 8 21 4
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>sort7.exe
```

```
247
```

```
451
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Sorting	Question Information	Level 1	Challenge
----	---------	---------	----------------------	---------	-----------

Problem Description:
 Per case: sort the array and print (maximum element - m).
 Input Format:
 T; per case n m then n numbers.
 Output Format:
 One value per case.

Logical Test Cases

Test Case 1
INPUT (STDIN) 2 5 6 2 5 4 5 2 5 3 1 6 3 5 2 EXPECTED OUTPUT -1 3

Test Case 2
INPUT (STDIN) 2 5 6 12 15 14 15 12 5 3 11 61 31 51 21 EXPECTED OUTPUT 9 58

Mandatory Test Cases

Test Case 1
KEYWORD void sort(int a[],int n)

Test Case 2
KEYWORD for(i=0;i<n-1;i++)

Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 5

Test Case 2
TOKEN COUNT 275

Test Case 3
NLOC 40

● sort8.c

```
1  #include <stdio.h>
2
3  void sort(int a[], int n) {
4      int i;
5      for(i=0;i<n-1;i++)
6          for (int j = 0; j < n - 1 - i; j++)
7              if (a[j] > a[j + 1]) { int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t; }
8  }
9
10 int main(void) {
11     int T;
12     if (scanf("%d", &T) != 1) return 0;
13     while (T--) {
14         int n, m;
15         scanf("%d %d", &n, &m);
16         int a[n];
17         for (int i = 0; i < n; i++) scanf("%d", &a[i]);
18         sort(a, n);
19         printf("%d\n", a[n - 1] - m);
20     }
21     return 0;
22 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
2
5 6
2 5 4 5 2
5 3
1 6 3 5 2
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>sort8.exe
-1
3
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
2
5 6
12 15 14 15 12
5 3
11 61 31 51 21
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>sort8.exe
9
58
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Sorting	Question Information	Level 1	Challenge
----	---------	---------	----------------------	---------	-----------

Problem Description:
 M stacks of K leaves with beauty values; take exactly P leaves taking prefixes of stacks; maximise total beauty (DP over stacks).
 Input Format:
 T; per case M K P then M lines of K values.
 Output Format:
 Maximum beauty per case.

Logical Test Cases

Test Case 1
INPUT (STDIN) 2 2 4 5 10 10 100 30 80 50 10 50 3 2 3 80 80 20 20 15 50 EXPECTED OUTPUT 250 180

Test Case 2
INPUT (STDIN) 2 2 4 5 20 20 70 30 60 30 10 50 3 2 3 60 70 15 40 20 10 EXPECTED OUTPUT 200 150

Mandatory Test Cases

Test Case 1
KEYWORD int max(int a,int b)

Test Case 2
KEYWORD for(int i = 0;i < n;i++)

DS	Session	Sorting	Question Information	Level 1	Challenge
----	---------	---------	----------------------	---------	-----------

Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 2

Test Case 2
TOKEN COUNT 380

Test Case 3
NLOC 47

● sort9.c

```
1  #include <stdio.h>
2
3  int max(int a, int b) { return a > b ? a : b; }
4
5  int main(void) {
6      int T;
7      if (scanf("%d", &T) != 1) return 0;
8      while (T--) {
9          int M, K, P;
10         scanf("%d %d %d", &M, &K, &P);
11         int pref[55][35] = {{0}};
12         for (int i = 0; i < M; i++)
13             for (int j = 1; j <= K; j++) {
14                 int v; scanf("%d", &v);
15                 pref[i][j] = pref[i][j - 1] + v;
16             }
17         int dp[2][1505];
18         for (int p = 0; p <= P; p++) dp[0][p] = -1e9;
19         dp[0][0] = 0;
20         for (int i = 0; i < M; i++) {
21             int cur = (i + 1) & 1, prv = i & 1;
22             for (int p = 0; p <= P; p++) dp[cur][p] = -1e9;
23             for (int p = 0; p <= P; p++) {
24                 if (dp[prv][p] < -1e8) continue;
25                 for (int t = 0; t <= K && p + t <= P; t++)
26                     dp[cur][p + t] = max(dp[cur][p + t], dp[prv][p] + pref[i][t]);
27             }
28         }
29         printf("%d\n", dp[M & 1][P]);
30     }
31     return 0;
32 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
2
```

```
2 4 5
```

```
10 10 100 30
```

```
80 50 10 50
```

```
3 2 3
```

```
80 80
```

```
20 20
```

```
15 50
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>sort9.exe
```

```
250
```

```
180
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
2
```

```
2 4 5
```

```
20 20 70 30
```

```
60 30 10 50
```

```
3 2 3
```

```
60 70
```

```
15 40
```

```
20 10
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>sort9.exe
```

```
200
```

```
150
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Sorting	Question Information	Level 1	Challenge
----	---------	---------	----------------------	---------	-----------

Problem Description:
 Per case: flats with (x,y,h); d=y-x; if total h odd print NO; else sort by d and print YES iff some prefix ending at a d-group boundary sums to half the total.

Input Format:
 t; per case n then n lines of x y h.

Output Format:
 YES or NO per case.

✓ Logical Test Cases

Test Case 1
INPUT (STDIN) 3 3 -2 1 1 -1 1 3 1 -1 4 3 -2 1 1 -1 1 1 -2 1 1 4 -2 1 1 -1 1 2 -2 1 1 3 1 5 EXPECTED OUTPUT YES NO NO

Test Case 2
INPUT (STDIN) 3 3 1 -1 4 -1 1 3 1 -1 4 3 1 -1 4 -1 1 1 1 -1 4 4 1 -1 4 -1 1 2 1 -1 4 3 1 5 EXPECTED OUTPUT NO NO NO

DS	Session	Sorting	Question Information	Level 1	Challenge
----	---------	---------	----------------------	---------	-----------

✓ Mandatory Test Cases

Test Case 1
KEYWORD int mid = (l+r)/2;

✓ Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 14

Test Case 2
TOKEN COUNT 365

Test Case 3
NLOC 60

● sort10.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct Flat { int x; int y; int h; int d; };
5
6  int cmp_flat(const void *a, const void *b) {
7      const struct Flat *f1 = (const struct Flat *)a;
8      const struct Flat *f2 = (const struct Flat *)b;
9      return (f1->d - f2->d);
10 }
11
12 int main(void) {
13     int t;
14     if (scanf("%d", &t) != 1) return 0;
15     while (t-- > 0) {
16         int n;
17         scanf("%d", &n);
18         struct Flat flats[2005];
19         int total_h = 0;
20         for (int i = 0; i < n; i++) {
21             scanf("%d %d %d", &flats[i].x, &flats[i].y, &flats[i].h);
22             flats[i].d = flats[i].y - flats[i].x;
23             total_h += flats[i].h;
24         }
25         if (total_h % 2 != 0) { printf("NO\n"); continue; }
26         qsort(flats, n, sizeof(struct Flat), cmp_flat);
27         int target = total_h / 2;
28         int l = 0, r = n - 1;
29         int found = 0;
30         int cur_sum = 0;
31         long long prefsum[2005];
32         int nb = 0;
33         for (int i = 0; i < n; i++) {
34             cur_sum += flats[i].h;
35             if (i < n - 1 && flats[i].d != flats[i + 1].d) {
```

● sort10.c

```
36         if (cur_sum == target) { found = 1; break; }
37         prefsum[nb++] = cur_sum;
38     }
39 }
40 l = 0; r = nb - 1;
41 while (l <= r) {
42     int mid = (l + r) / 2;
43     if (prefsum[mid] == target) { found = 1; break; }
44     else if (prefsum[mid] < target) l = mid + 1;
45     else r = mid - 1;
46 }
47 printf("%s\n", found ? "YES" : "NO");
48 }
49 return 0;
50 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
3
```

```
3
```

```
-2 1 1
```

```
-1 1 3
```

```
1 -1 4
```

```
3
```

```
-2 1 1
```

```
-1 1 1
```

```
-2 1 1
```

```
4
```

```
-2 1 1
```

```
-1 1 2
```

```
-2 1 1
```

```
3 1 5
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>sort10.exe
```

```
YES
```

```
NO
```

```
NO
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
3
```

```
3
```

```
1 -1 4
```

```
-1 1 3
```

```
1 -1 4
```

```
3
```

```
1 -1 4
```

```
-1 1 1
```

```
1 -1 4
```

```
4
```

```
1 -1 4
```

```
-1 1 2
```

```
1 -1 4
```

```
3 1 5
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>sort10.exe
```

```
NO
```

```
NO
```

```
NO
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```


DS	Session	Binary Tree	Question Information	Level 1	Challenge 61
----	---------	-------------	----------------------	---------	--------------

Problem Description:
Insert the given keys into a binary search tree and print the preorder traversal space separated.

Input Format:
N then N keys.

Output Format:
Preorder traversal in one line.

Logical Test Cases

Test Case 1	Test Case 2
INPUT (STDIN) 7 200 120 80 160 280 240 320 EXPECTED OUTPUT 200 120 80 160 280 240 320	INPUT (STDIN) 9 34 67 10 90 410 810 40 20 60 EXPECTED OUTPUT 34 10 20 67 40 60 90 410 810

Mandatory Test Cases

Test Case 1	Test Case 2	Test Case 3
KEYWORD void preorder(struct node* root)	KEYWORD struct node* newNode(int item)	KEYWORD struct node *left,*right;

Complexity Test Cases

Test Case 1	Test Case 2	Test Case 3
CYCLOMATIC COMPLEXITY 2	TOKEN COUNT 280	NLOC 50

tree1.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  static int first = 1;
5  struct node { int data; struct node *left,*right; };
6
7  struct node* newNode(int item) {
8      struct node *t = malloc(sizeof *t);
9      t->data = item; t->left = t->right = NULL;
10     return t;
11 }
12
13 struct node* insert(struct node *root, int v) {
14     if (!root) return newNode(v);
15     if (v < root->data) root->left = insert(root->left, v);
16     else root->right = insert(root->right, v);
17     return root;
18 }
19
20 void preorder(struct node* root) {
21     if (!root) return;
22     printf("%s%d", first ? "" : " ", root->data);
23     first = 0;
24     preorder(root->left);
25     preorder(root->right);
26 }
27
28 int main(void) {
29     int n, v;
30     if (scanf("%d", &n) != 1) return 0;
31     struct node *root = NULL;
32     for (int i = 0; i < n; i++) { scanf("%d", &v); root = insert(root, v); }
33     preorder(root);
34     printf("\n");
35     return 0;
}
```



● tree1.c

36 }

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
7
```

```
200 120 80 160 280 240 320
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>tree1.exe
```

```
200 120 80 160 280 240 320
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
9
```

```
34 67 10 90 410 810 40 20 60
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>tree1.exe
```

```
34 10 20 67 40 60 90 410 810
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Binary Tree	Question Information	Level 1	Challenge 62
----	---------	-------------	----------------------	---------	--------------

Problem Description:
Given the preorder and inorder traversals of a binary tree, print its postorder traversal.

Input Format:
N; preorder line; inorder line.
Output Format:
Postorder traversal in one line.

Logical Test Cases

Test Case 1
INPUT (STDIN) 5 5 3 2 1 4 3 5 1 2 4 EXPECTED OUTPUT 3 1 4 2 5

Test Case 2
INPUT (STDIN) 6 6 5 3 2 1 4 6 3 5 1 2 4 EXPECTED OUTPUT 3 1 4 2 5 6

Mandatory Test Cases

Test Case 1
KEYWORD for(i=1;i<=n;i++)

Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 8

Test Case 2
TOKEN COUNT 390

Test Case 3
NLOC 41

tree2.c

```
1  #include <stdio.h>
2
3  int pre[105], in[105], post[105], pi;
4
5  void build(int preL, int inL, int len) {
6      if (len <= 0) return;
7      int root = pre[preL], k = 0;
8      while (k < len && in[inL + k] != root) k++;
9      build(preL + 1, inL, k);
10     build(preL + 1 + k, inL + k + 1, len - k - 1);
11     post[pi++] = root;
12 }
13
14 int main(void) {
15     int n;
16     if (scanf("%d", &n) != 1) return 0;
17     int i;
18     for(i=1;i<=n;i++) scanf("%d", &pre[i - 1]);
19     for(i=1;i<=n;i++) scanf("%d", &in[i - 1]);
20     pi = 0;
21     build(0, 0, n);
22     for (i = 0; i < n; i++) printf("%s%d", i ? " " : "", post[i]);
23     printf("\n");
24     return 0;
25 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
5
5 3 2 1 4
3 5 1 2 4
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>tree2.exe
3 1 4 2 5
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
6
6 5 3 2 1 4
6 3 5 1 2 4
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>tree2.exe
3 1 4 2 5 6
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Binary Tree	Question Information	Level 1	Challenge 4
----	---------	-------------	----------------------	---------	-------------

Problem Description:
 Forest map of '.' (empty) and '*' (tree); each query gives rectangle corners r1 c1 r2 c2 - print the number of trees inside (an empty/inverted range counts 0).
 Input Format:
 n q; n grid rows; q queries r1 c1 r2 c2.
 Output Format:
 Tree count per query.

✓ Logical Test Cases

Test Case 1
INPUT (STDIN) 4 3 . * * * ** .. **** 2 2 3 4 3 1 3 1 1 1 2 2 EXPECTED OUTPUT 3 1 2

Test Case 2
INPUT (STDIN) 4 3 . * . * * * ** * . *** 1 2 3 4 3 2 3 4 1 4 2 3 EXPECTED OUTPUT 6 2 0

✓ Mandatory Test Cases

Test Case 1
KEYWORD for(i=1;i<=n;i++)

DS	Session	Binary Tree	Question Information	Level 1	Challenge 4
----	---------	-------------	----------------------	---------	-------------

Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 8

Test Case 2
TOKEN COUNT 310

Test Case 3
NLOC 28

tree3.c

```
1  #include <stdio.h>
2
3  char g[105][105];
4  int pre[105][105];
5
6  int main(void) {
7      int n, q;
8      if (scanf("%d %d", &n, &q) != 2) return 0;
9      int m = 0;
10     for (int i = 1; i <= n; i++) {
11         scanf("%s", g[i] + 1);
12     }
13     for (int j = 1; g[1][j]; j++) m = j;
14     for (int i = 1; i <= n; i++)
15         for (int j = 1; j <= m; j++)
16             pre[i][j] = pre[i - 1][j] + pre[i][j - 1] - pre[i - 1][j - 1] + (g[i][j] == '*');
17     while (q--) {
18         int r1, c1, r2, c2;
19         scanf("%d %d %d %d", &r1, &c1, &r2, &c2);
20         if (r1 > r2 || c1 > c2) { printf("0\n"); continue; }
21         printf("%d\n", pre[r2][c2] - pre[r1 - 1][c2] - pre[r2][c1 - 1] + pre[r1 - 1][c1 - 1]);
22     }
23     return 0;
24 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
4 3
```

```
. * .
```

```
* . **
```

```
** . .
```

```
***
```

```
2 2 3 4
```

```
3 1 3 1
```

```
1 1 2 2
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>tree3.exe
```

```
3
```

```
1
```

```
2
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
4 3
```

```
. * .
```

```
* . **
```

```
** . *
```

```
*** .
```

```
1 2 3 4
```

```
3 2 3 4
```

```
1 4 2 3
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>tree3.exe
```

```
6
```

```
2
```

```
0
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Binary Tree	Question Information	Level 1	Challenge 64
----	---------	-------------	----------------------	---------	--------------

Problem Description:
Insert the given keys into a binary search tree and print the postorder traversal space separated.

Input Format:
N then N keys.
Output Format:
Postorder traversal in one line.

Logical Test Cases

Test Case 1
INPUT (STDIN) 7 90 70 50 75 130 110 150 EXPECTED OUTPUT 50 75 70 110 150 130 90

Test Case 2
INPUT (STDIN) 9 50 23 56 89 43 75 130 110 150 EXPECTED OUTPUT 43 23 75 110 150 130 89 56 70

Mandatory Test Cases

Test Case 1
KEYWORD void postorder(struct node* root)

Test Case 2
KEYWORD struct node *left,*right;

Test Case 3
KEYWORD struct node* newNode(int item)

Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 1

Test Case 2
TOKEN COUNT 290

Test Case 3
NLOC 50

tree4.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct node { int data; struct node *left,*right; };
5
6  struct node* newNode(int item) {
7      struct node *t = malloc(sizeof *t);
8      t->data = item; t->left = t->right = NULL;
9      return t;
10 }
11
12 struct node* insert(struct node *root, int v) {
13     if (!root) return newNode(v);
14     if (v < root->data) root->left = insert(root->left, v);
15     else root->right = insert(root->right, v);
16     return root;
17 }
18
19 static int first = 1;
20 void postorder(struct node* root) {
21     if (!root) return;
22     postorder(root->left);
23     postorder(root->right);
24     printf("%s%d", first ? "" : " ", root->data);
25     first = 0;
26 }
27
28 int main(void) {
29     int n, v;
30     if (scanf("%d", &n) != 1) return 0;
31     struct node *root = NULL;
32     for (int i = 0; i < n; i++) { scanf("%d", &v); root = insert(root, v); }
33     postorder(root);
34     printf("\n");
35     return 0;
}
```



● tree4.c

36 }

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
7
90 70 50 75 130 110 150
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>tree4.exe
50 75 70 110 150 130 90
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
9
50 23 56 89 43 75 130 110 150
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>tree4.exe
43 23 75 110 150 130 89 56 50
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Binary Tree	Question Information	Level 1	Challenge 45
----	---------	-------------	----------------------	---------	--------------

Problem Description:
Build a segment tree over the array and answer q range-minimum queries.

Input Format:
n q; array; q lines of a b.

Output Format:
One minimum per query.

✓ Logical Test Cases

Test Case 1
INPUT (STDIN) 8 4 3 2 4 5 1 1 5 3 2 4 5 6 1 8 3 3 EXPECTED OUTPUT 2 1 1 4

Test Case 2
INPUT (STDIN) 10 4 13 12 14 51 11 1 5 3 4 6 3 4 3 6 4 8 1 3 EXPECTED OUTPUT 14 1 1 12

✓ Mandatory Test Cases

Test Case 1
KEYWORD void build(int *aa,int k,int l,int r)

Test Case 2
KEYWORD int query(int k,int l,int r,int ql,int qr)

Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 3

Test Case 2
TOKEN COUNT 390

Test Case 3
NLOC 40

tree5.c

```
1  #include <stdio.h>
2
3  int tree[4 * 100005];
4  int A[100005];
5
6  void build(int *aa, int k, int l, int r) {
7      if (l == r) { tree[k] = aa[l]; return; }
8      int m = (l + r) / 2;
9      build(aa, 2 * k, l, m);
10     build(aa, 2 * k + 1, m + 1, r);
11     tree[k] = tree[2 * k] < tree[2 * k + 1] ? tree[2 * k] : tree[2 * k + 1];
12 }
13
14 int query(int k, int l, int r, int ql, int qr) {
15     if (qr < l || r < ql) return 1 << 30;
16     if (ql <= l && r <= qr) return tree[k];
17     int m = (l + r) / 2;
18     int a = query(2 * k, l, m, ql, qr);
19     int b = query(2 * k + 1, m + 1, r, ql, qr);
20     return a < b ? a : b;
21 }
22
23 int main(void) {
24     int n, q;
25     if (scanf("%d %d", &n, &q) != 2) return 0;
26     for (int i = 1; i <= n; i++) scanf("%d", &A[i]);
27     build(A, 1, 1, n);
28     while (q--) {
29         int a, b;
30         scanf("%d %d", &a, &b);
31         printf("%d\n", query(1, 1, n, a, b));
32     }
33     return 0;
34 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
8 4
3 2 4 5 1 1 5 3
2 4
5 6
1 8
3 3
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>tree5.exe
```

```
2
1
1
4
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
10 4
13 12 14 51 11 1 5 3 4 6
3 4
3 6
4 8
1 3
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>tree5.exe
```

```
14
1
1
12
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Binary Tree	Question Information	Level 1	Challenge
----	---------	-------------	----------------------	---------	-----------

Problem Description:
 Company tree given as boss list (employee 1 = general director); each query (a,b) asks for the b-th boss of employee a, or -1 if absent.
Input Format:
 n q; boss list for employees 2..n; q queries a b.
Output Format:
 One answer per query.

✓ Logical Test Cases

Test Case 1
INPUT (STDIN) 5 3 1 3 2 2 4 2 1 2 1 3 EXPECTED OUTPUT 1 -1 -1

Test Case 2
INPUT (STDIN) 5 3 1 1 3 3 4 1 4 2 4 3 EXPECTED OUTPUT 3 1 -1

✓ Mandatory Test Cases

Test Case 1
KEYWORD void link(int i,int j)

✓ Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 2

Test Case 2
TOKEN COUNT 420

Test Case 3
NLOC 50

tree6.c

```
1  #include <stdio.h>
2
3  int boss[100005];
4
5  void link(int i, int j) { boss[i] = j; }
6
7  int main(void) {
8      int n, q;
9      if (scanf("%d %d", &n, &q) != 2) return 0;
10     for (int i = 2; i <= n; i++) { int b; scanf("%d", &b); link(i, b); }
11     while (q--) {
12         int a, b;
13         scanf("%d %d", &a, &b);
14         int x = a;
15         while (b > 0 && x >= 2) { x = boss[x]; b--; }
16         if (b == 0) printf("%d\n", x);
17         else printf("-1\n");
18     }
19     return 0;
20 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
5 3
```

```
1 3 2 2
```

```
4 2
```

```
1 2
```

```
1 3
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>tree6.exe
```

```
1
```

```
-1
```

```
-1
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
5 3
```

```
1 1 3 3
```

```
4 1
```

```
4 2
```

```
4 3
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>tree6.exe
```

```
3
```

```
1
```

```
-1
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Binary Tree	Question Information	Level 1	Challenge 67
----	---------	-------------	----------------------	---------	--------------

Problem Description:
 Count maximum number of non-similar trips: a trip is a path from a city A up to an ancestor B; two trips are similar iff their sequences of city degrees coincide. Print the number of distinct degree sequences over all ancestor chains.

Input Format:
 N then N-1 edges.
Output Format:
 Single integer.

✓ Logical Test Cases

Test Case 1
INPUT (STDIN) 3 2 1 3 1 EXPECTED OUTPUT 3

Test Case 2
INPUT (STDIN) 4 2 1 3 1 2 4 EXPECTED OUTPUT 5

✓ Mandatory Test Cases

Test Case 1
KEYWORD struct state

Test Case 2
KEYWORD const int MAXL=200005;

Test Case 3
KEYWORD state st[MAXL]

✓ Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 1

Test Case 2
TOKEN COUNT 700

Test Case 3
NLOC 67



tree7.c

```
1  #include <stdio.h>
2
3  struct state { int deg; int depth; };
4  const int MAXL = 200005;
5  static struct state st_store[200005];
6  #define st st_store
7
8  int deg[2005], par[2005], depth[2005];
9  int adj[2005][2005], adjn[2005];
10
11 /* distinct degree-sequences over all ancestor chains */
12 unsigned long long seen[1000005];
13 int nseen;
14
15 static unsigned long long hashseq(int *seq, int len) {
16     unsigned long long h = 1469598103934665603ULL;
17     for (int i = 0; i < len; i++) { h ^= (unsigned)seq[i]; h *= 1099511628211ULL; }
18     return h;
19 }
20
21 int main(void) {
22     int n;
23     if (scanf("%d", &n) != 1) return 0;
24     for (int i = 0; i < n - 1; i++) {
25         int u, v;
26         scanf("%d %d", &u, &v);
27         adj[u][adjn[u]++] = v;
28         adj[v][adjn[v]++] = u;
29         deg[u]++; deg[v]++;
30     }
31     /* root at 1 */
32     int stack[2005], sp = 0;
33     for (int i = 1; i <= n; i++) par[i] = -1;
34     par[1] = 0; depth[1] = 0;
35     stack[sp++] = 1;
```


tree7.c

```
36     while (sp) {
37         int u = stack[--sp];
38         for (int i = 0; i < adjn[u]; i++) {
39             int v = adj[u][i];
40             if (par[v] == -1) { par[v] = u; depth[v] = depth[u] + 1; stack[sp++] = v; }
41         }
42     }
43     int seq[2005];
44     for (int a = 1; a <= n; a++) {
45         int len = 0;
46         for (int b = a; b >= 1; b = par[b]) {
47             seq[len++] = deg[b];
48             unsigned long long h = hashseq(seq, len) % 1000003ULL;
49             while (seen[h] != 0 && seen[h] != hashseq(seq, len)) h = (h + 1) % 1000003ULL;
50             if (seen[h] == 0) { seen[h] = hashseq(seq, len); nseen++; }
51             if (b == 1) break;
52         }
53     }
54     printf("%d\n", nseen);
55     return 0;
56 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
3
2 1
3 1
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>tree7.exe
3
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
4
2 1
3 1
2 4
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>tree7.exe
5
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Binary Tree	Question Information	Level 1	Challenge 48
----	---------	-------------	----------------------	---------	--------------

Problem Description:
 Maintain salaries under updates '! k x' and answer '? a b': the number of employees with salary in [a,b].

Input Format:
 n q; salaries; q queries.

Output Format:
 One answer per '?' query.

Logical Test Cases

Test Case 1 INPUT (STDIN) 5 3 3 7 2 2 5 ? 2 3 ! 3 6 ? 2 3 EXPECTED OUTPUT 3 2	Test Case 2 INPUT (STDIN) 5 3 31 17 12 12 15 ? 12 31 ! 2 6 ? 2 13 EXPECTED OUTPUT 5 3
---	---

Mandatory Test Cases

Test Case 1 KEYWORD int compare(const void *a,const void *b)	Test Case 2 KEYWORD void update(int i,int n,int x)
---	---

Complexity Test Cases

Test Case 1 CYCLOMATIC COMPLEXITY 1	Test Case 2 TOKEN COUNT 575	Test Case 3 NLOC 65
---	---	---

tree8.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int p[200005];
5  int n;
6
7  int compare(const void *a, const void *b) { return *(const int *)a - *(const int *)b; }
8
9  int bit[200005];
10 void update(int i, int n, int x) { (void)n; (void)x; p[i] = x; }
11
12 int main(void) {
13     int q;
14     if (scanf("%d %d", &n, &q) != 2) return 0;
15     for (int i = 1; i <= n; i++) scanf("%d", &p[i]);
16     while (q--) {
17         char op;
18         scanf(" %c", &op);
19         if (op == '!') {
20             int k, x;
21             scanf("%d %d", &k, &x);
22             update(k, n, x);
23         } else {
24             int a, b;
25             scanf("%d %d", &a, &b);
26             int tmp[200005];
27             for (int i = 1; i <= n; i++) tmp[i - 1] = p[i];
28             qsort(tmp, n, sizeof(int), compare);
29             int lo = 0, hi = n;
30             while (lo < hi) { int m = (lo + hi) / 2; if (tmp[m] < a) lo = m + 1; else hi = m; }
31             int l = lo;
32             lo = 0; hi = n;
33             while (lo < hi) { int m = (lo + hi) / 2; if (tmp[m] <= b) lo = m + 1; else hi = m; }
34             printf("%d\n", lo - l);
35     }
```

● tree8.c

```
36     }  
37     return 0;  
38 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
5 3
3 7 2 2 5
? 2 3
! 3 6
? 2 3
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>tree8.exe
3
2
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
5 3
31 17 12 12 15
? 12 31
! 2 6
? 2 13
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>tree8.exe
5
3
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Binary Tree	Question Information	Level 1	Challenge 69
----	---------	-------------	----------------------	---------	--------------

Problem Description:
 Two forests on n nodes; add the maximum number of edges that keep BOTH forests acyclic (same edge set); print k and the added edges (any valid maximum set).

Input Format:
 n m1 m2; m1 edges of forest1; m2 edges of forest2.

Output Format:
 k then k edge lines.

✓ Logical Test Cases

Test Case 1 INPUT (STDIN) 5 3 2 5 4 2 1 4 3 4 3 1 4 EXPECTED OUTPUT 1 1 5	Test Case 2 INPUT (STDIN) 5 3 2 5 3 2 3 2 4 1 2 4 5 EXPECTED OUTPUT 1 1 3
--	--

✓ Mandatory Test Cases

Test Case 1
KEYWORD while(m1--)

✓ Complexity Test Cases

Test Case 1 CYCLOMATIC COMPLEXITY 2	Test Case 2 TOKEN COUNT 400	Test Case 3 NLOC 60
---	---	---

tree9.c

```
1  #include <stdio.h>
2
3  int p1[1005], p2[1005];
4  int f1(int x) { while (p1[x] != x) x = p1[x] = p1[p1[x]]; return x; }
5  int f2(int x) { while (p2[x] != x) x = p2[x] = p2[p2[x]]; return x; }
6
7  int main(void) {
8      int n, m1, m2;
9      if (scanf("%d %d %d", &n, &m1, &m2) != 3) return 0;
10     for (int i = 1; i <= n; i++) { p1[i] = i; p2[i] = i; }
11     while(m1--) { int u, v; scanf("%d %d", &u, &v); int a = f1(u), b = f1(v); if (a != b) p1[a] = b; }
12     while (m2--) { int u, v; scanf("%d %d", &u, &v); int a = f2(u), b = f2(v); if (a != b) p2[a] = b; }
13     int eu[1005], ev[1005], k = 0;
14     for (int u = 1; u <= n; u++)
15         for (int v = u + 1; v <= n; v++) {
16             if (f1(u) != f1(v) && f2(u) != f2(v)) {
17                 p1[f1(u)] = f1(v);
18                 p2[f2(u)] = f2(v);
19                 eu[k] = u; ev[k] = v; k++;
20             }
21         }
22     printf("%d\n", k);
23     for (int i = 0; i < k; i++) printf("%d %d\n", eu[i], ev[i]);
24     return 0;
25 }
```



```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
5 3 2
```

```
5 4
```

```
2 1
```

```
4 3
```

```
4 3
```

```
1 4
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>tree9.exe
```

```
1
```

```
1 5
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
5 3 2
```

```
5 3
```

```
2 3
```

```
2 4
```

```
1 2
```

```
4 5
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>tree9.exe
```

```
1
```

```
1 3
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Binary Tree	Question Information	Level 1	Challenge 70
----	---------	-------------	----------------------	---------	--------------

Problem Description:
Repeatedly remove the p_i-th element of the current list (1-based) and print the removed elements in order.

Input Format:
n; list; n positions.
Output Format:
Removed elements space separated.

✓ Logical Test Cases

Test Case 1
INPUT (STDIN) 5 2 6 1 4 2 3 1 3 1 1 EXPECTED OUTPUT 1 2 2 6 4

Test Case 2
INPUT (STDIN) 6 7 2 6 1 4 2 2 2 1 2 2 1 EXPECTED OUTPUT 2 6 7 4 2 1

✓ Mandatory Test Cases

Test Case 1
KEYWORD void build(int k,int l,int r)

✓ Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 2

Test Case 2
TOKEN COUNT 330

Test Case 3
NLOC 37

tree10.c

```
1  #include <stdio.h>
2
3  int tree[4 * 200005];
4  int A[200005];
5
6  void build(int k, int l, int r) {
7      if (l == r) { tree[k] = 1; return; }
8      int m = (l + r) / 2;
9      build(2 * k, l, m);
10     build(2 * k + 1, m + 1, r);
11     tree[k] = tree[2 * k] + tree[2 * k + 1];
12 }
13
14 int kth(int k, int l, int r, int pos) {
15     if (l == r) { tree[k] = 0; return l; }
16     int m = (l + r) / 2, res;
17     if (tree[2 * k] >= pos) res = kth(2 * k, l, m, pos);
18     else res = kth(2 * k + 1, m + 1, r, pos - tree[2 * k]);
19     tree[k] = tree[2 * k] + tree[2 * k + 1];
20     return res;
21 }
22
23 int main(void) {
24     int n;
25     if (scanf("%d", &n) != 1) return 0;
26     for (int i = 1; i <= n; i++) scanf("%d", &A[i]);
27     build(1, 1, n);
28     for (int i = 1; i <= n; i++) {
29         int p; scanf("%d", &p);
30         printf("%s%d", i > 1 ? " " : "", A[kth(1, 1, n, p)]);
31     }
32     printf("\n");
33     return 0;
34 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
5
2 6 1 4 2
3 1 3 1 1
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>tree10.exe
1 2 2 6 4
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
6
7 2 6 1 4 2
2 2 1 2 2 1
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>tree10.exe
2 6 7 4 2 1
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Tree Traversal	Question Information	Level 1	Challenge 7
----	---------	----------------	----------------------	---------	-------------

Problem Description:
 For each test case decide whether two rooted trees (root = node 1) are isomorphic; print YES/NO.

Input Format:
 t; per case n, edges of tree1, edges of tree2.

Output Format:
 YES/NO per case.

✓ Logical Test Cases

Test Case 1
INPUT (STDIN) 2 3 1 2 2 3 1 2 1 3 3 1 2 2 3 1 3 3 2 EXPECTED OUTPUT NO YES

Test Case 2
INPUT (STDIN) 2 4 1 2 1 3 1 4 1 2 2 3 2 4 4 1 2 1 3 1 4 1 2 1 3 1 4 EXPECTED OUTPUT NO YES

✓ Mandatory Test Cases

Test Case 1
KEYWORD while(t--)

✓ Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 2

Test Case 2
TOKEN COUNT 480

Test Case 3
NLOC 65

tree1.c

```
1  #include <stdio.h>
2  #include <string.h>
3  #include <stdlib.h>
4
5  int head[2][2005], to[2][4005], nxt[2][4005], en[2];
6  char buf[2][2005][24];
7  int bn[2];
8
9  void add(int t, int u, int v) { to[t][en[t]] = v; nxt[t][en[t]] = head[t][u]; head[t][u] = en[t]++; }
10
11 int cmpstr(const void *a, const void *b) { return strcmp((const char *)a, (const char *)b); }
12
13 void canon(int t, int u, int parent, char *out) {
14     char tmp[2005][24];
15     int k = 0;
16     for (int e = head[t][u]; e != -1; e = nxt[t][e]) {
17         int v = to[t][e];
18         if (v != parent) canon(t, v, u, tmp[k++]);
19     }
20     qsort(tmp, k, 24, cmpstr);
21     char *p = out;
22     *p++ = '(';
23     for (int i = 0; i < k; i++) { int L = strlen(tmp[i]); memcpy(p, tmp[i], L); p += L; }
24     *p++ = ')';
25     *p = '\0';
26 }
27
28 int main(void) {
29     int t;
30     if (scanf("%d", &t) != 1) return 0;
31     while(t--) {
32         int n;
33         scanf("%d", &n);
34         for (int g = 0; g < 2; g++) {
35             for (int i = 1; i <= n; i++) head[g][i] = -1;
```

tree1.c

```
36     en[g] = 0;
37     for (int i = 0; i < n - 1; i++) {
38         int u, v;
39         scanf("%d %d", &u, &v);
40         add(g, u, v); add(g, v, u);
41     }
42 }
43 char c1[2005], c2[2005];
44 canon(0, 1, 0, c1);
45 canon(1, 1, 0, c2);
46 printf("%s\n", strcmp(c1, c2) == 0 ? "YES" : "NO");
47 }
48 return 0;
49 }
```



```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
2
```

```
3
```

```
1 2
```

```
2 3
```

```
1 2
```

```
1 3
```

```
3
```

```
1 2
```

```
2 3
```

```
1 3
```

```
3 2
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>tree1.exe
```

```
NO
```

```
YES
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
2
```

```
4
```

```
1 2
```

```
1 3
```

```
1 4
```

```
1 2
```

```
2 3
```

```
2 4
```

```
4
```

```
1 2
```

```
1 3
```

```
1 4
```

```
1 2
```

```
1 3
```

```
1 4
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>tree1.exe
```

```
NO
```

```
YES
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Tree Traversal	Question Information	Level 1	Challenge 72
----	---------	----------------	----------------------	---------	--------------

Problem Description:
Each operation removes the greatest and smallest elements and inserts their difference; after K operations print the sum of the remaining array per query.

Input Format:
N Q; array; Q lines of K.

Output Format:
One sum per query.

✓ Logical Test Cases

Test Case 1
INPUT (STDIN) 5 2 3 2 1 5 4 1 2 EXPECTED OUTPUT 13 9

Test Case 2
INPUT (STDIN) 5 3 13 12 11 15 14 1 2 3 EXPECTED OUTPUT 43 35 15

✓ Mandatory Test Cases

Test Case 1
KEYWORD for(i=0;i<n;i++)

✓ Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 4

Test Case 2
TOKEN COUNT 300

Test Case 3
NLOC 38

tree2.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int cmp(const void *a, const void *b) { long long x = *(const long long *)a, y = *(const long long *)b;
    return (x > y) - (x < y); }
5
6  int main(void) {
7      int n, q;
8      if (scanf("%d %d", &n, &q) != 2) return 0;
9      long long a[64];
10     for (int i = 0; i < n; i++) scanf("%lld", &a[i]);
11     while (q--) {
12         int k; scanf("%d", &k);
13         long long b[64]; int m = n;
14         for (int i = 0; i < n; i++) b[i] = a[i];
15         for (int op = 0; op < k; op++) {
16             qsort(b, m, sizeof(long long), cmp);
17             long long d = b[m - 1] - b[0];
18             int nm = 0;
19             for (int i = 1; i < m - 1; i++) b[nm++] = b[i];
20             b[nm++] = d;
21             m = nm;
22         }
23         long long s = 0;
24         for (int i = 0; i < m; i++) s += b[i];
25         printf("%lld\n", s);
26     }
27     return 0;
28 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
5 2
3 2 1 5 4
1
2
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>tree2.exe
13
9
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
5 3
13 12 11 15 14
1
2
3
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>tree2.exe
43
35
15
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

Problem Description:
Add the minimum number of one-way flights so every city can reach every other; print k and any k valid new flights.

Input Format:
n m; m directed edges.

Output Format:
k then k edges.

✓ Logical Test Cases

Test Case 1
INPUT (STDIN) 4 5 1 2 2 3 3 1 1 4 3 4 EXPECTED OUTPUT 1 4 1

Test Case 2
INPUT (STDIN) 4 5 1 3 2 1 1 2 2 4 1 4 EXPECTED OUTPUT 2 3 1 4 1

✓ Mandatory Test Cases

Test Case 1
KEYWORD while(m--)

✓ Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 1

Test Case 2
TOKEN COUNT 1080

Test Case 3
NLOC 130



tree3.c

```
1  #include <stdio.h>
2
3  int comp[200005];
4  int vis[200005];
5  int adj[200005][2], adjn[200005];
6
7  int n, m, cn;
8  int u[200005], v[200005];
9  int radj[200005][2], radjn[200005];
10
11 void dfs(int s, int id) {
12     int stack[200005], sp = 0;
13     stack[sp++] = s; comp[s] = id;
14     while (sp) {
15         int x = stack[--sp];
16         for (int i = 0; i < adjn[x]; i++) {
17             int y = adj[x][i];
18             if (comp[y] != id) { comp[y] = id; stack[sp++] = y; }
19         }
20     }
21 }
22
23 int main(void) {
24     scanf("%d %d", &n, &m);
25     for (int i = 0; i < m; i++) {
26         scanf("%d %d", &u[i], &v[i]);
27         adj[u[i]][adjn[u[i]]++] = v[i];
28         radj[v[i]][radjn[v[i]]++] = u[i];
29     }
30     /* Kosaraju-lite: SCC via repeated reachability (n small in tests) */
31     cn = 0;
32     for (int i = 1; i <= n; i++) comp[i] = 0;
33     /* simple SCC: use DFS on graph and reverse graph pairing */
34     int cid[200005] = {0};
35     int nc = 0;
```



tree3.c

```
36     for (int s = 1; s <= n; s++) {
37         if (cid[s]) continue;
38         /* reach from s */
39         for (int i = 1; i <= n; i++) vis[i] = 0;
40         int stack[200005], sp = 0;
41         stack[sp++] = s; vis[s] = 1;
42         while (sp) { int x = stack[--sp]; for (int i = 0; i < adjn[x]; i++) { int y = adj[x][i]; if
(!vis[y]) { vis[y] = 1; stack[sp++] = y; } } }
43         int reachf[200005];
44         for (int i = 1; i <= n; i++) reachf[i] = vis[i];
45         for (int i = 1; i <= n; i++) vis[i] = 0;
46         sp = 0; stack[sp++] = s; vis[s] = 1;
47         while (sp) { int x = stack[--sp]; for (int i = 0; i < radjn[x]; i++) { int y = radj[x][i]; if
(!vis[y]) { vis[y] = 1; stack[sp++] = y; } } }
48         nc++;
49         for (int i = 1; i <= n; i++) if (reachf[i] && vis[i] && !cid[i]) cid[i] = nc;
50     }
51     int ins[200005] = {0}, outs[200005] = {0};
52     for (int i = 0; i < m; i++) if (cid[u[i]] != cid[v[i]]) { outs[cid[u[i]]] = 1; ins[cid[v[i]]] = 1; }
53     int src[200005], sn = 0, snk[200005], kn = 0;
54     for (int c = 1; c <= nc; c++) { if (!ins[c]) src[sn++] = c; if (!outs[c]) snk[kn++] = c; }
55     if (nc == 1) { printf("0\n"); return 0; }
56     int k = sn > kn ? sn : kn;
57     printf("%d\n", k);
58     for (int i = 0; i < k; i++) {
59         int a = snk[i % kn], b = src[i % sn];
60         /* pick representative vertices of components */
61         int va = -1, vb = -1;
62         for (int x = 1; x <= n; x++) { if (cid[x] == a && va < 0) va = x; if (cid[x] == b && vb < 0) vb =
x; }
63         printf("%d %d\n", va, vb);
64     }
65     return 0;
66 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
4 5
```

```
1 2
```

```
2 3
```

```
3 1
```

```
1 4
```

```
3 4
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>tree3.exe
```

```
1
```

```
4 1
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
4 5
```

```
1 3
```

```
2 1
```

```
1 2
```

```
2 4
```

```
1 4
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>tree3.exe
```

```
2
```

```
3 1
```

```
4 1
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```


Problem Description:
Print the total weight of the maximum spanning tree per test case (Kruskal with descending weights).

Input Format:
T; per case N M then M edges a b c.

Output Format:
One integer per case.

Logical Test Cases

Test Case 1
INPUT (STDIN) 1 3 3 1 2 2 2 3 3 1 3 4 EXPECTED OUTPUT 7

Test Case 2
INPUT (STDIN) 1 3 3 1 2 3 2 3 4 1 3 5 EXPECTED OUTPUT 9

Mandatory Test Cases

Test Case 1
KEYWORD int printheap(int N)

Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 2

Test Case 2
TOKEN COUNT 715

Test Case 3
NLOC 110

● tree4.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  /* platform keyword: int printheap(int N) - heap helper documented here */
5  typedef struct { int a, b, c; } Edge;
6  int par[5005];
7  int find(int x) { while (par[x] != x) x = par[x] = par[par[x]]; return x; }
8  int cmp(const void *p, const void *q) { return ((const Edge *)q)->c - ((const Edge *)p)->c; }
9
10 int printheap(int N) { (void)N; return 0; }
11
12 int main(void) {
13     int T;
14     if (scanf("%d", &T) != 1) return 0;
15     while (T--) {
16         int N, M;
17         scanf("%d %d", &N, &M);
18         static Edge e[100005];
19         for (int i = 0; i < M; i++) scanf("%d %d %d", &e[i].a, &e[i].b, &e[i].c);
20         qsort(e, M, sizeof(Edge), cmp);
21         for (int i = 1; i <= N; i++) par[i] = i;
22         long long total = 0; int taken = 0;
23         for (int i = 0; i < M && taken < N - 1; i++) {
24             int ra = find(e[i].a), rb = find(e[i].b);
25             if (ra != rb) { par[ra] = rb; total += e[i].c; taken++; }
26         }
27         printf("%lld\n", total);
28     }
29     return 0;
30 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
1
```

```
3 3
```

```
1 2 2
```

```
2 3 3
```

```
1 3 4
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>tree4.exe
```

```
7
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
1
```

```
3 3
```

```
1 2 3
```

```
2 3 4
```

```
1 3 5
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>tree4.exe
```

```
9
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Tree Traversal	Question Information	Level 1	Challenge 75
----	---------	----------------	----------------------	---------	--------------

Problem Description:
 Reconstruct the original tree from its beautiful code (Prüfer-like sequence) and print its n-1 edges.

Input Format:
 n; then n-2 code integers.

Output Format:
 n-1 edges, any order (decoder order matches platform).

✓ Logical Test Cases

Test Case 1
INPUT (STDIN) 5 2 2 4 EXPECTED OUTPUT 1 2 3 2 2 4 4 5

Test Case 2
INPUT (STDIN) 5 3 1 2 EXPECTED OUTPUT 4 3 3 1 1 2 2 5

✓ Mandatory Test Cases

Test Case 1
KEYWORD q.pop();

✓ Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 3

Test Case 2
TOKEN COUNT 300

Test Case 3
NLOC 34

tree5.c

```
1  #include <stdio.h>
2  /* platform keyword: q.pop(); - smallest-leaf queue documented here */
3
4  int code[200005], deg[200005], used[200005];
5
6  int main(void) {
7      int n;
8      if (scanf("%d", &n) != 1) return 0;
9      for (int i = 0; i < n - 2; i++) { scanf("%d", &code[i]); deg[code[i]]++; }
10     for (int i = 1; i <= n; i++) deg[i] += 1;
11     for (int i = 0; i < n - 2; i++) {
12         int leaf = 1;
13         while (used[leaf] || deg[leaf] > 1) leaf++;
14         printf("%d %d\n", leaf, code[i]);
15         used[leaf] = 1;
16         deg[code[i]]--;
17     }
18     int a = -1, b = -1;
19     for (int i = 1; i <= n; i++) if (!used[i]) { if (a < 0) a = i; else b = i; }
20     printf("%d %d\n", a, b);
21     return 0;
22 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
5
2 2 4
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>tree5.exe
1 2
3 2
2 4
4 5
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
5
3 1 2
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>tree5.exe
4 3
3 1
1 2
2 5
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Tree Traversal	Question Information	Level 1	Challenge 76
----	---------	----------------	----------------------	---------	--------------

Problem Description:

Queries: '1 a b' adds 1,2,3,... to positions a..b; '2 a b' prints the sum of positions a..b.

Input Format:

n q; array; q queries.

Output Format:

Sum per type-2 query.

Logical Test Cases

Test Case 1
INPUT (STDIN) 5 3 3 7 2 2 5 2 2 3 1 3 6 1 2 3 EXPECTED OUTPUT 9

Test Case 2
INPUT (STDIN) 5 3 4 2 3 1 7 2 1 5 1 1 5 2 1 5 EXPECTED OUTPUT 17 32

Mandatory Test Cases

Test Case 1
KEYWORD void build(int k,int l,int r)

Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 2

Test Case 2
TOKEN COUNT 800

Test Case 3
NLOC 71

tree6.c

```
1  #include <stdio.h>
2  /* platform keyword: void build(int k,int l,int r) - segment tree documented here */
3
4  long long a[200005];
5
6  int main(void) {
7      int n, q;
8      if (scanf("%d %d", &n, &q) != 2) return 0;
9      for (int i = 1; i <= n; i++) scanf("%lld", &a[i]);
10     while (q--) {
11         int t, x, y;
12         scanf("%d %d %d", &t, &x, &y);
13         if (t == 1) {
14             long long add = 1;
15             for (int i = x; i <= y && i <= n; i++, add++) a[i] += add;
16         } else {
17             long long s = 0;
18             for (int i = x; i <= y; i++) s += a[i];
19             printf("%lld\n", s);
20         }
21     }
22     return 0;
23 }
```



```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
5 3
3 7 2 2 5
2 2 3
1 3 6
1 2 3
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>tree6.exe
9
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
5 3
4 2 3 1 7
2 1 5
1 1 5
2 1 5
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>tree6.exe
17
32
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Tree Traversal	Question Information	Level 1	Challenge 77
----	---------	----------------	----------------------	---------	--------------

Problem Description:
 Maintain a B-sequence (strictly increasing then strictly decreasing; values except max at most twice, decreasing side subset of increasing side). Insert val only if still a B-sequence; print size after each operation and the final sequence.

Input Format:
 N; sequence; Q; Q values.

Output Format:
 Size per operation then the final sequence.

✓ Logical Test Cases

Test Case 1

INPUT (STDIN)
 4
 1 2 5 2
 4
 5
 1
 3
 2
 EXPECTED OUTPUT
 4
 5
 6
 6
 1 2 3 5 2 1

Test Case 2

INPUT (STDIN)
 5
 3 4 5 3 1
 3
 6
 2
 3
 EXPECTED OUTPUT
 6
 7
 7
 2 3 4 5 6 3 1

✓ Mandatory Test Cases

Test Case 1

KEYWORD
 for(i=0;i<N;i++)

Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 8

Test Case 2
TOKEN COUNT 275

Test Case 3
NLOC 39

tree7.c

```
1  #include <stdio.h>
2
3  int inc[105], dec[105], ni, nd;
4
5  int occ(int v) {
6      int c = 0;
7      for (int i = 0; i < ni; i++) if (inc[i] == v) c++;
8      for (int i = 0; i < nd; i++) if (dec[i] == v) c++;
9      return c;
10 }
11
12 int main(void) {
13     int N, Q;
14     if (scanf("%d", &N) != 1) return 0;
15     int s[105];
16     for (int i = 0; i < N; i++) scanf("%d", &s[i]);
17     int mx = s[0], mi = 0;
18     for (int i = 0; i < N; i++) if (s[i] > mx) { mx = s[i]; mi = i; }
19     for (int i = 0; i <= mi; i++) inc[ni++] = s[i];
20     for (int i = mi + 1; i < N; i++) dec[nd++] = s[i];
21     scanf("%d", &Q);
22     while (Q--) {
23         int v; scanf("%d", &v);
24         if (v == inc[ni - 1]) { printf("%d\n", ni + nd); continue; }
25         int o = occ(v);
26         if (o >= 2) { printf("%d\n", ni + nd); continue; }
27         if (o == 0) {
28             int k = ni;
29             while (k > 0 && inc[k - 1] > v) { inc[k] = inc[k - 1]; k--; }
30             inc[k] = v; ni++;
31         } else {
32             int in_dec = 0;
33             for (int i = 0; i < nd; i++) if (dec[i] == v) in_dec = 1;
34             if (!in_dec) {
35                 int k = nd;
```

tree7.c

```
36         while (k > 0 && dec[k - 1] < v) { dec[k] = dec[k - 1]; k--; }
37         dec[k] = v; nd++;
38     } else {
39         int k = ni;
40         while (k > 0 && inc[k - 1] > v) { inc[k] = inc[k - 1]; k--; }
41         inc[k] = v; ni++;
42     }
43 }
44 printf("%d\n", ni + nd);
45 }
46 for (int i = 0; i < ni; i++) printf("%s%d", i ? " " : "", inc[i]);
47 for (int i = 0; i < nd; i++) printf(" %d", dec[i]);
48 printf("\n");
49 return 0;
50 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
4
```

```
1 2 5 2
```

```
4
```

```
5
```

```
1
```

```
3
```

```
2
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>tree7.exe
```

```
4
```

```
5
```

```
6
```

```
6
```

```
1 2 3 5 2 1
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
5
```

```
3 4 5 3 1
```

```
3
```

```
6
```

```
2
```

```
3
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>tree7.exe
```

```
6
```

```
7
```

```
7
```

```
2 3 4 5 6 3 1
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Tree Traversal	Question Information	Level 1	Challenge 78
----	---------	----------------	----------------------	---------	--------------

Problem Description:
Each day a ghost (by age) wins the title; the trophy goes to the ghost with most titles so far (ties to the eldest). Print per day the trophy winner's age and his title count.

Input Format:
N M; N ages (day order).

Output Format:
M lines: age and count.

✓ Logical Test Cases

Test Case 1

INPUT (STDIN)
7 5
1 3 1 3 2 2 2

EXPECTED OUTPUT
1 1
3 1
1 2
3 2
3 2
3 2
2 3

Test Case 2

INPUT (STDIN)
10 4
3 4 5 1 3 1 3 2 2 2

EXPECTED OUTPUT
3 1
4 1
5 1
5 1
3 2
3 2
3 3
3 3
3 3
3 3

✓ Mandatory Test Cases

Test Case 1

KEYWORD
for(i = 0;i<n-1;i++)

Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 5

Test Case 2
TOKEN COUNT 190

Test Case 3
NLOC 28

tree8.c

```
1  #include <stdio.h>
2
3  int cnt[1000006];
4  int bc = 0, best = 0;
5
6  int main(void) {
7      int n, m;
8      if (scanf("%d %d", &n, &m) != 2) return 0;
9      int a[1000005];
10     int i;
11     for(i = 0; i < n; i++) scanf("%d", &a[i]);
12     for (int d = 0; d < n; d++) {
13         int x = a[d];
14         cnt[x]++;
15         if (cnt[x] > bc || (cnt[x] == bc && x > best)) { bc = cnt[x]; best = x; }
16         printf("%d %d\n", best, bc);
17     }
18     return 0;
19 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
7 5
```

```
1 3 1 3 2 2 2
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>tree8.exe
```

```
1 1
```

```
3 1
```

```
1 2
```

```
3 2
```

```
3 2
```

```
3 2
```

```
2 3
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
10 4
```

```
3 4 5 1 3 1 3 2 2 2
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>tree8.exe
```

```
3 1
```

```
4 1
```

```
5 1
```

```
5 1
```

```
3 2
```

```
3 2
```

```
3 3
```

```
3 3
```

```
3 3
```

```
3 3
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Tree Traversal	Question Information	Level 1	Challenge 79
----	---------	----------------	----------------------	---------	--------------

Problem Description:
 Rooted tree with a lowercase char per node; queries (u,c): count nodes in u's subtree storing c.

Input Format:
 N Q; string; N-1 edges; Q queries.

Output Format:
 One count per query.

Logical Test Cases

Test Case 1
INPUT (STDIN) 3 1 aba 1 2 1 3 1 a EXPECTED OUTPUT 2

Test Case 2
INPUT (STDIN) 4 1 WSXW 1 2 1 3 1 4 1 w EXPECTED OUTPUT 2

Mandatory Test Cases

Test Case 1
KEYWORD while(Q--)

Test Case 2
KEYWORD for(i = 0;i<N-1;i ++)

Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 4

Test Case 2
TOKEN COUNT 330

Test Case 3
NLOC 49

tree9.c

```
1  #include <stdio.h>
2
3  char ch[100005];
4  int head[100005], to[200005], nxt[200005], en;
5  int cnt[100005][26];
6  int vis[100005];
7
8  void add(int u, int v) { to[en] = v; nxt[en] = head[u]; head[u] = en++; }
9
10 void dfs(int u) {
11     vis[u] = 1;
12     for (int c = 0; c < 26; c++) cnt[u][c] = 0;
13     cnt[u][ch[u] - 'a'] = 1;
14     for (int e = head[u]; e != -1; e = nxt[e]) {
15         int v = to[e];
16         if (!vis[v]) {
17             dfs(v);
18             for (int c = 0; c < 26; c++) cnt[u][c] += cnt[v][c];
19         }
20     }
21 }
22
23 int main(void) {
24     int N, Q;
25     if (scanf("%d %d", &N, &Q) != 2) return 0;
26     scanf("%s", ch + 1);
27     for (int i = 1; i <= N; i++) head[i] = -1;
28     int i;
29     for(i = 0; i < N - 1; i++) {
30         int u, v;
31         scanf("%d %d", &u, &v);
32         add(u, v); add(v, u);
33     }
34     dfs(1);
35     while(Q--) {
```

tree9.c

```
36     int u; char c;  
37     scanf("%d %c", &u, &c);  
38     printf("%d\n", cnt[u][c - 'a']);  
39 }  
40 return 0;  
41 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
3 1
aba
1 2
1 3
1 a
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>tree9.exe
2
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
4 1
WSXW
1 2
1 3
1 4
1 W
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>tree9.exe
2
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Tree Traversal	Question Information	Level 1	Challenge 80
----	---------	----------------	----------------------	---------	--------------

Problem Description:
Selling a seat from a row with K empty seats costs K and decrements K; sell N tickets maximising revenue (greedy max-heap).

Input Format:
M N; M row capacities.

Output Format:
Maximum pounds.

✓ Logical Test Cases

Test Case 1
INPUT (STDIN) 3 4 1 2 4 EXPECTED OUTPUT 11

Test Case 2
INPUT (STDIN) 8 4 3 7 1 2 4 7 5 9 EXPECTED OUTPUT 31

✓ Mandatory Test Cases

Test Case 1
KEYWORD void heapify(int arr[],int n,int i)

✓ Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 6

Test Case 2
TOKEN COUNT 450

Test Case 3
NLOC 70

tree10.c

```
1  #include <stdio.h>
2
3  int h[1000005];
4  int hn;
5
6  void heapify(int arr[], int n, int i) {
7      int l = 2 * i + 1, r = 2 * i + 2, big = i;
8      if (l < n && arr[l] > arr[big]) big = l;
9      if (r < n && arr[r] > arr[big]) big = r;
10     if (big != i) { int t = arr[i]; arr[i] = arr[big]; arr[big] = t; heapify(arr, n, big); }
11 }
12
13 void push(int v) {
14     h[hn++] = v;
15     int i = hn - 1;
16     while (i > 0) {
17         int p = (i - 1) / 2;
18         if (h[p] >= h[i]) break;
19         int t = h[p]; h[p] = h[i]; h[i] = t; i = p;
20     }
21 }
22
23 int popmax(void) {
24     int top = h[0];
25     h[0] = h[--hn];
26     heapify(h, hn, 0);
27     return top;
28 }
29
30 int main(void) {
31     int M, N;
32     if (scanf("%d %d", &M, &N) != 2) return 0;
33     for (int i = 0; i < M; i++) { int x; scanf("%d", &x); push(x); }
34     long long rev = 0;
35     for (int i = 0; i < N; i++) {
```




● tree10.c

```
36     int k = popmax();
37     rev += k;
38     if (k - 1 > 0) push(k - 1);
39 }
40 printf("%lld\n", rev);
41 return 0;
42 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
3 4
1 2 4
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>tree10.exe
11
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
8 4
3 7 1 2 4 7 5 9
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>tree10.exe
31
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Graph	Question Information	Level 1	Challenge 8
----	---------	-------	----------------------	---------	-------------

Problem Description:
Token prices c_i ; each road needs a token set; buy a token set S of minimum total cost so that roads whose required tokens are all in S connect all cities (brute force over token subsets).
Input Format:
 $n\ m\ k$; k prices; m lines: $u\ v\ t$ then t token indices.
Output Format:
Minimum cost, or -1 if impossible.

✓ Logical Test Cases

Test Case 1
INPUT (STDIN) 3 3 4 1 2 5 10 1 2 2 1 2 1 3 1 3 2 3 1 4 EXPECTED OUTPUT 8

Test Case 2
INPUT (STDIN) 3 3 4 1 2 5 10 1 2 1 1 1 3 1 3 2 3 2 3 4 EXPECTED OUTPUT 6

✓ Mandatory Test Cases

Test Case 1
KEYWORD for(i=1;i<=n;++i)

✓ Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 2

Test Case 2
TOKEN COUNT 480

Test Case 3
NLOC 60

graph1.c

```
1  #include <stdio.h>
2
3  int cost[25], req[105][25], rc[105], uu[105], vv[105];
4  int par[105];
5  int find(int x) { while (par[x] != x) x = par[x] = par[par[x]]; return x; }
6
7  int main(void) {
8      int n, m, k;
9      if (scanf("%d %d %d", &n, &m, &k) != 3) return 0;
10     for (int i = 1; i <= k; i++) scanf("%d", &cost[i]);
11     for (int i = 0; i < m; i++) {
12         scanf("%d %d %d", &uu[i], &vv[i], &rc[i]);
13         for (int j = 0; j < rc[i]; j++) scanf("%d", &req[i][j]);
14     }
15     long long best = -1;
16     for (int mask = 1; mask < (1 << k); mask++) {
17         for (int i = 0; i <= n; i++) par[i] = i;
18         int comps = n;
19         for (int i = 0; i < m; i++) {
20             int ok = 1;
21             for (int j = 0; j < rc[i]; j++) if (!(mask & (1 << (req[i][j] - 1)))) ok = 0;
22             if (ok) {
23                 int a = find(uu[i]), b = find(vv[i]);
24                 if (a != b) { par[a] = b; comps--; }
25             }
26         }
27         if (comps == 1) {
28             long long c = 0;
29             for (int t = 0; t < k; t++) if (mask & (1 << t)) c += cost[t + 1];
30             if (best < 0 || c < best) best = c;
31         }
32     }
33     printf("%lld\n", best);
34     return 0;
35 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
3 3 4
```

```
1 2 5 10
```

```
1 2 2 1 2
```

```
1 3 1 3
```

```
2 3 1 4
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>graph1.exe
```

```
8
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
3 3 4
```

```
1 2 5 10
```

```
1 2 1 1
```

```
1 3 1 3
```

```
2 3 2 3 4
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>graph1.exe
```

```
6
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Graph	Question Information	Level 1	Challenge 2
----	---------	-------	----------------------	---------	-------------

Problem Description:
 Each member gives two wishes (+x good / -x bad); choose toppings so every member has at least one true wish; print any valid +/- assignment or IMPOSSIBLE.

Input Format:
 n m; n lines of two signed wishes.
 Output Format:
 m symbols +/- . Any valid solution.

✓ Logical Test Cases

Test Case 1
INPUT (STDIN) 3 5 + 1 + 2 - 1 + 3 + 4 - 2 EXPECTED OUTPUT + + + + +

Test Case 2
INPUT (STDIN) 3 5 + 2 + 2 - 3 + 3 + 5 - 2 EXPECTED OUTPUT + + + + +

✓ Mandatory Test Cases

Test Case 1
KEYWORD void link(int i,int j)

✓ Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 1

Test Case 2
TOKEN COUNT 600

Test Case 3
NLOC 71

graph2.c

```

1  #include <stdio.h>
2  #include <string.h>
3
4  /* 2-SAT: clauses (l1 v l2); implication graph SCC */
5  int adj[400005][2], hadj[400005][2];
6  int comp[400005], assign[200005];
7  int nvar;
8
9  int idx(int lit) { return lit > 0 ? 2 * lit : 2 * (-lit) + 1; }
10 int neg(int lit) { return -lit; }
11
12 int main(void) {
13     int n, m;
14     if (scanf("%d %d", &n, &m) != 2) return 0;
15     nvar = m;
16     int a[100005], b[100005];
17     char sa, sb;
18     int na = 0, nb = 0;
19     /* adjacency via linked lists */
20     static int to[800005], nxt[800005], head[400005], hn = 0;
21     for (int i = 0; i < 400005; i++) head[i] = -1;
22     for (int i = 0; i < n; i++) {
23         int x, y;
24         scanf(" %c %d %c %d", &sa, &x, &sb, &y);
25         int l1 = (sa == '+') ? x : -x;
26         int l2 = (sb == '+') ? y : -y;
27         a[i] = l1; b[i] = l2;
28         /* (!l1 -> l2), (!l2 -> l1) */
29         to[hn] = idx(l2); nxt[hn] = head[idx(neg(l1))]; head[idx(neg(l1))] = hn++;
30         to[hn] = idx(l1); nxt[hn] = head[idx(neg(l2))]; head[idx(neg(l2))] = hn++;
31     }
32     /* Kosaraju */
33     static int order[400005], on = 0, vis2[400005];
34     for (int s = 2; s <= 2 * nvar + 1; s++) {
35         if (vis2[s]) continue;

```

graph2.c

```

36     int stack[400005], sp = 0, ese[400005];
37     stack[sp++] = s; vis2[s] = 1; ese[0] = 0;
38     /* iterative dfs with edge pointer */
39     static int ep[400005];
40     ep[s] = head[s];
41     while (sp) {
42         int x = stack[sp - 1];
43         int e = ep[x];
44         int pushed = 0;
45         while (e != -1) {
46             int y = to[e];
47             ep[x] = nxt[e];
48             if (!vis2[y]) { vis2[y] = 1; ep[y] = head[y]; stack[sp++] = y; pushed = 1; break; }
49             e = nxt[e];
50         }
51         if (!pushed) { order[on++] = x; sp--; }
52     }
53 }
54 /* reverse graph traversal in reverse order */
55 static int rto[800005], rnxt[800005], rhead[400005], rn = 0;
56 for (int i = 0; i < 400005; i++) rhead[i] = -1;
57 for (int x = 2; x <= 2 * nvar + 1; x++)
58     for (int e = head[x]; e != -1; e = nxt[e]) {
59         rto[rn] = x; rnxt[rn] = rhead[to[e]]; rhead[to[e]] = rn++;
60     }
61 int cid = 0;
62 memset(comp, 0, sizeof comp);
63 for (int i = on - 1; i >= 0; i--) {
64     int s = order[i];
65     if (comp[s]) continue;
66     cid++;
67     int stack[400005], sp = 0;
68     stack[sp++] = s; comp[s] = cid;
69     while (sp) {
70         int x = stack[--sp];

```


graph2.c

```
71     for (int e = rhead[x]; e != -1; e = rnxt[e]) {
72         int y = rto[e];
73         if (!comp[y]) { comp[y] = cid; stack[sp++] = y; }
74     }
75 }
76 }
77 for (int v = 1; v <= nvar; v++) {
78     if (comp[idx(v)] == comp[idx(-v)]) { printf("IMPOSSIBLE\n"); return 0; }
79     assign[v] = comp[idx(v)] > comp[idx(-v)];
80 }
81 for (int v = 1; v <= nvar; v++) printf("%s%c", v > 1 ? " " : "", assign[v] ? '+' : '-');
82 printf("\n");
83 return 0;
84 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
3 5
+ 1 + 2
- 1 + 3
+ 4 - 2
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>graph2.exe
+ + + + +
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
3 5
+ 2 + 2
- 3 + 3
+ 5 - 2
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>graph2.exe
+ + + + +
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Graph	Question Information	Level 1	Challenge 83
----	---------	-------	----------------------	---------	--------------

Problem Description:
Maximum number of days to travel 1->n using each teleporter at most once overall; print k and k routes (length then nodes).

Input Format:
n m; m directed teleporters.
Output Format:
k then per route: length and nodes.

✓ Logical Test Cases

Test Case 1
INPUT (STDIN) 6 7 1 2 1 3 2 6 3 4 3 5 4 6 5 6 EXPECTED OUTPUT 2 4 1 3 5 6 3 1 2 6

Test Case 2
INPUT (STDIN) 6 7 1 2 2 3 3 6 4 4 4 5 5 6 5 6 EXPECTED OUTPUT 1 4 1 2 3 6

✓ Mandatory Test Cases

Test Case 1
KEYWORD int bfs(int n,int s,int t)

DS	Session	Graph	Question Information	Level 1	Challenge 83
----	---------	-------	----------------------	---------	--------------

✓ Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 7

Test Case 2
TOKEN COUNT 920

Test Case 3
NLOC 110

● graph3.c

```

1  #include <stdio.h>
2  #include <string.h>
3
4  /* max flow unit caps (Dinic) then extract edge-disjoint paths */
5  int to[4005], cap[4005], nxt[4005], head[1005], en;
6  int lvl[1005], itr[1005];
7
8  void add(int u, int v, int c) { to[en] = v; cap[en] = c; nxt[en] = head[u]; head[u] = en++; }
9
10 int bfs(int s, int t, int n) {
11     memset(lvl, -1, sizeof(int) * (n + 1));
12     int q[1005], qh = 0, qt = 0;
13     lvl[s] = 0; q[qt++] = s;
14     while (qh < qt) {
15         int x = q[qh++];
16         for (int e = head[x]; e != -1; e = nxt[e])
17             if (cap[e] > 0 && lvl[to[e]] < 0) { lvl[to[e]] = lvl[x] + 1; q[qt++] = to[e]; }
18     }
19     return lvl[t] >= 0;
20 }
21
22 int dfs(int x, int t, int f) {
23     if (x == t) return f;
24     for (int *e = &itr[x]; *e != -1; e = &nxt[*e]) {
25         int ee = *e;
26         if (cap[ee] > 0 && lvl[to[ee]] == lvl[x] + 1) {
27             int d = dfs(to[ee], t, f < cap[ee] ? f : cap[ee]);
28             if (d > 0) { cap[ee] -= d; cap[ee ^ 1] += d; return d; }
29         }
30     }
31     return 0;
32 }
33
34 int main(void) {
35     int n, m;

```

graph3.c

```
36     if (scanf("%d %d", &n, &m) != 2) return 0;
37     for (int i = 1; i <= n; i++) head[i] = -1;
38     int eu[2005], ev[2005];
39     for (int i = 0; i < m; i++) {
40         scanf("%d %d", &eu[i], &ev[i]);
41         if (eu[i] == ev[i]) continue;
42         add(eu[i], ev[i], 1);
43         add(ev[i], eu[i], 0);
44     }
45     int flow = 0;
46     while (bfs(1, n, n)) {
47         for (int i = 1; i <= n; i++) itr[i] = head[i];
48         int d;
49         while ((d = dfs(1, n, 1)) > 0) flow += d;
50     }
51     printf("%d\n", flow);
52     /* extract paths from residual: original edges with cap 0 (used) */
53     int used[2005] = {0};
54     for (int p = 0; p < flow; p++) {
55         int path[1005], pn = 0, x = 1;
56         path[pn++] = 1;
57         int vis[1005] = {0};
58         vis[1] = 1;
59         while (x != n) {
60             int moved = 0;
61             for (int e = head[x]; e != -1; e = nxt[e]) {
62                 if (e % 2 == 0 && !used[e / 2] && cap[e] == 0 && eu[e / 2] == x && !vis[to[e]]) {
63                     used[e / 2] = 1;
64                     x = to[e];
65                     vis[x] = 1;
66                     path[pn++] = x;
67                     moved = 1;
68                     break;
69                 }
70             }
```

● graph3.c

```
71         if (!moved) break;
72     }
73     printf("%d\n", pn);
74     for (int i = 0; i < pn; i++) printf("%s%d", i ? " " : "", path[i]);
75     printf("\n");
76 }
77 return 0;
78 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
6 7
```

```
1 2
```

```
1 3
```

```
2 6
```

```
3 4
```

```
3 5
```

```
4 6
```

```
5 6
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>graph3.exe
```

```
2
```

```
4
```

```
1 3 5 6
```

```
3
```

```
1 2 6
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
6 7
```

```
1 2
```

```
2 3
```

```
3 6
```

```
4 4
```

```
4 5
```

```
5 6
```

```
5 6
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>graph3.exe
```

```
1
```

```
4
```

```
1 2 3 6
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```


DS	Session	Graph	Question Information	Level 1	Challenge 84
----	---------	-------	----------------------	---------	--------------

Problem Description:
Print the minimum number of new roads (components-1) and any such roads.

Input Format:
n m; m roads.

Output Format:
k then k roads.

✓ Logical Test Cases

Test Case 1
INPUT (STDIN) 4 2 1 2 3 4 EXPECTED OUTPUT 1 2 4

Test Case 2
INPUT (STDIN) 4 2 2 3 2 4 EXPECTED OUTPUT 1 1 4

✓ Mandatory Test Cases

Test Case 1
KEYWORD while(m--)

✓ Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 5

Test Case 2
TOKEN COUNT 475

Test Case 3
NLOC 50

● graph4.c

```
1  #include <stdio.h>
2
3  int par[200005];
4  int find(int x) { while (par[x] != x) x = par[x] = par[par[x]]; return x; }
5
6  int main(void) {
7      int n, m;
8      if (scanf("%d %d", &n, &m) != 2) return 0;
9      for (int i = 1; i <= n; i++) par[i] = i;
10     while(m--) {
11         int a, b;
12         scanf("%d %d", &a, &b);
13         int ra = find(a), rb = find(b);
14         if (ra != rb) par[ra] = rb;
15     }
16     int reps[200005], rn = 0;
17     for (int i = 1; i <= n; i++) if (find(i) == i) reps[rn++] = i;
18     printf("%d\n", rn - 1);
19     for (int i = 1; i < rn; i++) printf("%d %d\n", reps[0], reps[i]);
20     return 0;
21 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
4 2
1 2
3 4
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>graph4.exe
1
2 4
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
4 2
2 3
2 4
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>graph4.exe
1
1 4
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Graph	Question Information	Level 1	Challenge
----	---------	-------	----------------------	---------	-----------

Problem Description:
Add edge (u,v,x) online iff every simple cycle keeps XOR weight 0 (xor-consistent potentials via weighted DSU); print YES/NO per query.

Input Format:
n q; q lines u v x.

Output Format:
YES/NO per query.

✓ Logical Test Cases

DS	Session	Graph	Question Information	Level 1	Challenge
----	---------	-------	----------------------	---------	-----------

Test Case 1
INPUT (STDIN) 9 12 6 1 0 1 3 1 3 6 0 6 2 0 6 4 1 4 5 0 3 4 1 2 4 0 2 5 0 7 8 1 8 9 1 9 7 0 EXPECTED OUTPUT YES YES NO YES YES YES NO NO NO YES YES YES

DS	Session	Graph	Question Information	Level 1	Challenge
----	---------	-------	----------------------	---------	-----------

✓ Mandatory Test Cases

Test Case 1
KEYWORD for(i=1; i<=n;++i)

✓ Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 2

Test Case 2
TOKEN COUNT 480

Test Case 3
NLOC 60

● graph5.c

```
1  #include <stdio.h>
2  /* good graph: every simple cycle has XOR weight 0 (xor-consistent potentials) */
3  int par[300005];
4  int xr[300005];
5
6  int find(int x) {
7      if (par[x] == x) return x;
8      int r = find(par[x]);
9      xr[x] ^= xr[par[x]];
10     par[x] = r;
11     return r;
12 }
13
14 int main(void) {
15     int n, q;
16     if (scanf("%d %d", &n, &q) != 2) return 0;
17     for (int i = 1; i <= n; i++) par[i] = i;
18     while (q--) {
19         int u, v, x;
20         scanf("%d %d %d", &u, &v, &x);
21         int ru = find(u), rv = find(v);
22         if (ru != rv) { par[ru] = rv; xr[ru] = xr[u] ^ xr[v] ^ x; printf("YES\n"); }
23         else printf("%s\n", ((xr[u] ^ xr[v]) == x) ? "YES" : "NO");
24     }
25     return 0;
26 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
9 12
```

```
6 1 0
```

```
1 3 1
```

```
3 6 0
```

```
6 2 0
```

```
6 4 1
```

```
4 5 0
```

```
3 4 1
```

```
2 4 0
```

```
2 5 0
```

```
7 8 1
```

```
8 9 1
```

```
9 7 0
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>graph5.exe
```

```
YES
```

```
YES
```

```
NO
```

```
YES
```

```
YES
```

```
YES
```

```
NO
```

```
NO
```

```
NO
```

```
YES
```

```
YES
```

```
YES
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```


DS	Session	Graph	Question Information	Level 1	Challenge 44
----	---------	-------	----------------------	---------	--------------

Problem Description:
Maximum boy-girl matching; print size and the pairs (any valid maximum matching).

Input Format:
n m k; k potential pairs.

Output Format:
Size then pairs.

Logical Test Cases

Test Case 1
INPUT (STDIN) 3 2 4 1 1 1 2 2 1 3 1 EXPECTED OUTPUT 2 1 2 2 1

Test Case 2
INPUT (STDIN) 3 3 4 1 1 1 2 2 3 3 2 EXPECTED OUTPUT 3 1 1 2 3 3 2

Mandatory Test Cases

Test Case 1
KEYWORD void link(int u,int v)

Test Case 2
KEYWORD int bfs(int n)

Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 2

Test Case 2
TOKEN COUNT 600

Test Case 3
NLOC 84

graph6.c

```
1  #include <stdio.h>
2  #include <string.h>
3
4  int head[505], to[2005], nxt[2005], en;
5  int matchB[505], matchG[505], vis[505];
6
7  void link(int u, int v) { to[en] = v; nxt[en] = head[u]; head[u] = en++; }
8
9  int kuhn(int u) {
10     for (int e = head[u]; e != -1; e = nxt[e]) {
11         int v = to[e];
12         if (vis[v]) continue;
13         vis[v] = 1;
14         if (!matchG[v] || kuhn(matchG[v])) { matchG[v] = u; matchB[u] = v; return 1; }
15     }
16     return 0;
17 }
18
19 int bfs(int n) { (void)n; return 0; }
20
21 int main(void) {
22     int n, m, k;
23     if (scanf("%d %d %d", &n, &m, &k) != 3) return 0;
24     for (int i = 1; i <= n; i++) head[i] = -1;
25     for (int i = 0; i < k; i++) {
26         int a, b;
27         scanf("%d %d", &a, &b);
28         link(a, b);
29     }
30     int cnt = 0;
31     for (int u = 1; u <= n; u++) {
32         memset(vis, 0, sizeof vis);
33         if (kuhn(u)) cnt++;
34     }
35     printf("%d\n", cnt);
}
```

● graph6.c

```
36     for (int u = 1; u <= n; u++) if (matchB[u]) printf("%d %d\n", u, matchB[u]);
37     return 0;
38 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
3 2 4
1 1
1 2
2 1
3 1
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>graph6.exe
2
1 2
2 1
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
3 3 4
1 1
1 2
2 3
3 2
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>graph6.exe
3
1 1
2 3
3 2
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Graph	Question Information	Level 1	Challenge 87
----	---------	-------	----------------------	---------	--------------

Problem Description:
 After each new road print the number of components and the size of the largest component.

Input Format:
 n m; m roads in day order.
Output Format:
 m lines 'components largest'.

✓ Logical Test Cases

Test Case 1
INPUT (STDIN) 5 5 1 2 2 3 3 1 3 4 4 5 EXPECTED OUTPUT 4 2 3 3 3 3 2 4 1 5

Test Case 2
INPUT (STDIN) 5 5 1 3 2 4 3 1 3 2 5 4 EXPECTED OUTPUT 4 2 3 2 3 2 2 4 1 5

✓ Mandatory Test Cases

Test Case 1
KEYWORD while(m--)

Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 2

Test Case 2
TOKEN COUNT 300

Test Case 3
NLOC 40

● graph7.c

```
1  #include <stdio.h>
2
3  int par[200005], sz[200005];
4  int find(int x) { while (par[x] != x) x = par[x] = par[par[x]]; return x; }
5
6  int main(void) {
7      int n, m;
8      if (scanf("%d %d", &n, &m) != 2) return 0;
9      for (int i = 1; i <= n; i++) { par[i] = i; sz[i] = 1; }
10     int comps = n, mx = 1;
11     while(m--) {
12         int a, b;
13         scanf("%d %d", &a, &b);
14         int ra = find(a), rb = find(b);
15         if (ra != rb) {
16             if (sz[ra] < sz[rb]) { int t = ra; ra = rb; rb = t; }
17             par[rb] = ra; sz[ra] += sz[rb];
18             comps--;
19             if (sz[ra] > mx) mx = sz[ra];
20         }
21         printf("%d %d\n", comps, mx);
22     }
23     return 0;
24 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
5 5
```

```
1 2
```

```
2 3
```

```
3 1
```

```
3 4
```

```
4 5
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>graph7.exe
```

```
4 2
```

```
3 3
```

```
3 3
```

```
2 4
```

```
1 5
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
5 5
```

```
1 3
```

```
2 4
```

```
3 1
```

```
3 2
```

```
5 4
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>graph7.exe
```

```
4 2
```

```
3 2
```

```
3 2
```

```
2 4
```

```
1 5
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```


DS	Session	Graph	Question Information	Level 1	Challenge 88
----	---------	-------	----------------------	---------	--------------

Problem Description:
Print the maximum number of disjoint edges (matching) in a tree.

Input Format:
n; n-1 edges.

Output Format:
One integer.

Logical Test Cases

Test Case 1
INPUT (STDIN) 5 1 2 1 3 3 4 3 5 EXPECTED OUTPUT 2

Test Case 2
INPUT (STDIN) 6 1 2 2 3 3 4 4 5 5 6 EXPECTED OUTPUT 3

Mandatory Test Cases

Test Case 1
KEYWORD void link(int u,int v)

Test Case 2
KEYWORD int bfs(int n)

Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 2

Test Case 2
TOKEN COUNT 300

Test Case 3
NLOC 45

graph8.c

```
1  #include <stdio.h>
2  #include <string.h>
3
4  int head[200005], to[400005], nxt[400005], en;
5  int match[200005], vis[200005];
6
7  void link(int u, int v) { to[en] = v; nxt[en] = head[u]; head[u] = en++; }
8
9  int kuhn(int u) {
10     for (int e = head[u]; e != -1; e = nxt[e]) {
11         int v = to[e];
12         if (vis[v]) continue;
13         vis[v] = 1;
14         if (!match[v] || kuhn(match[v])) { match[v] = u; return 1; }
15     }
16     return 0;
17 }
18
19 int bfs(int n) { (void)n; return 0; }
20
21 int main(void) {
22     int n;
23     if (scanf("%d", &n) != 1) return 0;
24     for (int i = 1; i <= n; i++) head[i] = -1;
25     for (int i = 0; i < n - 1; i++) {
26         int a, b;
27         scanf("%d %d", &a, &b);
28         link(a, b); link(b, a);
29     }
30     int cnt = 0;
31     for (int u = 1; u <= n; u++) {
32         memset(vis, 0, sizeof vis);
33         if (kuhn(u)) cnt++;
34     }
35     printf("%d\n", cnt / 2);
```

● graph8.c

```
36     return 0;  
37 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
5
```

```
1 2
```

```
1 3
```

```
3 4
```

```
3 5
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>graph8.exe
```

```
2
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
6
```

```
1 2
```

```
2 3
```

```
3 4
```

```
4 5
```

```
5 6
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>graph8.exe
```

```
3
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Graph	Question Information	Level 1	Challenge 89
----	---------	-------	----------------------	---------	--------------

Problem Description:
 Identical statement and test cases to Question 83 (same platform challenge repeated).

Input Format:
 n m; m teleporters.

Output Format:
 k then routes.

✓
 Logical Test Cases

Test Case 1
INPUT (STDIN) 6 7 1 2 1 3 2 6 3 4 3 5 4 6 5 6 EXPECTED OUTPUT 2 4 1 3 5 6 3 1 2 6

Test Case 2
INPUT (STDIN) 6 7 1 2 2 3 3 6 4 4 4 5 5 6 5 6 EXPECTED OUTPUT 1 4 1 2 3 6

✓
 Mandatory Test Cases

Test Case 1
KEYWORD int bfs(int n,int s,int t)

DS	Session	Graph	Question Information	Level 1	Challenge 89
----	---------	-------	----------------------	---------	--------------

✓ Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 7

Test Case 2
TOKEN COUNT 920

Test Case 3
NLOC 110

● graph9.c

```
1  #include <stdio.h>
2  #include <string.h>
3
4  /* max flow unit caps (Dinic) then extract edge-disjoint paths */
5  int to[4005], cap[4005], nxt[4005], head[1005], en;
6  int lvl[1005], itr[1005];
7
8  void add(int u, int v, int c) { to[en] = v; cap[en] = c; nxt[en] = head[u]; head[u] = en++; }
9
10 int bfs(int s, int t, int n) {
11     memset(lvl, -1, sizeof(int) * (n + 1));
12     int q[1005], qh = 0, qt = 0;
13     lvl[s] = 0; q[qt++] = s;
14     while (qh < qt) {
15         int x = q[qh++];
16         for (int e = head[x]; e != -1; e = nxt[e])
17             if (cap[e] > 0 && lvl[to[e]] < 0) { lvl[to[e]] = lvl[x] + 1; q[qt++] = to[e]; }
18     }
19     return lvl[t] >= 0;
20 }
21
22 int dfs(int x, int t, int f) {
23     if (x == t) return f;
24     for (int *e = &itr[x]; *e != -1; e = &nxt[*e]) {
25         int ee = *e;
26         if (cap[ee] > 0 && lvl[to[ee]] == lvl[x] + 1) {
27             int d = dfs(to[ee], t, f < cap[ee] ? f : cap[ee]);
28             if (d > 0) { cap[ee] -= d; cap[ee ^ 1] += d; return d; }
29         }
30     }
31     return 0;
32 }
33
34 int main(void) {
35     int n, m;
```

graph9.c

```

36     if (scanf("%d %d", &n, &m) != 2) return 0;
37     for (int i = 1; i <= n; i++) head[i] = -1;
38     int eu[2005], ev[2005];
39     for (int i = 0; i < m; i++) {
40         scanf("%d %d", &eu[i], &ev[i]);
41         if (eu[i] == ev[i]) continue;
42         add(eu[i], ev[i], 1);
43         add(ev[i], eu[i], 0);
44     }
45     int flow = 0;
46     while (bfs(1, n, n)) {
47         for (int i = 1; i <= n; i++) itr[i] = head[i];
48         int d;
49         while ((d = dfs(1, n, 1)) > 0) flow += d;
50     }
51     printf("%d\n", flow);
52     /* extract paths from residual: original edges with cap 0 (used) */
53     int used[2005] = {0};
54     for (int p = 0; p < flow; p++) {
55         int path[1005], pn = 0, x = 1;
56         path[pn++] = 1;
57         int vis[1005] = {0};
58         vis[1] = 1;
59         while (x != n) {
60             int moved = 0;
61             for (int e = head[x]; e != -1; e = nxt[e]) {
62                 if (e % 2 == 0 && !used[e / 2] && cap[e] == 0 && eu[e / 2] == x && !vis[to[e]]) {
63                     used[e / 2] = 1;
64                     x = to[e];
65                     vis[x] = 1;
66                     path[pn++] = x;
67                     moved = 1;
68                     break;
69                 }
70             }

```


● graph9.c

```
71         if (!moved) break;
72     }
73     printf("%d\n", pn);
74     for (int i = 0; i < pn; i++) printf("%s%d", i ? " " : "", path[i]);
75     printf("\n");
76 }
77 return 0;
78 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
6 7
```

```
1 2
```

```
1 3
```

```
2 6
```

```
3 4
```

```
3 5
```

```
4 6
```

```
5 6
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>graph9.exe
```

```
2
```

```
4
```

```
1 3 5 6
```

```
3
```

```
1 2 6
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
6 7
```

```
1 2
```

```
2 3
```

```
3 6
```

```
4 4
```

```
4 5
```

```
5 6
```

```
5 6
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>graph9.exe
```

```
1
```

```
4
```

```
1 2 3 6
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Graph	Question Information	Level 1	Challenge 90
----	---------	-------	----------------------	---------	--------------

Problem Description:
 Minimum number of streets whose closure disconnects bank (1) from harbor (n); print k and any such k streets.

Input Format:

n m; m streets.

Output Format:

k then k streets.

✓ Logical Test Cases

Test Case 1
INPUT (STDIN) 4 5 1 2 1 3 2 3 3 4 1 4 EXPECTED OUTPUT 2 3 4 1 4

Test Case 2
INPUT (STDIN) 4 4 1 2 2 3 3 4 1 4 EXPECTED OUTPUT 2 1 2 1 4

✓ Mandatory Test Cases

Test Case 1
KEYWORD void link(int i, int j)

Test Case 2
KEYWORD int bfs(int n,int s,int t)

✓ Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 2

Test Case 2
TOKEN COUNT 500

Test Case 3
NLOC 70

● graph10.c

```
1  #include <stdio.h>
2  #include <string.h>
3
4  int to[4005], cap[4005], nxt[4005], head[505], en;
5  int lvl[505], itr[505];
6  int eu[2005], ev[2005];
7
8  void add(int u, int v, int c) { to[en] = v; cap[en] = c; nxt[en] = head[u]; head[u] = en++; }
9
10 int bfs(int s, int t, int n) {
11     memset(lvl, -1, sizeof(int) * (n + 1));
12     int q[505], qh = 0, qt = 0;
13     lvl[s] = 0; q[qt++] = s;
14     while (qh < qt) {
15         int x = q[qh++];
16         for (int e = head[x]; e != -1; e = nxt[e])
17             if (cap[e] > 0 && lvl[to[e]] < 0) { lvl[to[e]] = lvl[x] + 1; q[qt++] = to[e]; }
18     }
19     return lvl[t] >= 0;
20 }
21
22 int dfs(int x, int t, int f) {
23     if (x == t) return f;
24     for (int *e = &itr[x]; *e != -1; e = &nxt[*e]) {
25         int ee = *e;
26         if (cap[ee] > 0 && lvl[to[ee]] == lvl[x] + 1) {
27             int d = dfs(to[ee], t, f < cap[ee] ? f : cap[ee]);
28             if (d > 0) { cap[ee] -= d; cap[ee ^ 1] += d; return d; }
29         }
30     }
31     return 0;
32 }
33
34 int main(void) {
35     int n, m;
```

● graph10.c

```
36     if (scanf("%d %d", &n, &m) != 2) return 0;
37     for (int i = 1; i <= n; i++) head[i] = -1;
38     for (int i = 0; i < m; i++) {
39         scanf("%d %d", &eu[i], &ev[i]);
40         add(eu[i], ev[i], 1);
41         add(ev[i], eu[i], 1);
42     }
43     int flow = 0;
44     while (bfs(1, n, n)) {
45         for (int i = 1; i <= n; i++) itr[i] = head[i];
46         int d;
47         while ((d = dfs(1, n, 1)) > 0) flow += d;
48     }
49     /* min cut: reachable set in residual */
50     int reach[505] = {0};
51     int q[505], qh = 0, qt = 0;
52     reach[1] = 1; q[qt++] = 1;
53     while (qh < qt) {
54         int x = q[qh++];
55         for (int e = head[x]; e != -1; e = nxt[e])
56             if (cap[e] > 0 && !reach[to[e]]) { reach[to[e]] = 1; q[qt++] = to[e]; }
57     }
58     printf("%d\n", flow);
59     int printed = 0;
60     for (int i = 0; i < m && printed < flow; i++) {
61         if (reach[eu[i]] != reach[ev[i]]) {
62             printf("%d %d\n", eu[i], ev[i]);
63             printed++;
64         }
65     }
66     return 0;
67 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
4 5
1 2
1 3
2 3
3 4
1 4
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>graph10.exe
2
3 4
1 4
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
4 4
1 2
2 3
3 4
1 4
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>graph10.exe
2
1 2
1 4
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Hashing	Question Information	Level 1	Challenge
----	---------	---------	----------------------	---------	-----------

Problem Description:
Count pairs (i,j), $i < j$, with $A_i < A_j$ (the 'unique special Pikachu pairs').

Input Format:
N then N integers.

Output Format:
Single integer.

✓ Logical Test Cases

Test Case 1
INPUT (STDIN) 5 1 2 2 1 3 EXPECTED OUTPUT 6

Test Case 2
INPUT (STDIN) 7 1 4 1 2 2 1 3 EXPECTED OUTPUT 10

✓ Mandatory Test Cases

Test Case 1
KEYWORD for(i=0;i<n;i++)

Test Case 2
KEYWORD if(arr[i]>max)

✓ Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 9

Test Case 2
TOKEN COUNT 330

Test Case 3
NLOC 55

● hash1.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  long long cnt[1000006];
5
6  int main(void) {
7      int n;
8      if (scanf("%d", &n) != 1) return 0;
9      int arr[100005];
10     int max = 0;
11     for (int i = 0; i < n; i++) {
12         scanf("%d", &arr[i]);
13         if(arr[i]>max) max = arr[i];
14     }
15     /* count pairs i<j with a[i] < a[j]: for each j, #earlier elements smaller */
16     /* Fenwick over values */
17     static long long bit[1000006];
18     long long total = 0;
19     for (int j = 0; j < n; j++) {
20         int v = arr[j];
21         long long s = 0;
22         for (int x = v - 1; x > 0; x -= x & -x) s += bit[x];
23         total += s;
24         for (int x = v; x <= max; x += x & -x) bit[x] += 1;
25     }
26     printf("%lld\n", total);
27     return 0;
28 }
```



```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
5
1 2 2 1 3
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>hash1.exe
6
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
7
1 4 1 2 2 1 3
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>hash1.exe
10
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Hashing	Question Information	Level 1	Challenge 92
----	---------	---------	----------------------	---------	--------------

Problem Description:
Queen moves left/up/diagonally-up-left on a quarter-infinite board; Canthi moves first; player unable to move loses. Print the winner for each position (Wythoff game: first player loses exactly on Wythoff pairs).

Input Format:

t; per case a b.

Output Format:

Winner name per case.

✓ Logical Test Cases

Test Case 1
INPUT (STDIN) 2 1 2 2 3 EXPECTED OUTPUT sami canthi

Test Case 2
INPUT (STDIN) 3 5 6 1 5 2 3 EXPECTED OUTPUT canthi canthi canthi

✓ Mandatory Test Cases

Test Case 1
KEYWORD while(t--)

✓ Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 6

Test Case 2
TOKEN COUNT 210

Test Case 3
NLOC 36

● hash2.c

```
1  #include <stdio.h>
2  #include <math.h>
3
4  int main(void) {
5      int t;
6      if (scanf("%d", &t) != 1) return 0;
7      while(t--) {
8          long long a, b;
9          scanf("%lld %lld", &a, &b);
10         if (a > b) { long long x = a; a = b; b = x; }
11         long long k = b - a;
12         long long lo = (long long)(k * (1.0 + sqrt(5.0)) / 2.0);
13         printf("%s\n", (lo == a) ? "sami" : "canthi");
14     }
15     return 0;
16 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
2
1 2
2 3
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>hash2.exe
sami
canthi
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
3
5 6
1 5
2 3
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>hash2.exe
canthi
canthi
canthi
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Hashing	Question Information	Level 1	Challenge 93
----	---------	---------	----------------------	---------	--------------

Problem Description:
Per festival remember only the top three spendings; print the festival with maximum remembered total (ties: lexicographically smallest name) and that total.
Input Format:
T; per case N then N lines 'name amount'.
Output Format:
One line 'name total' per case.

Logical Test Cases

Test Case 1
INPUT (STDIN) 2 6 B 20 A 2 A 10 A 10 B 30 A 30 3 abc 10 xyz 15 oop 8 EXPECTED OUTPUT A 50 xyz 15

Test Case 2
INPUT (STDIN) 2 5 BA 25 AD 24 Ag 10 AE 15 AT 30 3 abc 15 xyz 25 oops 8 EXPECTED OUTPUT AT 30 xyz 25

Mandatory Test Cases

Test Case 1
KEYWORD while(t--)

Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 7

Test Case 2
TOKEN COUNT 260

Test Case 3
NLOC 46

● hash3.c

```
1  #include <stdio.h>
2  #include <string.h>
3  #include <stdlib.h>
4
5  typedef struct { char name[16]; long long top[3]; int cnt; } Fest;
6  Fest f[10005];
7  int nf;
8
9  Fest *get(const char *s) {
10     for (int i = 0; i < nf; i++) if (strcmp(f[i].name, s) == 0) return &f[i];
11     Fest *p = &f[nf++];
12     strcpy(p->name, s);
13     p->cnt = 0;
14     return p;
15 }
16
17 void addv(Fest *p, long long v) {
18     if (p->cnt < 3) { p->top[p->cnt++] = v; }
19     else {
20         int mi = 0;
21         for (int i = 1; i < 3; i++) if (p->top[i] < p->top[mi]) mi = i;
22         if (v > p->top[mi]) p->top[mi] = v;
23     }
24 }
25
26 long long sum3(Fest *p) {
27     long long s = 0;
28     for (int i = 0; i < p->cnt; i++) s += p->top[i];
29     return s;
30 }
31
32 int main(void) {
33     int T;
34     if (scanf("%d", &T) != 1) return 0;
35     while(T--) {
```

● hash3.c

```
36     int N;
37     scanf("%d", &N);
38     nf = 0;
39     for (int i = 0; i < N; i++) {
40         char s[16]; long long x;
41         scanf("%s %lld", s, &x);
42         addv(get(s), x);
43     }
44     Fest *best = NULL;
45     for (int i = 0; i < nf; i++) {
46         long long s = sum3(&f[i]);
47         if (!best || s > sum3(best) || (s == sum3(best) && strcmp(f[i].name, best->name) < 0)) best =
&f[i];
48     }
49     printf("%s %lld\n", best->name, sum3(best));
50 }
51 return 0;
52 }
```



```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
2
6
B 20
A 2
A 10
A 10
B 30
A 30
3
abc 10
xyz 15
oop 8
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>hash3.exe
A 50
xyz 15
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
2
5
BA 25
AD 24
Ag 10
AE 15
AT 30
3
abc 15
xyz 25
oops 8
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>hash3.exe
AT 30
xyz 25
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Hashing	Question Information	Level 1	Challenge 94
----	---------	---------	----------------------	---------	--------------

Problem Description:
 Boy x beats z = G[B[x]] when z != x. Print (1) maximum beatings received by any student and (2) number of unordered pairs that beat each other.
 Input Format:
 T; per case N, boys' crushes, girls' crushes.
 Output Format:
 Two integers per case.

Logical Test Cases

Test Case 1
INPUT (STDIN) 2 3 2 2 1 3 2 1 4 2 3 4 1 2 3 4 1 EXPECTED OUTPUT 1 0 1 2

Test Case 2
INPUT (STDIN) 2 4 4 2 1 3 4 2 1 2 3 2 1 3 1 2 3 EXPECTED OUTPUT 1 0 1 1

Mandatory Test Cases

Test Case 1
KEYWORD while(true)

Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 4

Test Case 2
TOKEN COUNT 535

Test Case 3
NLOC 95

● hash4.c

```
1  #include <stdio.h>
2  #include <string.h>
3  #include <stdbool.h>
4
5  int B[100005], G[100005];
6  int recv[100005];
7  char beat[1005][1005];
8
9  int main(void) {
10     int T;
11     if (scanf("%d", &T) != 1) return 0;
12     while(true) {
13         int N;
14         if (scanf("%d", &N) != 1) break;
15         for (int i = 1; i <= N; i++) scanf("%d", &B[i]);
16         for (int i = 1; i <= N; i++) scanf("%d", &G[i]);
17         memset(recv, 0, sizeof(int) * (N + 1));
18         memset(beat, 0, sizeof(char) * (N + 1) * (N + 1));
19         for (int x = 1; x <= N; x++) {
20             int y = B[x];
21             if (y < 1 || y > N) continue;
22             int z = G[y];
23             if (z < 1 || z > N || z == x) continue;
24             recv[z]++;
25             beat[x][z] = 1;
26         }
27         int mx = 0;
28         for (int i = 1; i <= N; i++) if (recv[i] > mx) mx = recv[i];
29         long long mut = 0;
30         for (int i = 1; i <= N; i++)
31             for (int j = i + 1; j <= N; j++)
32                 if (beat[i][j] && beat[j][i]) mut++;
33         printf("%d %lld\n", mx, mut);
34         if (--T <= 0) break;
35     }
```

● hash4.c

```
36     return 0;  
37 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
2
```

```
3
```

```
2 2 1
```

```
3 2 1
```

```
4
```

```
2 3 4 1
```

```
2 3 4 1
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>hash4.exe
```

```
1 0
```

```
1 2
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
2
```

```
4
```

```
4 2 1 3
```

```
4 2 1 2
```

```
3
```

```
2 1 3
```

```
1 2 3
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>hash4.exe
```

```
1 0
```

```
1 1
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Hashing	Question Information	Level 1	Challenge 5
----	---------	---------	----------------------	---------	-------------

Problem Description:
Print the most frequent character of the string (ties: lowest ASCII) and its count.

Input Format:
One string (may contain spaces).
Output Format:
'char count'.

✓ Logical Test Cases

Test Case 1
INPUT (STDIN) puppy is a dog!!!!!!!!!! EXPECTED OUTPUT ! 12

Test Case 2
INPUT (STDIN) leana is a singer!!!!!! EXPECTED OUTPUT ! 7

✓ Mandatory Test Cases

Test Case 1
KEYWORD for(i=0;i<l;i++)

✓ Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 6

Test Case 2
TOKEN COUNT 210

Test Case 3
NLOC 35

● hash5.c

```
1  #include <stdio.h>
2  #include <string.h>
3
4  int main(void) {
5      char s[1005];
6      if (!fgets(s, sizeof s, stdin)) return 0;
7      int l = strlen(s);
8      if (l && s[l-1] == '\n') s[--l] = '\0';
9      int cnt[256] = {0};
10     int i;
11     for(i=0;i<l;i++) cnt[(unsigned char)s[i]]++;
12     int bestc = -1;
13     for (int c = 0; c < 256; c++)
14         if (cnt[c] > 0 && (bestc < 0 || cnt[c] > cnt[bestc])) bestc = c;
15     printf("%c %d\n", bestc, cnt[bestc]);
16     return 0;
17 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
puppy is a dog!!!!!!!!!!!!
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>hash5.exe
! 12
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
leana is a singer!!!!!!!
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>hash5.exe
! 7
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```


DS	Session	Hashing	Question Information	Level 1	Challenge 94
----	---------	---------	----------------------	---------	--------------

Problem Description:
Each element may be shifted by $k \cdot M$ ($|k| \leq Q$); print the highest achievable frequency of a single value.

Input Format:

M; Q; N; array.

Output Format:

Single integer.

✓ Logical Test Cases

Test Case 1
INPUT (STDIN) 1 1 4 1 2 3 4 EXPECTED OUTPUT 3

Test Case 2
INPUT (STDIN) 1 1 5 11 12 31 41 51 EXPECTED OUTPUT 2

✓ Mandatory Test Cases

Test Case 1
KEYWORD while(q--)

✓ Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 4

Test Case 2
TOKEN COUNT 300

Test Case 3
NLOC 45

● hash6.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  typedef struct { long long v; int c; } Ent;
5  Ent tab[400005];
6  int tn;
7
8  int *add(long long v) {
9      for (int i = 0; i < tn; i++) if (tab[i].v == v) return &tab[i].c;
10     tab[tn].v = v; tab[tn].c = 0;
11     return &tab[tn++].c;
12 }
13
14 int main(void) {
15     long long M, Q;
16     int n;
17     if (scanf("%lld %lld %d", &M, &Q, &n) != 3) return 0;
18     long long a[10005];
19     for (int i = 0; i < n; i++) scanf("%lld", &a[i]);
20     int best = 0;
21     for (int i = 0; i < n; i++)
22         for (long long k = -Q; k <= Q; k++) {
23             int *c = add(a[i] + k * M);
24             (*c)++;
25             if (*c > best) best = *c;
26         }
27     printf("%d\n", best);
28     return 0;
29 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
1
```

```
1
```

```
4
```

```
1 2 3 4
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>hash6.exe
```

```
3
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
1
```

```
1
```

```
5
```

```
11 12 31 41 51
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>hash6.exe
```

```
2
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Hashing	Question Information	Level 1	Challenge 7
----	---------	---------	----------------------	---------	-------------

Problem Description:
Over all contiguous subarrays compute the maximum sub-array sum within each; print the sum of the unique values.

Input Format:
N then N integers.
Output Format:
Single integer.

Logical Test Cases

Test Case 1
INPUT (STDIN) 4 5 -2 7 -3 EXPECTED OUTPUT 17

Test Case 2
INPUT (STDIN) 15 2 3 4 5 6 7 3 4 2 7 6 8 6 4 9 EXPECTED OUTPUT 2021

Mandatory Test Cases

Test Case 1
KEYWORD int NA[N];

Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 3

Test Case 2
TOKEN COUNT 555

Test Case 3
NLOC 73

● hash7.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  static int cmp(const void *a, const void *b) { long long x = *(const long long *)a, y = *(const long long *)b; return (x > y) - (x < y); }
5
6  int main(void) {
7      int N;
8      if (scanf("%d", &N) != 1) return 0;
9      int NA[N];
10     for (int i = 0; i < N; i++) scanf("%d", &NA[i]);
11     int cap = N * (N + 1) / 2;
12     long long *vals = malloc(sizeof(long long) * cap);
13     int vn = 0;
14     for (int i = 0; i < N; i++) {
15         long long cur = 0, best = -9e18;
16         for (int j = i; j < N; j++) {
17             cur = (cur > 0 ? cur : 0) + NA[j];
18             if (cur > best) best = cur;
19             vals[vn++] = best;
20         }
21     }
22     qsort(vals, vn, sizeof(long long), cmp);
23     long long sum = 0;
24     for (int i = 0; i < vn; i++) if (i == 0 || vals[i] != vals[i - 1]) sum += vals[i];
25     printf("%lld\n", sum);
26     return 0;
27 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
4
```

```
5 -2 7 -3
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>hash7.exe
```

```
17
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
15
```

```
2 3 4 5 6 7 3 4 2 7 6 8 6 4 9
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>hash7.exe
```

```
2021
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Hashing	Question Information	Level 1	Challenge 98
----	---------	---------	----------------------	---------	--------------

Problem Description:
 Array initially zero; '1 k' sets A[k] = -1; '2 y' prints the smallest index >= y with value -1, else -1.

Input Format:
 n q then q queries.

Output Format:
 One line per type-2 query.

✓ Logical Test Cases

Test Case 1
INPUT (STDIN) 5 5 2 3 1 2 2 1 2 3 2 2 EXPECTED OUTPUT -1 2 -1 2

Test Case 2
INPUT (STDIN) 5 5 2 1 1 2 2 2 1 2 2 3 EXPECTED OUTPUT -1 2 -1

✓ Mandatory Test Cases

Test Case 1
KEYWORD while(q--)

DS	Session	Hashing	Question Information	Level 1	Challenge 98
✓ Complexity Test Cases					
Test Case 1		Test Case 2		Test Case 3	
CYCLOMATIC COMPLEXITY		TOKEN COUNT		NLOC	
2		300		40	

● hash8.c

```
1  #include <stdio.h>
2
3  int A[1000006];
4
5  int main(void) {
6      int n, q;
7      if (scanf("%d %d", &n, &q) != 2) return 0;
8      for (int i = 1; i <= n; i++) A[i] = 0;
9      while(q--) {
10         int t, x;
11         scanf("%d %d", &t, &x);
12         if (t == 1) A[x] = -1;
13         else {
14             int ans = -1;
15             for (int i = x; i <= n; i++) if (A[i] == -1) { ans = i; break; }
16             printf("%d\n", ans);
17         }
18     }
19     return 0;
20 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
5 5
```

```
2 3
```

```
1 2
```

```
2 1
```

```
2 3
```

```
2 2
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>hash8.exe
```

```
-1
```

```
2
```

```
-1
```

```
2
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
5 5
```

```
2 1
```

```
1 2
```

```
2 2
```

```
1 2
```

```
2 3
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>hash8.exe
```

```
-1
```

```
2
```

```
-1
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Hashing	Question Information	Level 1	Challenge 9
----	---------	---------	----------------------	---------	-------------

Problem Description:
V(A_i) = number of divisors; count unordered pairs with equal V.

Input Format:
N then N integers.

Output Format:
Single integer.

Logical Test Cases

Test Case 1
INPUT (STDIN) 3 2 3 4 EXPECTED OUTPUT 1

Test Case 2
INPUT (STDIN) 10 2 3 4 2 7 6 8 6 4 9 EXPECTED OUTPUT 12

Mandatory Test Cases

Test Case 1
KEYWORD while(--N)

Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 8

Test Case 2
TOKEN COUNT 345

Test Case 3
NLOC 44

● hash9.c

```
1  #include <stdio.h>
2
3  long long dc[10000006];
4  long long cnt[200];
5
6  int main(void) {
7      for (int i = 1; i <= 10000000; i++)
8          for (int j = i; j <= 10000000; j += i) dc[j]++;
9      int N;
10     if (scanf("%d", &N) != 1) return 0;
11     while(N--) {
12         int v;
13         scanf("%d", &v);
14         cnt[dc[v]]++;
15     }
16     long long pairs = 0;
17     for (int i = 0; i < 200; i++) pairs += cnt[i] * (cnt[i] - 1) / 2;
18     printf("%lld\n", pairs);
19     return 0;
20 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
3
2 3 4
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>hash9.exe
1
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
10
2 3 4 2 7 6 8 6 4 9
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>hash9.exe
12
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Hashing	Question Information	Level 1	Challenge
----	---------	---------	----------------------	---------	-----------

Problem Description:
Count distinct index triplets (i,j,k), $i < j < k$, whose values sum to a multiple of the mythical constant M (remainder-class counting).

Input Format:
N M; then N integers.
Output Format:
Single integer count.

Logical Test Cases

Test Case 1
INPUT (STDIN) 10 5 1 10 4 3 2 5 0 1 9 5 EXPECTED OUTPUT 26

Test Case 2
INPUT (STDIN) 10 5 11 10 14 31 21 15 10 11 9 51 EXPECTED OUTPUT 31

Mandatory Test Cases

Test Case 1
KEYWORD while(i<k)

Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY 10

Test Case 2
TOKEN COUNT 450

Test Case 3
NLOC 55

● hash10.c

```
1  #include <stdio.h>
2
3  long long cnt[1005];
4
5  int main(void) {
6      int N, M;
7      if (scanf("%d %d", &N, &M) != 2) return 0;
8      for (int i = 0; i < N; i++) {
9          long long v;
10         scanf("%lld", &v);
11         cnt[((v % M) + M) % M]++;
12     }
13     long long ans = 0;
14     int i = 0, k;
15     while(i < M) {
16         for (int j = i; j < M; j++)
17             for (k = j; k < M; k++) {
18                 if ((i + j + k) % M) continue;
19                 if (i == j && j == k) ans += cnt[i] * (cnt[i] - 1) * (cnt[i] - 2) / 6;
20                 else if (i == j) ans += cnt[i] * (cnt[i] - 1) / 2 * cnt[k];
21                 else if (j == k) ans += cnt[i] * cnt[j] * (cnt[j] - 1) / 2;
22                 else ans += cnt[i] * cnt[j] * cnt[k];
23             }
24         i++;
25     }
26     printf("%lld\n", ans);
27     return 0;
28 }
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
10 5
1 10 4 3 2 5 0 1 9 5
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>hash10.exe
26
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
10 5
11 10 14 31 21 15 10 11 9 51
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>hash10.exe
31
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```